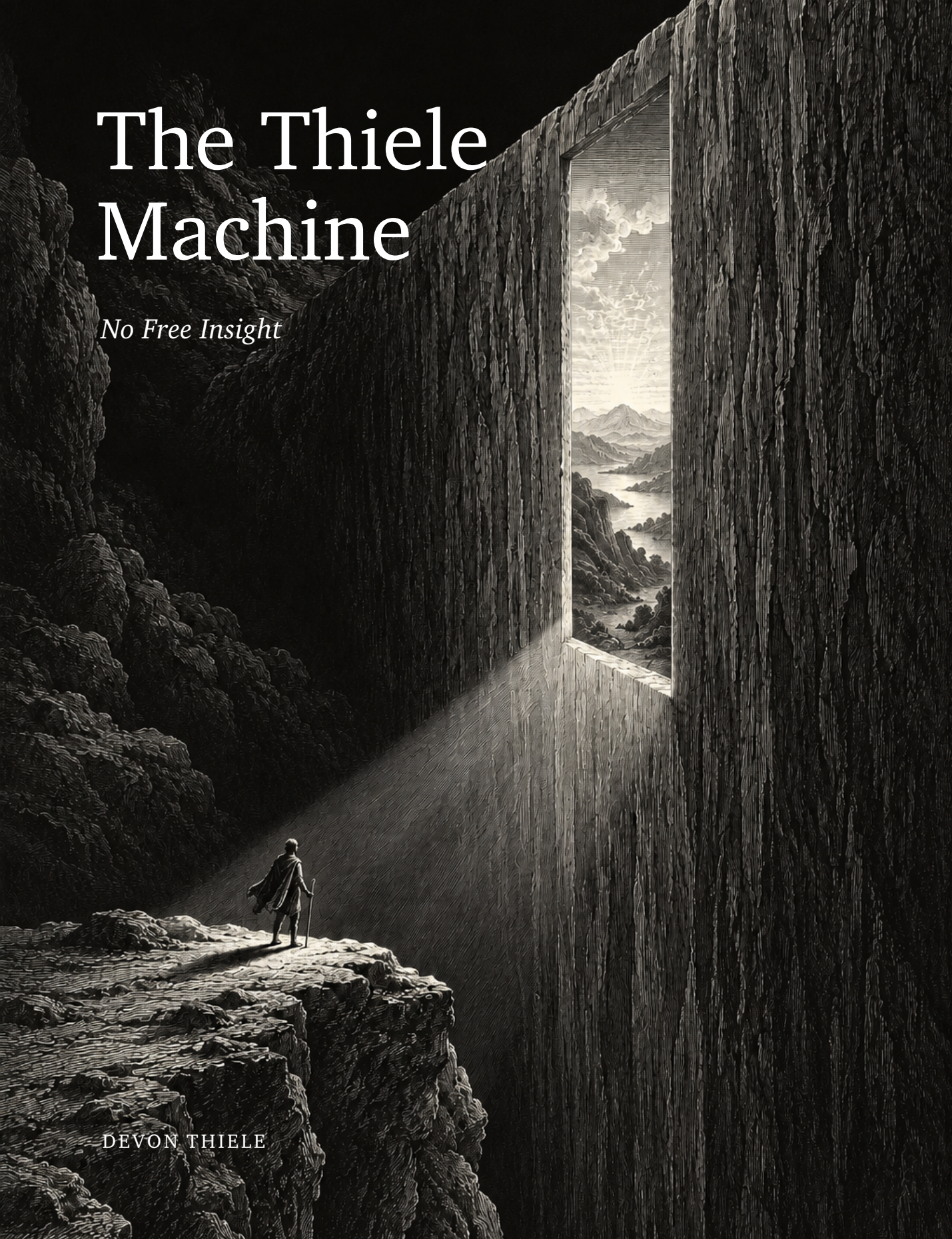
The Thiele Machine

No Free Insight



DEVON THIELE

DEVON THIELE

The Thiele Machine

No Free Insight

The Thiele Machine: No Free Insight

Devon Thiele

Copyright 2025–2026 Devon Thiele. Licensed under Creative Commons Attribution-ShareAlike 4.0 (CC BY-SA 4.0).

The source of this book, its proofs and its build are public: github.com/sethirus/The-Thiele-Machine. Archived at doi.org/10.5281/zenodo.17316437.

Set in Charter. Built from its source with pdfTeX.

The cover and part illustrations were generated with AI in the engraving tradition of Gustave Doré. They are new illustrations, not reproductions of his work.

For Julie, and for my dad.

Julie,

Three years ago, a few months into the rendering engine, I came out of my office and told you I thought I was onto something. You said okay. You didn't ask what it was, or what it was for, or when it would be done, and you didn't ask me for anything I could have shown you, because there was nothing to show. There was a program that had done something it shouldn't have been able to do, and a man who didn't yet know what a Turing machine was.

You gave me the three years anyway. The nights after the lot closed. The weekends. The hours I was in the house and not in the house, because my head was in a wall I couldn't get past. Every dead end, and there were more of those than there are pages here. You never once asked me to stop, and you never once said it was fine when it wasn't. You just kept giving me the next evening.

This book is what I did with them. It's the one thing I've built from nothing that I'm sure of, because I threw out every piece of it I couldn't prove, and what's left holds. I know that isn't what you gave the years for. You gave them for me. But it's what I have to show, and I wanted it to be worth showing.

Dad,

The name on the cover was yours first. I asked you before I used it and you said go ahead. You and Mom taught me that nothing is true just because somebody says it is. This book is that lesson, hundreds of pages long. I put our name on it because I wanted one thing in the world with that name on it that nobody handed me, that I couldn't have faked, and that I can stand behind in front of anyone. Here it is.

Julie carries the name now too. So the machine has it third. This is for the two of you who had it first.

Contents

What this is 12

1 Any machine that steps 15
- 1.1 Is the pencil thing a computer? 15
- 1.2 How far does that go? 16
- 1.3 Try to break it 17
- 1.4 Now ask it about keeping 18
- 1.5 The machine that stops 18
- 1.6 What Turing actually proved 19

2 Running out of room 21
- 2.1 Four people, three chairs 21
- 2.2 This is very old 22
- 2.3 The hotel that never runs out 24
- 2.4 Cut the strip 25
- 2.5 How long can a finite machine hold off? 26
- 2.6 What's being counted 27

3 Why charge for it 28
- 3.1 A rule you can refuse 28
- 3.2 The first door: endless paper 29
- 3.3 The second door: let it come back down 29
- 3.4 The third door: don't charge for it 30
- 3.5 How much 31
- 3.6 Found, assumed, chosen 32

4 The smallest one 34
- 4.1 Clay, seals, and seven witnesses 34
- 4.2 Two columns and a stamp 35
- 4.3 Closer to now 36
- 4.4 Five jobs 37
- 4.5 Two boxes and a coin 38
- 4.6 What the tally is and isn't 39
- 4.7 Any machine can wear one 40
- 4.8 Which question gets charged? 40
- 4.9 The cheapest price list 41

5 More settled 42
- 5.1 A row of cups 42

5.2 Naming what's on the table 44
5.3 The toll on the whole direction 45
5.4 One cup isn't enough 46
5.5 Somebody got here first, from the side 47
5.6 Why crossing a line has to cost 47

6 Looking isn't keeping 49
6.1 A small being at a small door 49
6.2 Bennett moves where the cost is 50
6.3 The demon on a sheet of paper 51
6.4 Two rows of dots 52
6.5 Who holds the pen 53
6.6 Learning after the first look 54
6.7 Finishing together 54
6.8 The certainty count 55
6.9 Who gets charged is partly bookkeeping 55
6.10 Somebody got there first, goddamn it 56
6.11 What the toll is for, said at last 57

7 Looking through a window 59
7.1 Two boxes under a camera 59
7.2 A cave, and the people who got good at it 59
7.3 The shadow of a cube 60
7.4 The whole fact 61
7.5 When does one exist? 61
7.6 People who found this on their own 62
7.7 Notes passed across a room 63
7.8 Slips handed out at desks 64
7.9 Moves that keep to the tabletop 65
7.10 The clerk with two photographs 66
7.11 Look before the concrete covers it 67
7.12 Three ways past the window 67
7.13 What gets through 70

8 Established 71
8.1 Two computers that end up the same 71
8.2 Look harder 72
8.3 So keep the trace 72
8.4 More boxes to tick 74
8.5 Computed, stored, established 75
8.6 The clerk who forgets everything 75
8.7 Information doesn't have it either 76

8.8 Statues that walk off 77
8.9 What "could have failed" means 78
8.10 Computing already had a word for it 79
8.11 The two computers, put back together 81
8.12 What putting it in the step buys 82

9 Settled 84
9.1 Settled, whether it's true or not 84
9.2 A proof is an act 86
9.3 Not a switch 87
9.4 How a statement gets established 88
9.5 What settling looks like 90

10 What structure is 92
10.1 A string, a square, and a drowning nobody can place 93
10.2 Shapes poured into nothing 95
10.3 Prove it from what you and I both accept 96
10.4 The statue and the bronze 97
10.5 Where things are, without numbers 97
10.6 Naming a thing by saying when two are the same 98
10.7 Choose the changes, keep what survives 99
10.8 An invariance comes with a conserved quantity 100
10.9 All of mathematics as structures 100
10.10 A set with some named relations 101
10.11 The structure is in the arrows 102
10.12 Structure is the part that compresses 104
10.13 No place for a grouping 107
10.14 Laying them all down 108
10.15 The smallest piece 110
10.16 Buckets 112

11 The counter 115
11.1 A stick, split down the middle 115
11.2 Why it has to climb 116
11.3 The total is the charges and nothing else 118
11.4 Only one honest count 118
11.5 The toll is an old trick with a new name 119
11.6 What the counter is, said once 121

12 What a ticked box lets you lean on 123
12.1 The coin that moved 123
12.2 Plato nearly had it 124
12.3 Three things on hand 125

12.4 Changing your mind about the cups 127
12.5 Count the rivals 129
12.6 Narrower means a stronger test 130
12.7 What you've got 131
12.8 What standing behind a grouping gets you 132

13 What insight is 133
13.1 Sultan and the sticks 133
13.2 It isn't a feeling 134
13.3 What happens when a computation learns 135
13.4 Test, then commit 136
13.5 What the word means 137

14 A box over any machine 138
14.1 A cup and a die 138
14.2 What "answers only to the machine" has to mean 139
14.3 A base that never merges 140
14.4 The clean start 141
14.5 Both rules do work 143
14.6 Arguing with cups 144
14.7 A box that can change its mind 145
14.8 One shape, two choices nobody forces 146
14.9 What leaving the base open buys 147

15 The machine with the record dropped 148
15.1 What's left when you cover the cups 148
15.2 Pricing through a window, with a whole row 149
15.3 What a paid move destroys 149
15.4 What's left over is the ordinary computer 150
15.5 Shortcuts 151

16 What a machine can't decide about itself 152
16.1 Five names and a table 152
16.2 The same move, again and again 153
16.3 Where the diagonal gets misused 156
16.4 Now point it at the box 157
16.5 What this does to the book 158
16.6 Two walls 159

17 Five real systems 160
17.1 Proof-of-stake finality: the price is on going back 161
17.2 Gas: the one that charges a fee at the write 163
17.3 Attestation: a quote about the past 165
17.4 Certificate transparency: visible, unpriced 168

17.5 Proof-carrying code: no toll required 169
17.6 The same question, five ways 170

What this is

I think in shapes, so I'll start with one.

Take a thing. Anything. A coin on a table, a light switch, a page in a notebook. Right now it's one way. It could be another.

Now do something to it. Flip the coin, hit the switch, write on the page. That's a move: a direction that takes the thing from how it is to how it is next.

Here's a computer you can run with a pencil. Write 3 on a scrap of paper. The move is: erase the number and write the next one up. Do it twice. 3, 4, 5. You just added 2 and 3. Nothing happened there but a thing, a move, and the next thing. Hold on to that picture; whether it covers every computer is something I'll have to earn, and I will, later, with you watching.

Here's the part I couldn't let go of. Some moves just shuffle. Flip the coin and you can flip it back, and nothing is different for good. But some moves make a difference that stays. Write a word in pen. Blow a fuse. Check an answer and mark it checked. After that move something is the case that wasn't before, and it stays the case.

I figured there had to be a cost in that step. Not in the shuffling. In the step where something goes from not settled to settled and stays there. That was a hunch, so test it on the pencil computer.

Tell it when to stop: count up, and when you reach 5, stop. Ask one question of each number: is it done? 3, no. 4, no. 5, yes. And once it says yes it keeps saying yes, because the machine has stopped.

A machine that has stopped is still somewhere on the next tick. It stays where it is: 5 goes to 5. Draw the arrows. 4 goes to 5. 5 goes to 5. Two different starting places, one landing place. Standing on 5, you can't tell whether you just got there or have been there all along.

Could you dodge that? Try letting the stopped machine rock back and forth instead of sitting still: 5-tick, 5-tock, 5-tick, both of them saying done. Draw the arrows. 4 goes to 5-tick. 5-tock goes to 5-tick. Two arrows land on 5-tick, the spot where you came in. Rocking only moved the place where they meet. The one way out is to never land anywhere twice: make every tick write something new, a tally mark, the time, anything, so the paper never repeats. That works as long as there's paper. On a finite sheet you run out, and two arrows land on one spot again.

So whenever this machine settles its answer and keeps it, something gives. Either two states become one, or it uses up fresh paper on every tick for as long as it runs.

That part isn't a hunch. You drew it. You drew it for one small counter; the book proves it for any machine that goes from one state to a next, and shows you where.

What I do with it is a choice, and I'll say so: I charge for the move where the answer goes from not done to done. Charging for it is up to me. That something has to give once it makes that move isn't. The two arrows don't always meet on that very move. Give done two spots, 5a and 5b: 4 goes to 5a, 5a goes to 5b, and 5b stays put. Now the arrows from 5a and 5b meet on 5b, a tick after the answer settled. But after the move from not done to done, there's no avoiding it: two arrows meet somewhere, or the paper never repeats.

Two more things have to be pinned down before the rule means anything, and you can see both with what's already on the table.

First, take the coin again. Flip it, flip it back, forever, free. Ask whether it's settled. It never is. A rule that says "a move that settles something has to cost" never asks the coin for anything, because the coin never settles anything. So the rule alone would let a coin that does nothing count. The machine has to actually settle something, at least once.

Second, suppose a later move could pay you back. The move that settles charges you one, the next move hands you one back, and the whole run comes out free. So prices can be nothing or something, never a refund.

One last thing before the whole definition. Settled doesn't have to be just yes or no. Take a checklist with three boxes. None ticked, one, two, all three: each tick settles it a little more, and ticks don't come off. "Done or not" is the smallest case, a checklist with one box. So when I say "more settled" from here on, picture the checklist.

So here's the Thiele Machine, all of it. A thing. Moves that take it to the next thing. A way to tell how settled the thing is, where settled never goes back to unsettled. Some run, from a start you pick, that actually settles it more than it was. A price on every move, nothing or more, never a refund. And one rule: a move that settles something more than it was costs something.

That's the whole definition. The rest of the book works out what follows from it, one step at a time, and tries to break it.

Now look back at the pencil computer with a stop in it. That's Turing's machine, in miniature. Turing's machine, from his 1936 paper, is a tape and a table of rules: from where it is, the table says where it goes next. A thing and moves. It already has something that settles and stays settled: whether it has stopped, the same question you asked of 3, 4 and 5. What Turing never put in is a price on the move that stops it. Put one in, and a Turing machine that ever stops fits every line above. So this isn't a rival to Turing's machine, and it computes nothing his can't. His machine is one kind of Thiele machine, with the price left out.

Now try this. Give two people the same messy list of numbers. The first sorts it, then goes back down it checking that every number is no bigger than the next. The second copies a sorted list off somebody else's sheet. Now look at the two sheets. Same numbers, same order. Look as hard as you like: nothing on either sheet tells you which one was checked. The difference was real, and neither sheet kept it. It's the same thing you saw standing on 5: you can see where you are, and not how you got there.

So put a box on each sheet that says checked. A plain Thiele machine charges for ticking the box, but nothing in the rules above stops the copier from paying and ticking it anyway. Paying for the tick doesn't make it true. So add one more condition: the box can only be ticked right after a check of that very list passes. Now the copier's tick is refused, and a ticked box tells you a check of that list passed. I call a box like that established. It's a condition you put on the machine. The bare definition doesn't hand it to you.

It's called the Thiele Machine. The afterword says why.

CHAPTER 1

Any machine that steps

When I started all this I didn't know what a Turing machine was. I knew it had something to do with computers, and that's it. I had a program that had done something I didn't expect, and a hunch, and no words for either. Most of the words in this book I found the same way: I'd hit a wall, go looking for whoever had been there before me, and not move on until I could watch the idea actually work. This chapter is the first of those trips. I'll take you on it the way I took it.

The question I had was the one I left you with on page one. I'd been calling that pencil-and-scrap-of-paper thing a computer. Is it? Or did I just make it small enough to look like one?

1.1 Is the pencil thing a computer?

Start with what the word used to mean. Before about 1940 a computer wasn't a machine. It was a job. In the late 1800s the Harvard Observatory hired rooms full of women to do the arithmetic of the sky, cataloguing stars by hand for a fraction of what a man was paid. Some of them found things nobody else had; Henrietta Leavitt worked out a way to tell how far away a star is. (I got that from Dava Sobel's history of those women and from the historians at the American Institute of Physics.) Rooms like that worked out artillery tables in both world wars; that one's from David Alan Grier's history of human computers.

So the pencil thing wasn't a toy version of a computer. For a long time it was the only kind there was. Good start, but that's history, not proof. A person with a pencil can do a lot of things that aren't following rules. I wanted to know what was left if you took the person away and kept only the rules.

That's exactly the question Alan Turing asked, in a paper in 1936. Going in, all I knew was that it had something to do with computers. What's in there is a clerk. He wrote down, as plainly as he could, everything a person sitting at a desk with a pencil does when they follow a procedure, and threw out everything else. What was left had four parts, and you can build it on the table in front of you right now.

Draw a strip of three squares. That's the tape. Write 1 in the first square, 1 in the second, leave the third blank.

Put a coin on the first square. That's where the clerk is looking. Turing called it the head.

On a scrap of paper write the letter A. That's which step of the procedure the clerk is on. Turing called it the state.

Now the rules. Write these two on a card:

> If you're on A and looking at a 1: leave it, move the coin one square right, stay on A.
>
> If you're on A and looking at a blank: write a 1 there, don't move, and change the letter to STOP.

That's the whole machine: a tape, a head, a state, and a card of rules. Now run it. Each tick, read the card and do what it says. After each tick, write down everything about the table: what's on each square, where the coin is, and the letter. I'll put brackets around the square the coin is on.

[1]	1	␣	A
1	**[1]**	␣	A
1	1	**[␣]**	A
1	1	**[1]**	STOP

It started with two 1s and ended with three. It added one. That's the same thing the pencil computer did when it went from 4 to 5.

Now look at what you wrote down on each line. The whole tape, where the coin is, and the letter. Is there anything about this machine at that moment that isn't on the line? Try to think of something. The rules don't change. The tape, the coin and the letter are all there is. So one line is the whole thing, and the card takes one line to the next line. Thing, move, next thing.

So the pencil computer wasn't a cheat. Turing's machine is the pencil computer with a bigger piece of paper.

1.2 How far does that go?

It would be nice if the answer were "every computer, done." It isn't quite. Push on it and it came apart into three different claims that people tend to run together. Here they are, and what each one stands on.

The first one is the one we just did. Every machine you can write down as a whole state and a rule for the next state has the shape from page one. You checked that yourself for Turing's machine: one line, one card, next line.

The second is that the other ways people have written computation down, with registers instead of a tape, or with functions handed to functions (that's Alonzo Church's lambda calculus, from the same year as Turing's paper), compute exactly the same things Turing's machine does. I didn't prove that, and I won't pretend I did. Other people proved it, one pairing at a time; Turing himself sketched the

match with Church's version in an appendix to his 1936 paper. What you can check yourself is that each of those is also a whole state and a rule for the next one, the same way you just did with the coin.

The third is the big one, and here's where I had to be careful. Does this cover everything a person could ever do by following rules? Does it cover every machine anybody could ever build out of real stuff? Those turned out to be claims nobody can prove, and for a good reason. "Everything a person could do by following rules" isn't a precise enough thing to prove anything about. That claim has a name, the Church-Turing thesis. "Every machine you could build" is a claim about physics, and the only way to knock it down would be to build one that beats Turing's, which nobody has done. Neither is a theorem, and in this book neither will get treated like one.

So the debt from page one comes out like this. For every machine you can write down as a whole state and a rule for the next state, paid: you just checked it. For every machine anybody could ever build, that's a claim about the world, it's labeled as a claim, and I'm not going to lean on it.

1.3 Try to break it

Now the question that actually mattered to me. Page one showed that when the counter settled on 5, two arrows had to meet, unless it used fresh paper forever. That was one little counter. Is it true of every machine with this shape?

Don't take my word for it. Get a sheet of paper and do this.

Draw some dots. Ten, twenty, as many as you like. Those are states. From every dot, draw exactly one arrow to some dot. It can point anywhere, even back at the dot it starts from. That's the one rule: every state has exactly one next state.

Now pick a dot to start on and follow the arrows. Keep a list of every dot you've been on.

Do it a few times, with different drawings and different starts. Watch what happens at each step. You'll find that each step does one of three things.

It lands on a dot that isn't on your list yet. A new one.

It lands on the dot you started from. Now you're going around in a circle, and since every dot has only one arrow out, you'll go around that same circle forever.

Or it lands on some other dot that's already on your list. Look at the first time that happens. The first time you got to that dot, you came in on an arrow from the dot right before it on your list. Now you've come in again, on an arrow from the dot you were just on. Those have to be two different dots, because until this step you hadn't been on any dot twice. Two arrows, from two different dots, landing on one. That's a merge, the same thing as 4 and 5 both going to 5. And from there you're on a path you've walked before, so you walk it again, forever.

Now try to make it do a fourth thing. Really try. Draw whatever you want.

Here's why you can't. Wherever the next arrow points, that dot is either on your list or it isn't. If it isn't, it's new. If it is, it's either the start or it isn't the start. There's no other place for the arrow to point. Three cases, and they cover everything.

And notice what you never had to know: how many dots there were. Ten dots or a billion or an endless sheet, it's the same three cases. What you did need was that every dot has exactly one arrow out. Hold on to that, because it's about to matter.

1.4 Now ask it about keeping

Go back to your dots. Color the dot you start on white. Pick some other dots and color them black, and make one rule about the coloring: no arrow is allowed to go from a black dot to a white one. Once you're on black, you stay on black. That's "settled stays settled." White is not done, black is done.

Follow a run from your white start until it lands on black. Now which of the three things can happen after that?

It can't circle back to the start. The start is white, and you can never get back to white once you're on black.

So it's one of the other two. Either it keeps landing on new dots forever, or two arrows meet somewhere.

Try to draw a counterexample: a run that goes from white to black, stays black, never lands on a new dot forever, and never has two arrows meet. You'll be drawing for a while. There isn't one, and the three cases above are the reason.

That's what page one promised: "the book proves it for any machine that goes from one state to a next." You drew it for one counter there. Now you've drawn it for any machine at all, any number of dots.

What it doesn't tell you is anything about heat. Whether two arrows meeting gives off heat in a real device is a question about the device. People argue about it, and they get their own chapter. This is dots and arrows and nothing else.

1.5 The machine that stops

Here's where the "exactly one arrow out" part bites.

Go back to the tape and the coin. The last line was 1 1 [1], STOP. What does the card say to do next? Nothing. There's no rule for STOP. The machine just stops. In dots and arrows: the STOP dot has no arrow out.

That breaks the one thing the three cases needed. So try the keeping question on a machine that stops. Draw two dots. "Working" is white. "Stopped" is black. One arrow, from working to stopped, and no arrow out of stopped.

Follow it. White, then black, then nowhere. Did two arrows meet? No, there's only one arrow. Did it use up new dots forever? No, it stopped. The answer settled and nothing gave.

Now ask it the obvious question: what does a stopped machine do on the next tick? Not "nothing happens," because something is still there. The usual answer is that it stays where it is. So give stopped an arrow back to itself. Now look again. Working points to stopped. Stopped points to stopped. Two arrows, one dot. The merge is back.

What if you don't want it to stay put? Point stopped at a new dot, "parked," and point parked at itself. Now stopped and parked both point at parked. Two arrows, one dot again. The merge moved one dot over. It didn't go away. That's the rocking machine from page one.

So a machine that stops only dodges this by not answering what happens next. Answer it any way you like, and you're back in the three cases: either two arrows meet, or you draw new dots forever.

1.6 What Turing actually proved

Turing is usually credited with the famous halting problem: that no machine can look at every other machine and say whether it stops. Joel David Hamkins, a set theorist, and Theodor Nenu open their paper on it by questioning "the attribution commonly given to Turing's 1936 paper." Since the stopping machine is right in the middle of this chapter, I went and looked at what he actually wrote, with an AI walking me through the paper and the links it turned up, and it isn't quite what happened.

Turing's own machines in 1936 weren't built to stop at all. The ones he cared about printed out the digits of a number, forever. What he proved no machine could always decide was a question about those: whether a machine keeps printing digits, and whether it ever prints a given symbol. The second one is a few lines away from the halting problem, so in a real sense he had it. But he never wrote it down that way. Stephen Kleene stated it in 1952. Martin Davis seems to be the first to have called it "the halting problem," in 1958. That's from the same paper by Hamkins and Nenu, who went back to Turing's 1936 paper and checked, line by line.

So the machine that stops, the one sitting right in the hole, is Turing's machine the way people later learned to read it. It's the one page one meant when it said Turing's machine already has something that settles and stays settled: whether it has stopped.

Put it together. Read a Turing machine through "has it stopped?" Give the stopped line a next line by letting it stay put. Then the step that stops it is a step

where two arrows meet, which you just drew. That's the step I charge for. Charging for it is still my choice. That something gives there isn't.

The dots in this chapter could go on forever. The next question is what happens when they can't, when the paper runs out.

CHAPTER 2

Running out of room

The last chapter left one door open. A run that settles its answer and keeps it either has two arrows meet somewhere, or it lands on a new dot every step, forever. Endless fresh paper. On a sheet that never ends, that's a real way out.

But nobody has a sheet that never ends. So what happens when the paper is finite?

2.1 Four people, three chairs

Do this one with real chairs, or with coins on a table. It takes about a minute.

Draw a box on the floor and call it the done room. Put three chairs inside it and one chair outside, by the door. Sit four people down, one per chair. The people are the states the machine could be in. The three inside are already done. The one by the door isn't.

One move of the machine moves everybody at once: each person gets up and goes to whatever chair the rule sends them to. There are two rules about where they can go.

Nobody inside may end up outside. Done stays done.

The person by the door has to end up inside. This is the move that settles their answer.

Now try to finish the move with one person in each chair.

You can't. The three inside have to stay inside, so they fill the three inside chairs between them. The person from the door has to come in too, and there's no fourth chair. Somebody has to share. Two people, one chair: two arrows, one landing place.

Try it with more chairs. Ten inside, one by the door. Eleven people have to fit into ten chairs. A thousand inside, one by the door: a thousand and one people, a thousand chairs. It doesn't matter how big the room is, as long as it has a size. The insiders take every inside chair, and the newcomer has nowhere to go but somebody's lap.

Notice who ends up sharing. It doesn't have to be the newcomer. The rule could send the newcomer to an empty-looking chair and send two insiders to the same

chair somewhere else. Page one already showed you that: 4 went to 5a, and the arrows met a tick later on 5b. What's forced is that two arrows meet somewhere in that move. Where they meet depends on the rule.

Two words I'll need from here on. A move is one-to-one when two different starting places always end up in two different places: nobody shares a chair. A move that isn't one-to-one is a merge: two arrows, one landing place.

Now say what the chairs showed in those words. Take any move that keeps done states done and takes one state from not done to done, on a machine with K done states. If the move were one-to-one, the K people already inside would fill all K inside chairs with nobody sharing, and the newcomer would have nowhere to land. So it can't be one-to-one. It merges.

That's the whole argument, and it's the chairs. The only thing finiteness did was make "it fills all K done states" true: K people into K chairs with no sharing leaves no empty chair. That's the one place the size of the room mattered.

2.2 This is very old

The piece of mathematics doing the work there is old, and it isn't mine. Put more things into boxes than there are boxes, and some box gets two. People call it the pigeonhole principle. The earliest use of it I could find is Jean Leurechon in 1622, arguing that somewhere two people must have exactly the same number of hairs on their heads. The first name for it I could find came from Dirichlet, who used it in 1842 and later called it the drawer principle. It's the kind of fact a child accepts and a mathematician spends a whole career finding new uses for.

And the use I just made of it turned out to be old too. I found that out twice, and the second time stung more.

The first time was physics. In 1876 Josef Loschmidt pointed out, against Ludwig Boltzmann's account of how a gas settles down, that the laws of motion run just as well backwards: reverse every molecule and the gas retraces its steps. Twenty years later, in 1896, a young assistant of Max Planck's in Berlin pushed harder. He used Henri Poincaré's recurrence theorem from 1890, which says a system like that comes back as close as you like to where it started, again and again, to argue that nothing in it can climb forever. Give it time and it comes back down. The assistant was Ernst Zermelo, the same Zermelo who would put set theory on firm axioms twelve years later. Boltzmann answered the same year, not very politely, that you'd have to wait so long for a return that nobody would ever see one. (Jos Uffink, who wrote the Stanford Encyclopedia's entry on Boltzmann's physics, calls that reply drenched in sarcasm and bitterness, and calls Zermelo's paper a careful logical argument.)

You can see Zermelo's point on the dots from the last chapter. Draw a finite number of dots where no two arrows ever land on the same dot. Every dot has one

arrow in and one arrow out. Follow any run. Can it ever reach a dot it's been on before, other than the start? No: that would take two arrows into one dot. So since the dots are finite and it can't keep finding new ones, it has to come back to its start. Every run goes around in a loop. Now color the start white and some dots black, done stays done. Can a run go from white to black? If it did, it would have to come back around to its white start, and black can't go back to white. So it can't. A machine where no two arrows ever meet, on a finite sheet, can never settle anything new for good.

That's Zermelo's objection, with the infinities taken out. On a finite set you don't need Poincaré's theorem. And the wait isn't enormous; it's at most as many steps as there are dots. Zermelo was aiming it at the second law of thermodynamics, and settling an answer never came into it. A quantum version came in 1957, when P. Bocchieri and A. Loinger proved that a closed quantum system comes back close to where it started. That's a neighbour, not the same proof: even a small quantum system has endlessly many states, and you'd have to say how a record is read off it.

The second time was computer science, and that's the one that stung. Picture a small machine that reads letters, one move per letter. If every letter just reshuffles its states, no two arrows meeting, people call it a permutation automaton, and the sets of inputs such machines can pick out are called group languages. Jean-Éric Pin's 1992 paper on reversible automata sorts out exactly what those machines can recognise, and in that world the chairs argument is folklore: a machine like that can't recognise "has seen a b at some point," because recognising it would mean a done that stays done. Going quantum doesn't get around it. Alex Brodsky and Nicholas Pippenger showed in 2002 that a quantum version that only measures once, at the end, recognises exactly the same group languages. One that's allowed to measure after every letter, and stop on what it sees, does better, and "has seen a b" is one of the things it picks up. A measurement that can stop the machine is a record that stays. That's the extra thing that lets it in.

Then the closest one, and it's recent. In 2023 Thomas Ouldridge and David Wolpert worked out the physics of a small machine built the way real computers are: the same physical step run over and over. They sorted what such machines recognise into two kinds. If the smallest machine for a job has a letter where two arrows meet, every machine for that job does, and on their derivation running it gives off entropy that no cleverer design can get back. If the smallest machine only reshuffles, they find it can run with no unavoidable entropy at all. One of their worked examples is exactly "has not seen a b," which is "has seen a b" turned inside out. So the counting, and a physical price on it, were in their paper three years before this book. Their price comes out of the physics of the device. Mine, in the next chapter, is a choice I make and say so.

There's a cousin that speaks a different language. Constructor theory is David Deutsch and Chiara Marletto's way of stating physics as which transformations a device that runs in a cycle can and can't bring about, and in it a task is irreversible when it's possible and its reverse isn't. Marletto, Vlatko Vedral and their coauthors in 2022 built an irreversible task out of a machine of quantum bits that swap a little with each other, and ran it with photons. Maria Violaris and Marletto argued the same year that this machine, run in a cycle, can turn a pure quantum bit into a mixed one and never back, even though every piece of it can be reversed, and that this suggests erasing may cost more than the usual bound. The chairs are a small, exact, classical relative of that: a set of states the step can't leave, a task that's possible, and no way back. Theirs is quantum and approximate and aimed at physics. Mine is counted. Deutsch and Marletto's paper on information, from 2015, has another neighbour in its section 8.6: measuring one quantity disturbs another for good, and nothing can undo the disturbance while the result is still being kept. That's a second way of saying that a record that stays makes the step impossible to run backwards.

And Morgan Rogers, in his papers of 2019 and 2021 on what mathematicians call toposes of monoid actions, gives the general home for one more way to see it: if every move can be undone by some other sequence of moves, you can never enter a room you're not allowed to leave, because the undoing sequence would have to take you back out. On a finite set, a move where no two arrows meet can always be undone that way: repeat it enough times and everybody comes home.

So the counting isn't mine. It had homes in physics and in automata long before me, and a physical price on it a few years before me. What I do with it starts in the next chapter.

2.3 The hotel that never runs out

Now the way out. If the chairs run out, give the room more chairs. Endlessly many.

Take a strip of paper that never ends. Each move writes a 1 in the next empty box. Done means "there's at least one 1 on the strip." Check it against everything so far. Done stays done: once there's a 1, there's always a 1. The first move takes the empty strip to done. And no two arrows ever meet: two different strips always become two different strips, because you can always erase the last 1 and get back where you were. So here's a machine that settles something, keeps it, and never merges anything.

It got out of the chairs. It didn't get out of paying. Look at what every move does: it writes in a box nobody has used before. The first move does it, and so does every move after that, forever. That's the other column from the last chapter: fresh paper on every tick.

If the endless strip feels like a cheat, it's a famous one. David Hilbert told a story in a lecture course in the winter of 1924 to 1925 about a hotel with endlessly many rooms, every one of them taken, where a new guest still gets a room. Everybody moves from room n to room n + 1, and room 1 is left empty. Nobody shares. The one-to-one move leaves a gap. That's the strip.

The part worth checking is that Hilbert never published it. The hotel got famous because George Gamow put it in a popular book in 1947, and for a long time it wasn't clear Hilbert had told it at all. The historian of science Helge Kragh first guessed that Gamow made it up. Then Tilman Sauer, who had helped edit a volume of Hilbert's lectures, pointed him to Hilbert's lecture notes, which had just been printed in 2013, and in 2014 Kragh took his guess back in a paper called "The True (?) Story of Hilbert's Infinite Hotel." Keep the question mark. Even Kragh's paper puts the lecture in January 1924 in one place and January 1925 in another. Still, this time a story everybody repeated had a real lecture behind it. I've learned not to count on that.

Bring it back to the chairs. In the closed room the person from the door had nowhere to sit because the chairs ran out. In Hilbert's hotel they never do. So the whole argument was about running out of room, all along.

You can run the chairs backwards to see exactly how much room. Take a move that keeps done states done, settles one new state, and never lands two arrows on one place. Everybody already in the done room moves to a different done chair. The newcomer gets a done chair too, and nobody already inside is moving into it, because that would be two arrows meeting. So everybody already inside got seated in the done room with at least one chair to spare. With n chairs, that's n people into n - 1 chairs, which can't be done. So a move like that needs a done room with no end to it. Give the room any fixed size, and the move merges.

That's Hilbert's front desk run backwards. It's also exactly how Richard Dedekind defined "infinite" in 1888: a collection is infinite when you can match it one-to-one with a part of itself that leaves something out. So taking fresh paper isn't a way out of paying. It's the other way to pay, and only a machine with endlessly many states can keep paying it.

2.4 Cut the strip

Now cut the strip to eight boxes. Same rule: fill the next empty box from the left, never touch the earlier marks. There are nine possible strips, from empty to full. Write them down:

0 marks	`00000000`	next: `10000000`
1 mark	`10000000`	next: `11000000`
⋮	⋮	⋮
7 marks	`11111110`	next: `11111111`
8 marks	`11111111`	next: ?

After eight moves the strip is full. What should the ninth move do? Try each answer.

It stays full: then the strip with seven marks and the full strip both go to the full strip. Two arrows, one landing. The chairs.

It stops: then it has no next state, and that's the stopping hole from the last chapter.

It clears the strip: then done went back to not done.

Every answer gives up one of the things the chairs needed. None of them gets past the chairs.

Keeping the whole history like this is the trick Charles Bennett used in 1973, when he showed that any computation can be done without two arrows ever meeting, if you let it keep a copy of everything it did. Rolf Landauer, back in 1961, had already written that for a machine that runs a program that doesn't stop, the space to hold every step "cannot be there." Keeping the history is a real way to pay. It costs a memory that never fills, and every machine you can buy has a memory that fills.

Bennett wasn't the first there, or the last. Yves Lecerf showed in a short note to the French Academy of Sciences in 1963 that machines with no merging can imitate any Turing machine, and hardly anybody noticed. Tommaso Toffoli in 1980, and Edward Fredkin and Toffoli together in 1982, both then at MIT, built logic gates and circuits that compute anything and lose nothing, by carrying extra inputs in and extra outputs out. Their basic gate wasn't new even then: Landauer's own 1961 paper writes one down, calls it reversible and able to do an AND, and doubts anything like it could be useful. What Toffoli added was the proof that it computes everything. And Kenichi Morita proved in 1996 that even a tiny machine with two counters can compute everything with no two arrows ever meeting. Every one of them keeps what it doesn't forget somewhere: a history, extra outputs, a counter that codes the past. They all pay in fresh paper.

2.5 How long can a finite machine hold off?

One more question, because the 5a and 5b example on page one hinted at it. If the machine has a few spare done states, can it put off the merge for a while? Yes, and you can count exactly how long.

At three o'clock a screen says 42. The program is finished. Leave the camera running. At 3:01 it says 42. At 3:02 it says 42. The screen hasn't changed, but

the box behind it still has a next state every tick. It can stay exactly where it is, or something inside it can move while the screen stays put: picture a wheel turning from "first holding position" to "second holding position."

Say the box has K different ways of holding the answer 42. Start one tick before it finishes. Suppose the run goes through its holding ticks without two arrows ever meeting along the way. Could two of those holding ticks be the exact same state? If they were, follow the arrows backwards from both, one tick at a time. Since no two arrows meet along the way, being on the same state now means they were on the same state a tick ago. Keep walking both back. Eventually the earlier one reaches the tick before it finished, the state that wasn't done yet, while the later one is still on a done state. A state that isn't done would have to be the same state as one that is: impossible. So every holding tick needs its own state, and with K of them, it can last at most K ticks. Within K + 1 ticks of finishing, two arrows meet.

You can build one that lasts exactly that long: the unfinished state, then K holding states in a row, the last one stepping to itself. The answer holds for K ticks with nothing merging, and on the next tick the last two arrows meet. A bigger memory puts off the merge. It doesn't make it go away.

2.6 What's being counted

One thing to keep straight before going on. Whether a move merges depends on what you count as the state. The endless strip keeps its whole history in the state, so nothing ever merges; cut it to eight boxes and the same rule merges. Keep a list of every earlier state alongside the current one, and any machine's moves become one-to-one, because two different histories never look the same; a later chapter proves that. Keep only the state one step back, and it isn't enough: two states that differ only in what came before them still land together.

So "this move merges" is always a fact about a move on a stated set of states. Every merge in this book is counted on the whole state the machine says it has, and nothing outside that state is counted.

What I've got now is counting and nothing else: on a finite machine, settling something for good means two arrows meet. The next chapter is about what I do with that, and which parts of it are my choice.

CHAPTER 3

Why charge for it

Everything so far has been counting, and the counting isn't mine. On a finite machine, settling something for good means two arrows meet somewhere. On an endless one, it can mean fresh paper forever instead. Nobody gets to choose that. You drew it.

Page one also said something I do get to choose: I charge for the move where the answer goes from not done to done. This chapter is about how much of a choice that is. If it's nothing but my preference, then the whole machine is a long way of telling you my favourite colour, and you should treat it that way. So I want to find out what it rests on.

3.1 A rule you can refuse

I'm not going to ask you to believe a price. I'm going to write one down as a rule, in the smallest form I can, and you can accept it or refuse it.

Any move where two arrows meet costs at least one.

That's all it says. It doesn't mention done or not done. It doesn't mention checking or settling. It puts a price on one thing: a move that sends two different states to the same place.

It isn't my rule. Charles Bennett, who spent decades working out what computing has to cost, put it this way in 2003: what has to cost is "the erasure of a bit or the merging of two computation paths." Two arrows meeting is two paths merging. I'm taking his rule as it stands and writing it on the move.

Now put it next to the chairs. The chairs said: on a finite machine, a move that settles something for good is a move where two arrows meet. The rule says: a move where two arrows meet costs at least one. Put those two sentences side by side and read them together. A move that settles something for good costs at least one.

That's the rule from page one, the one I said was my choice. I call it the toll. That was my word for it before I could prove any of this: there had to be a toll, a cost in the transition, in the step itself. And look at where it came from just now. Nobody handed it to the machine. It fell out of the chairs and Bennett's rule together.

So it isn't only my preference. But it isn't free of choices either. Count what went into it. There are three, and you can refuse each one.

The done states have to be finitely many, so the chairs run out.

Done has to stay done.

And two arrows meeting has to cost something.

Accept all three and you don't get to refuse the toll. It comes with them. Refuse any one, and the toll goes back to being my choice. So the question "did I have to pick it?" has turned into three smaller questions, and those are better things to argue about. For each one there's a machine that walks out through that door, and you should see each one, because a rule you can't picture breaking is a rule you're taking on faith.

3.2 The first door: endless paper

You've already seen this machine. It's the strip from the last chapter that never runs out. Each move writes a 1 in the next empty box. Done stays done, the first move settles it, and no two arrows ever meet.

So Bennett's rule never charges it anything. Two arrows never meet, so there's nothing to charge. The strip settles its answer at no cost on the price list.

But it didn't get out of paying. It pays in fresh paper, on every move, forever. A price list that only charges for arrows meeting just isn't looking at that column. And on any real machine, the paper runs out, and the chairs come back.

3.3 The second door: let it come back down

Take one coin. The only move is: turn it over. Heads is done, tails is not done.

Draw the arrows. Tails goes to heads. Heads goes to tails. Two dots, two arrows, and they never meet. So Bennett's rule never charges it.

And it does settle something: tails goes to heads, not done to done. At no cost.

What it broke is the second condition. The same move that took tails to heads takes heads back to tails. Done doesn't stay done. So it's not keeping anything. It's flipping a switch back and forth, and the page-one definition already throws it out: a Thiele machine's done never goes back.

Here's the part of this I found sharpest. Forget about done staying done entirely, and keep only the finite chairs. Take any move that takes some state from not done to done. Now go through the done states, one at a time, and ask: does this move send any of them back to not done?

If it does, you've found the coin: the move takes something back somewhere.

If it doesn't, then this move keeps every done state done. That's all the chairs needed, for this one move. So two arrows meet.

There's no third answer. On a finite machine, a move that settles something either has two arrows meet, or it takes something back somewhere else. A move that settles for free and never takes anything back doesn't exist.

That's narrower than "every record costs." It's still true, and I'll take it.

One more picture, so you don't think it's tidier than it is. Three cards, T0, T1 and T2, with a star on T1. The move sends T0 to T1, T1 to T2, and T2 to T1. Look at what it does. T0 gains the star: something settled. T1 loses the star: something taken back. And T0 and T2 both land on T1: two arrows meet. So Bennett's rule charges this move, and yet it doesn't keep anything for good. Charged moves and permanent moves aren't the same list. What's true is one direction: on a finite machine, every move that settles something for good is a move Bennett's rule charges.

And there's an exact way to say which moves Bennett's rule charges no matter what the rest of the price list looks like. A move where two arrows meet gets charged by every price list that follows the rule; that's what the rule says. A move where they never meet doesn't have to be: price that one move at zero and every other move at one, and the rule is still followed. So the moves the rule forces you to charge are exactly the moves where two arrows meet. That one holds on any machine, finite or not.

3.4 The third door: don't charge for it

Take a sheet of paper. It's either blank or stamped. One move: stamp it. Blank goes to stamped, and stamped stays stamped.

Check the conditions. Two states, so the chairs run out. Stamped stays stamped. Blank goes to stamped, so something settles. And draw the arrows: blank goes to stamped, stamped goes to stamped. Two arrows meet. Everything the chairs needed is here.

Now set the price of the stamp to zero.

Nothing in counting stops you. The stamp merges two states, and you just decided merges are free. The answer settled, and the bill says zero.

Price the stamp at one instead, and every condition holds at once, so the three can live together. That stamp at one is the smallest Thiele machine there is, and it gets its own chapter. But the third door asks whether they have to live together, and counting can't say so. The only thing that could close this door is a claim about the world: that two arrows meeting really does cost something, out there, in a real device.

The nearest such claim is Rolf Landauer's, from 1961: that forgetting gives off heat. It's measured for a few devices and argued over for everything else, and it gets its own chapter, with the people who doubt it named. I'll say one thing about it now, because it matters here. Bennett's rule as I wrote it charges every move where two

arrows meet, whether or not you already knew which state you were in. Landauer's heat, as that chapter will show, is only owed for what you didn't know. The rule I'm using is the worst case on purpose: a move that has to work whatever it's handed pays for the worst it could do.

So that's where the toll stands. Accept finite chairs, done stays done, and a price on two arrows meeting, and the toll isn't a choice; it's what the three say together. Each of the three is a choice, and I've shown you a machine walking out through each door. The physics that would close the third door is a premise, and I'll always call it one.

3.5 How much

The toll says at least one. Counting can say more, if you let it.

Take four cards and write A, B, C and DONE on them. Pick one without showing me. Now cover its face with a label that says DONE. Whatever you picked, the face now says DONE. Before the label, how many yes-or-no questions would it take me to find out which card you had? Two: "Is it A or B?" and then one more. After the label, I don't need to ask anything, because there's only one thing the face can say. Two questions' worth of difference got squeezed out.

That's four states landing on one. Each question you no longer need halves the number of places you could have been: four to two, two to one. Two halvings.

If you're willing to go one step further than Bennett's rule, and say each unit of cost pays for at most one halving, then that move costs at least two. And two is enough. In general: if m states were already done and the move settles k more, all m + k of them land on the m done ones, and the move has to pay at least as many halvings as it takes to get from m + k down to m. One newcomer next to one done state is one halving: the toll again.

(The label can be peeled off, so the cards only show the squeeze on what the face says. Whether anything physical got erased is a different question, for the heat chapter.)

There's a plainer way to say what the halving rule asks for. Look at one move and find the place where the most arrows land. Call that its worst pile-up. A cost of one allows piles of two. A cost of two allows piles of four. A pile of three already needs two. That's why the m + k states in the done room had to pay at least that much: spread m + k arrows across m chairs as evenly as you like, and the fullest chair still gets at least (m + k) / m of them. So the rule asks every move to pay for the worst it could ever do, whichever state it's actually handed.

That worst case has a home in physics. Philippe Faist, Frédéric Dupuis, Jonathan Oppenheim and Renato Renner worked out in 2015 the least work any logical step needs when it has to succeed every time, on every input, with no averaging. It

comes to exactly that logarithm of the worst pile-up. An AND gate sends three of its four inputs to 0, so it needs about 1.6 halvings' worth, two in whole units. My rule counts the same thing in units of cost. Joules don't come into it.

You can also count the whole run this way. Write N different starting states on cards. Run each card through the rule once, and stack the cards that land on the same state into one pile. Do it again with the piles. Each tick, count how many different states you're down to. The drops add up exactly: what you started with equals everything lost along the way plus what's left at the end. If you end up with fewer than you started with, some tick had two arrows meet. Turn the cards over first, though. Two runs can show the same answer on the front and still be different underneath, and those are different states.

A word about the k in all this. Physicists write kT for heat at a temperature, and when the counting meets the physics in the heat chapter you'll see k with a little B hung under it, for Boltzmann. That letter has a story I didn't expect. Boltzmann's grave in Vienna has $S = k \log W$ carved on it: entropy is a constant times the logarithm of the number of ways the parts could be arranged. The idea is his. But the constant, and the equation written that way, came from Max Planck, around 1900. In his Nobel lecture in 1920 Planck said the constant "is frequently termed Boltzmann's constant, although to the best of my knowledge Boltzmann himself never introduced it." So the most famous constant in this corner of physics wears the name of a man who never wrote it down. Take the constant away and what's left is the logarithm of a count, which is a number of halvings, which is what the cards just gave you. The constant is the exchange rate between halvings and heat. Whether the world honours that rate is the premise.

3.6 Found, assumed, chosen

That's the place to sort everything so far into three piles, because the rest of the book depends on where each thing falls.

I see it like this: I didn't invent a machine. I found one. That walks straight into one of the oldest fights in mathematics, so here's a short trip into it. G. H. Hardy, the Cambridge number theorist who wrote A Mathematician's Apology in 1940, came down on the found side about as hard as anyone ever has: he wrote that mathematical reality lies outside the people who study it, and the theorems they like to call their creations are really notes on what they saw. The German mathematician Leopold Kronecker supposedly split the difference: God made the whole numbers, and everything else is the work of people. Supposedly, because push on that line and it gets thin fast. It comes down through Heinrich Weber, in a memorial piece written after Kronecker died, pointing back to a lecture from 1886, and that's the only source I could find anybody citing. It's a great line with one witness.

None of that settles whether I found the Thiele Machine or made it up, and a proof wouldn't settle it either. So it's a conviction, and I'll call it one. What it changes is how I hold the thing: like something that was already there, which means I can be wrong about its shape and the shape won't care. And "found" only covers what counting forced. Here are the three piles.

Found: what counting forces on any machine.

Every step of a machine with one next state for each state lands somewhere new, circles back to its start, or has two arrows meet, and after it comes back it repeats forever. You drew that in chapter one.

A run that settles something and keeps it can't circle back to its start, so it has two arrows meet or takes fresh states forever. When only finitely many states are done, the move that settles it has two arrows meet. You sat people in chairs for that.

A machine that stops, with no next state, slips past all of it, and gets caught the moment you say what it does on the next tick.

Assumed: what the found pile is taken to stand on in the world.

That two arrows meeting in a real device, when the device really could have been in either state, gives off heat. That's Landauer's principle, and it's an assumption here, never proved, with its critics named in its chapter.

That a real device can only tell apart finitely many states in any bounded space, so its fresh paper runs out. Nobody has measured where that ceiling is.

That the machine starts clean: if done means "this happened," that's only because it started at not done. That start is put in by hand.

Chosen: what I add, each of which could have gone another way.

To put the price inside the step, as part of what a step is.

That done stays done. That's in the definition because I put it there.

The toll: at least one on the move that settles something. With finite chairs and done staying done, it follows from a price on two arrows meeting, and that price is a choice too.

Which thing gets to count as settling, and what the price list charges. Nothing I've shown you forces either one.

Everything after this chapter is built on those three piles, and I'll keep saying which pile each piece comes from.

CHAPTER 4

The smallest one

The last chapter put a price on the move that settles something. Before I build the smallest machine that carries that price, there's a question I had to ask first. People have been making marks that other people are supposed to rely on for a very long time. Did any of them put a price on the moment the mark goes on and stays? I wanted to know what they thought they were doing, and what it cost them. So here's what I could reach, and where the search ran out.

4.1 Clay, seals, and seven witnesses

Start a little over five thousand years ago, in the Near East, with people who counted sheep with little clay tokens. To send a count across a distance without the courier changing it, they put the tokens inside a hollow clay ball, pressed the shapes of the tokens into the outside while the clay was soft, and sealed it. The archaeologist Denise Schmandt-Besserat has spent a career on these, and she puts the practice at around 3300 BC; the ball is called a bulla.

You can do a version of it with a cookie tin. Glue a paper seal across the lid and the rim, let it dry, and then try to open the lid without tearing the seal. The bare lid would come straight off. The seal is what turns opening it into something you can see afterwards. A substitute seal could fool you, so the tin shows that somebody disturbed it, not that nobody could ever fake it. The clay ball does the same: the outside shows what's inside, and opening it breaks it. Keep that idea, because it comes back: a record is worth something only if it can't be quietly undone.

Now law, which is where people get serious about who's allowed to say what happened. The book of Deuteronomy, chapter nineteen: nobody is convicted on the word of one witness; it takes "two or three." The Roman jurist Gaius, writing in the second century, says a will the court will honour is one sealed by seven witnesses. Why seven? Gaius doesn't say. The legal historian Henry Maine, in 1861, had an answer: the older Roman will was a mock sale, done in front of five witnesses, a man holding the scales, and the buyer of the estate, seven people in all, and when the ceremony fell away the seven seals kept the count. Maine's is the only account of that I found, so treat it as his.

Try it. Write “paid five dollars, signed A” on a napkin and ask a friend to forge it. It takes them ten seconds. Now have two other people initial it at the moment of payment and each keep a copy. If whoever reads it later compares those copies and trusts the people holding them, changing your napkin alone gets caught. Stolen copies, forged initials or a reader who never compares them can still beat it. The claim didn’t change. What stands behind it did.

Which brings me to seals, and the medieval historian Michael Clanchy, whose book is called From Memory to Written Record. England between the Norman Conquest and the early 1300s was learning to trust writing. Until then it had trusted people who remembered. The lesson was painful: seals were forged, charters were forged, and historians today have a whole craft of telling real charters from fakes because there are so many fakes. A seal is only as good as whoever holds the stamp. In 2011 a company called DigiNotar, which issued the certificates web browsers trust, found out the same thing at the cost of its existence: a mathematically perfect signature is worth exactly what the custody of the key is worth, and intruders had taken over the machines that signed. It’s the same failure nine hundred years apart, and I find that oddly comforting.

And then the obvious next question: if seals could be forged, what caught the forgeries? It wasn’t the seals. The most famous forged record in European history is the Donation of Constantine, a document in which the emperor Constantine supposedly handed rule over Rome and the western empire to the Pope. It was most likely written in the eighth century, some four hundred years after Constantine died, and it was leaned on for centuries. Around 1440 a scholar named Lorenzo Valla took it apart by its Latin: the language belonged to a much later age, and it used words, “satraps” among them, that no Roman of Constantine’s day would have used for his officials. (Valla wasn’t even first. Nicholas of Cusa, a German churchman who later became a cardinal, had already called it a forgery in 1433. I know that from the reference works; I haven’t read his book. Valla’s is the version people remember, which is a pattern that keeps turning up.)

Look at what kind of check that was. The question wasn’t “is the stamp real.” It was “could these words have been written when they say they were.” A record can pass the first and fail the second. That was my first hint that what I kept calling one thing, a record, was doing more than one job.

4.2 Two columns and a stamp

The one I’d never have guessed mattered is bookkeeping. Around 1300, the ledger of a Florentine company called Farolfi, kept at its branch in Provence, shows the earliest complete example anyone has found of a trick merchants were working out: write every transaction twice, once where the money came from and once where

it went, and the two columns have to balance. The treasurers of the city of Genoa kept their ledger for 1340 that way. And in 1494 a friar named Luca Pacioli printed the method in a mathematics book, and it spread.

Do it for one day. Keep two columns of every dollar that moves, and total them at night. Have a friend secretly change one entry in one column. The totals disagree; you catch it. Now have them change the matching entry in both columns. The totals agree, and you've caught nothing. So that's what double entry actually is: a check for slips, which a careful liar walks straight through.

Did Pacioli see that? I can only get at it secondhand. A scholar named Forrester, in a paper on the University of Strathclyde's accounting-history pages, reads him as reporting that in many places the books were taken to the local consul to be stamped before a single entry went in. The stamp is a witness again, from outside. The two columns are a method. The stamp is somebody else agreeing to stand behind the fact that these are the books. Keep those two apart, because one is a habit and the other is a commitment.

Then I found where this got turned into a motto. The Royal Society, founded in London in 1660 and still running, picked as its motto Nullius in verba: take nobody's word for it. Where did they get it? From somebody's words, naturally. It's half a line of the Roman poet Horace, from a letter to his patron, saying he isn't bound to swear by the words of any master. So the society that takes nobody's word for it took its motto from a poet and cut the line to fit. That's not a contradiction. It's a nice picture of the method: borrow what's useful, say whose it was, and don't swear by it.

The historian Steven Shapin wrote about what that motto meant in practice, and the answer is more interesting than the slogan. Robert Boyle, pumping the air out of a glass globe to see what happens to a candle or a bird, had three problems: the machine, the report, and the witnesses. He solved the third by having gentlemen in the room who'd sign their names, and the second by writing the report so richly that a reader who wasn't there could picture the scene and count as a witness anyway. Shapin calls that virtual witnessing, and he notes, dryly, that the air pump was very expensive, the big science of the 1600s. So there was a cost. It just wasn't a cost of declaring the fact. It was the cost of the instrument and of getting enough people into the room. Hold on to that difference, because the rest of the book hangs on it.

4.3 Closer to now

The rest of the lineage points the same way, so I'll go through it quickly.

Regulators who made stockbrokers keep their records on media that can't be rewritten (the SEC required exactly that, write once and read many, from 1997 until

2022, when it allowed an audit trail instead) were paying for the hardware. The rule put no charge on the act of writing.

Stuart Haber and Scott Stornetta, in 1991, chained time-stamped records together so that changing an old entry breaks every entry after it. Bitcoin's footnotes lead back to them. Try the idea with a running total. Take five entries, 1, 2, 3, 4, 5, and add each to the total so far: 1, 3, 6, 10, 15. Now change the second entry to 3. The totals become 1, 4, 7, 11, 16. The change travels all the way down. Now also lower the third entry to 2, and the last total is 15 again. Simple addition can be fooled by changes that cancel out. A real chain uses a much harder function than addition, and somebody has to keep a trusted copy of the last value to compare against. (Ralph Merkle's hash tree, from his 1979 Stanford thesis, lets one short value stand for a great many records the same way.) What the running total shows you is how each entry depends on the ones before it. It doesn't put a price on writing one.

Proof of work, which Cynthia Dwork and Moni Naor proposed in 1992 to make junk mail expensive, and which Bitcoin later chained, is the one place where a record really does cost something. But what it charges for is a search: how many tries it took to find a lucky number. An empty block and a full one face the same target. The work pays for the search. Nothing extra gets charged for the moment anything becomes settled.

Robert Floyd and Tony Hoare, asking in 1967 and 1969 how you'd prove a program does what it's supposed to, attached claims to points in a program: "whenever you get here, this is true." The claim is backed by a proof, and the proof is paid for somewhere else, off the machine.

The certificates of complexity theory, going back to Stephen Cook in 1971, are answers designed to be quick to check: hand me the certificate and I can confirm the claim fast. Whether they're also hard to find is the famous open question of the field, P versus NP, and nobody knows. Either way, what's bounded is the checking. Nothing is charged for raising a flag.

And from statistics, a ledger that runs the other way. Dean Foster and Robert Stine's alpha-investing, from 2008, runs a long string of tests against a budget of allowed false alarms: every test spends some of the budget, and every test that finds something earns some back. That's a budget for how often you're allowed to be wrong. A reward there doesn't mean the discovery was true.

4.4 Five jobs

Lay them all down together, and what shows through isn't one thing. It's five different jobs the word "record" has been doing, and I'd been letting one word carry all five. Most of them come with a corpse.

Durability: the record can't be quietly undone. The clay ball, the write-once disk and the hash chain do this one. The corpse is the editable log, and the company that kept its debts off the books: Enron hid debt in side companies set up to keep it off the balance sheet, which is how the Justice Department described it in 2002, and its auditor, Arthur Andersen, shredded paper. Andersen was convicted that year, and the Supreme Court threw the conviction out in 2005 over how the jury had been instructed. Notice that this job doesn't need a witness at all. A copy will do, as long as somebody else holds it.

Authenticity: the record came from who it says. Seals, signatures and the consul's stamp do this one. The corpse is the forged charter, and DigiNotar. A perfectly authentic entry can still be erased, and a perfectly durable one can still be a forgery. They're different jobs.

Evidence: enough independent people stand behind the claim that lying gets expensive. Two witnesses, seven seals and Boyle's gentlemen. This one is about how many people put their own risk next to the claim. Paper has nothing to do with it.

Accounting: the record checks itself for slips. The two columns. And the second column isn't an outside witness. It's you, checking you. It catches a dropped digit, and a careful liar gets past it.

Quick checking: a certificate, signature or fingerprint that makes one particular check fast. Checking doesn't have to be cheaper than making, and I once thought it had to. Plenty of records are checked by doing the whole thing over, and that's fine when the thing is small.

Now the thing none of them says. Not one of those jobs puts a price on the moment a record goes on and stays. Proof of work prices a search. Bookkeeping prices a habit. Witnesses and seals price showing up and guarding the stamp. I looked, with everything I had, and that's where it ran out. If somebody priced that moment itself, they're on a shelf I never found.

So I had to try it myself, on the smallest thing I could build.

4.5 Two boxes and a coin

Take a sheet of paper. Draw two boxes and label them NO and YES. NO is not done, YES is done. Put a coin in NO. Draw a line underneath for tally marks; that's where you keep score.

You get one move. Call it the stamp. From NO, the coin slides to YES. From YES, it stays in YES. Every time you make the move, whichever box you're in, you put down one tally mark.

Play. Start in NO, make the move as many times as you like, and look at the tally. Now try to get to YES with no marks at all. You can't, and you can see why: the only way from NO to YES is the stamp, and the stamp makes a mark.

Now cross out the mark in the stamp's rule and play again. You get to YES with no marks, every time. That's how little the mark is. The board doesn't force it. Chapter three is the reason it's there: finite chairs, done stays done, and Bennett's rule. Here it's written on a card.

Draw the arrows for the stamp. NO goes to YES. YES goes to YES. Two arrows, one landing place. It's the same picture as 4 and 5 both going to 5 on page one.

That's a Thiele machine. A thing (where the coin is). A move (the stamp). A way to tell how settled it is (NO or YES), where YES never goes back to NO. A run that actually settles it (start in NO, stamp once). A price on the move (one mark, never a refund). And the toll holds: the move that takes NO to YES costs one.

Is there a smaller one? Try.

One box? Then there's nothing to settle. The coin is always in the same place, so no run ever takes it from not done to done.

Two boxes, then. A move either sends the two boxes to two different boxes, or it sends both to the same one. If it keeps them apart, there are only two ways to do that: leave each where it is, or swap them. Leaving them where they are never settles anything. Swapping them is the coin from the second door: it takes NO to YES, but it also takes YES back to NO, and done has to stay done. So a two-box machine that settles anything has to have a move that sends both boxes to the same box, and since YES has to stay YES, that box is YES. That's the stamp.

So nothing smaller works, and look at what that means. Chapters one and two found that keeping something takes two arrows meeting or endless fresh paper. Here there are only two boxes, so there's no fresh paper to be had, and the arrows meet. The smallest machine there is already has the merge inside it, in plain view. Nobody added it. A fuse that blows, a bead that drops into a dip and stays, a stamp pressed onto a box: each one is two states and one move that sends both to the done side, and each one, with that move priced at one, is this machine.

I'll call it the latch, because that's what it is: something that catches and holds.

4.6 What the tally is and isn't

Why tally marks? Because they're about the oldest receipt there is. The real ones, notched sticks split down the middle, get their own chapter later, fire and all. The tally on your paper is nothing like those. Nobody holds the other half. You keep it, beside the board, and you could rub a mark out and nobody would ever know.

Hold on to that. Of the five jobs, the latch does a small piece of the first one: none of its moves takes YES back to NO. It has no witnesses, no signature, no second column, and nothing in it checks anything. YES means the coin is in YES. That's all it means. Whether a YES can ever be made to mean "this was checked and can be relied on" is a real question, and it's the one the second half of the book is about.

For now the tally is a scorekeeper's receipt: you know what you paid, and nobody else has any reason to believe you.

4.7 Any machine can wear one

Here's something the latch lets you do to any machine at all.

Take any machine, any states, any moves. Pick one state, call it x, that some move lands on. Now build a new machine right beside it: the same states and the same moves, with one extra coin riding along in its own two boxes, NO and YES. The coin starts in NO. The first time a move lands on x, the coin goes to YES, and it never comes back. Price every move at one.

Check it against the definition. The coin never goes back to NO. Start anywhere the move that lands on x can be made, with the coin in NO, and make it: the coin goes to YES. Every move costs one, so the toll holds. It's a Thiele machine.

Now cover the extra coin with your hand and watch the rest. What you see is the machine you started with, move for move. Nothing about it changed. The coin rode along and the old machine never knew.

And with your hand over the coin, you can't tell which box it's in. Let any spot with the coin in NO count as a place to start. Now find a moment where the old machine is sitting on x. Did it just arrive there, which put the coin up? Or did it start there, with the coin still down? Both can happen, and the part you can see looks exactly the same. The coin is the only place the difference lives.

So every machine that has a move at all is what's left of a Thiele machine once you cover the coin. Even a machine with one state and one move that goes nowhere: the move lands on that one state, so the coin goes up, and what you've built is the latch. Turing's machine read through "has it stopped?" doesn't even need the extra coin: chapter one showed it already has a done that stays done, and stopping is already a step where two arrows meet. Both ways, it points the same direction page one did. The machine with the price in it is the general thing. The computer is that machine with the price left out.

4.8 Which question gets charged?

One more puzzle, because it shows why the definition picks one question and charges only that one.

Put a tile on a table and slide it one place to the right. Let that move be free. Now ask a new question: is the tile in the place it just reached? Before the slide, no. After it, yes. If every question had to follow the toll, that slide would have to cost at least one.

Try it with any free move that changes anything. Ask “is the machine where this move just put it?” The answer goes from no to yes. So if the toll applied to every question you could ask, every move that changes anything would cost something, and nothing could ever be free. That’s why the toll belongs to one question, the one the machine says is “how settled,” and not to every question you could think of.

And for that one question, a free move can’t touch it. It can’t take NO to YES, because the toll charges that. It can’t take YES to NO, because done stays done. So a free move leaves the settled question exactly where it found it. That’s the whole reason the free moves and the settling moves come apart so cleanly.

4.9 The cheapest price list

Last thing about the latch. Put three price cards next to a button: zero, one and two. Somewhere in the machine, pressing that button takes NO to YES. Cross out zero; the toll won’t allow it. One works. Two works too. The toll picked the smallest card it could, without forbidding the bigger ones.

Now take a button that never takes NO to YES, from anywhere. Zero works. So go through the buttons: one on each button that can settle something somewhere, zero on each button that never does. That’s the cheapest price list that follows the toll, and there’s only one cheapest. Notice the price stays on the button: the stamp costs one even when you press it on YES and nothing changes. The toll is a floor, not an exact bill.

And it counts. Every time a run goes from NO to YES, that step costs at least one, so a run costs at least as much as the number of times it settled something. On the latch that’s at most once, because YES stays YES. On a switch that can be turned off for free, you could settle it and unsettle it a hundred times, and you’d pay at least a hundred. A latch pays once, and keeps it.

CHAPTER 5

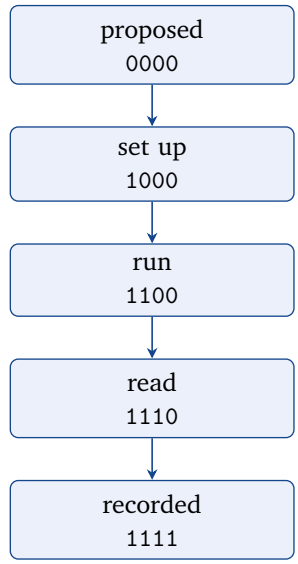
More settled

Up to now everything has had two places: not done and done, NO and YES, the coin in one box or the other. Page one promised more than that. It said settled doesn't have to be yes or no, and to picture a checklist.

That promise matters to me more than it might look. The hunch this whole thing started from was that there's another direction, one ordinary computing doesn't look along. A direction isn't a switch. A switch is just the first line drawn across it. So this chapter takes the switch and finds out what happens when there are many lines.

5.1 A row of cups

Get four cups and a strip of paper. Write five stages of an experiment along the strip, left to right: proposed, set up, run, read, recorded. Agree to do them in that order. Put an upside-down cup between each pair of neighbours. The first cup means "the setup happened," the second "the run happened," the third "the result was read," the fourth "the result was recorded." When you finish a stage, turn its cup right side up. Cups only ever get turned up. None of them ever gets turned back down.



Read down the drawing. Zero means cup down, one means cup up. Four cups give five stages along this run because each new stage turns up one unused cup.

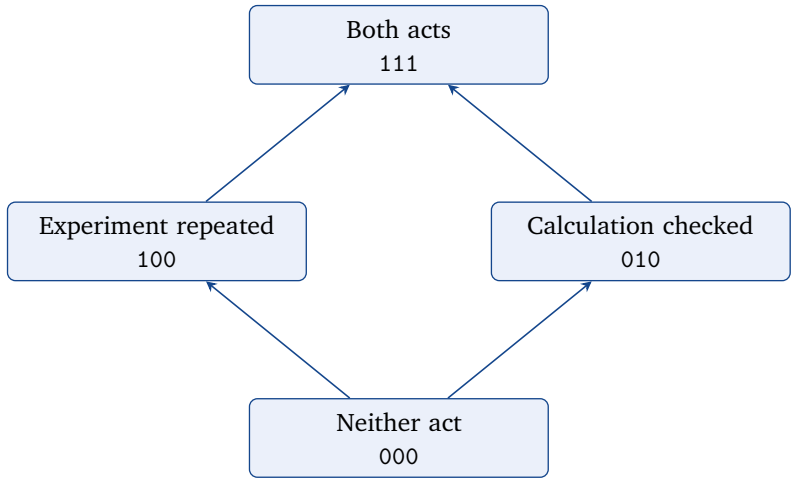
Two things to notice with the cups in front of you.

Each cup on its own is the latch from the last chapter: down or up, and once up, it stays up.

And four cups were enough for five stages. Try it with three. Turn them up one at a time, never back down, and count how many different arrangements you pass through. None up, one up, two up, three up. Four arrangements, and you needed five. You can't do it. Every time the arrangement changes, at least one more cup goes up, and with three cups that can only happen three times. So three cups give you at most four stages, and in general a row of cups that only go up can tell apart at most one more stage than it has cups. A straight line of stages needs one cup fewer than it has stages, and no fewer.

An ordinary binary counter holds a thousand stages in ten bits. That doesn't break anything here: a counter's bits go down as well as up. These cups don't.

Now a harder strip. Some checks don't line up. One person repeats the experiment. Somebody else, separately, works through the calculation. Neither one includes the other. Draw a diamond: "neither" at the bottom, "experiment repeated" on the left, "calculation checked" on the right, "both" at the top. There are two ways up, and the two middle points don't come before or after each other.



Read the cups as repeat, calculation, both. Neither middle point includes the other. The third cup repeats information already supplied by the first two; it is useful for showing thresholds, not needed for storage.

Put a cup on each point except the bottom: one for the repeat, one for the calculation, one for both. Each cup is still a latch. The cups tell you which things have happened. They don't tell you whether the experiment was set up right or the calculation started from the right place. That's a different question, and it comes later.

5.2 Naming what's on the table

Now that you've seen the pieces, here are the names I'll use for them.

The set of places the record can be, whether it's a straight strip or a diamond or something with a hundred points, I'll call the direction.

Each question of the form "has the record reached at least this place?" I'll call a line across it. One line is one cup.

To say "at least this place," you need to be able to compare two places: is this one at or below that one? The strip and the diamond let you see that by looking. Said exactly, it's a test you can put to any two places, and it has to follow two rules. Every place is at or below itself. And if one place is at or below a second, and the second is at or below a third, then the first is at or below the third.

There's a third rule you might expect: if two places are each at or below the other, they're the same place. Every strip and diamond on the table follows it. To see what a rule like that is doing, take it away.

So take two places, left and right, and say each one is at or below the other, without them being the same place. Now there are two cups: "at least left?" and "at least right?" Put the record on left. Both cups are up, because left is at or below

itself and at or below right. Put it on right. Both cups are up again. Look at the cups and you can't tell where the record is.

Now let the record swap between left and right on every step, for free. Is that allowed? It never goes below where it was, since each is at or below the other. No cup ever moves, so nothing ever settles, so the toll never asks for anything. The record moved on every single step, and nothing on the table noticed. It never settles anything either, so it isn't a Thiele machine.

Put the third rule back, and the cups tell you exactly where the record is. Suppose two places leave exactly the same cups up. The cup for the first place is up when the record sits on the first place, so it's up on the second place too, which says the first is at or below the second. The same goes the other way round. Each at or below the other: by the third rule, they're the same place. Without the third rule, the cups can miss moves. Neither is wrong. Dropping the rule just tells the cups not to care about moves between places on the same rung, and they don't.

5.3 The toll on the whole direction

One cup had one rule: the step that turns it up pays. Now there's a row of cups, or a diamond, and one step can turn up several at once. Here's the rule I'd guess first, and it turns out to be the right one: a step that turns up any cup pays at least one. Three cups at once still only has to pay one. The rule asks whether a step turned something up, not how many.

Does that still give you what one cup gave you? Take any run where some cup is up at the end that was down at the start. Somewhere along the run, one step turned that cup up, and that step paid. So the run paid at least one. And every step that turned anything up paid, so the run paid at least once for each of them.

Look at what that argument never used. It didn't care whether the cups sat in a row or a diamond. It didn't care how many cups there were. It didn't even use the fact that cups never come back down: if something could knock a cup back over, the argument still works, because all it needed was the one step where that cup went up.

There's a simple way to tell, from places alone, when a step turns up a cup. Say a step takes the record from place x to place y. The cup at y is up afterwards, because y is at or below itself. Was it down before? It was down exactly when y isn't at or below x. And if y is at or below x, every cup that's up at y was already up at x, so nothing turned up. So a step turns up some cup exactly when it lands somewhere that isn't at or below where it started. That's the toll from page one, written on any direction.

And checking it one cup at a time is the same as checking it on the whole thing. If every cup on its own follows the two-box toll, the whole direction follows the toll, and the other way round.

5.4 One cup isn't enough

Could one cleverer cup do the job of all four? Say you keep one latch and try to read the record's place off the latch together with whatever else the machine shows.

Take the smallest case: a machine with nothing else to show, and a record that climbs through three places, bottom, middle, top. One latch gives you two answers, down or up. Three places, two answers. Some two places get read the same. One cup can't do it.

Try a second guess, closer to how a single cup works. A cup's rule is: the new value is the old value, or the event that turns it up. For a whole row, the obvious copy is: the new place is the lowest place that's at or above both the old place and the event's place. Test it. Take the strip bottom, middle, top, with a cup at middle and a cup at top, and a record that jumps from bottom straight to top in one step, and stays put at middle. With nothing else for the event to look at, the event has to be one fixed place. From bottom, it has to take the record to top, so the event is top. But then from middle, the same rule takes the record to top too, and the record was supposed to stay at middle. The rule breaks.

What works is giving every cup its own event, and letting each cup's event look at where the record already is. A later chapter builds that out over any machine.

How many cups does a direction need? That depends on what you ask the cups to do, so start with the smallest job: tell where the record is, using cups that only ever go up as the record climbs. For that job a cup doesn't have to be "at least this one place." It can go up at several places, as long as climbing never knocks it back down.

A straight strip needs one fewer cup than it has stages; you counted that above. A direction that branches needs at least as many cups as the steps in its longest climb, and at least enough to tell all its places apart.

Put three places above a bottom, none of them above another. The longest climb is one step, but one cup can't tell four places apart, so it takes at least two. Two do it, if you let a cup go up at more than one place: one cup that goes up at the first and third places, one that goes up at the second and third. Every place gets its own pattern. But look at those two cups alone and the third place seems to sit above the first, because every cup that's up at the first is up at the third too. If you also want the cups to show which places sit above which, it takes three: one "at least this place" cup for each of the three.

And sometimes just telling the places apart needs more than both of those. Take two places, each below two others. One step of climb, four places, and two cups still won't do it, because both upper places would have to keep up every cup either lower place has up, and two cups don't leave room for that. It takes three, and showing which sits above which takes four.

5.5 Somebody got here first, from the side

People got here before me, from a direction I didn't expect: programming for many processors at once. Lindsey Kuper and Ryan Newton's LVars, from 2013, are memory cells whose value only climbs, where a program can't ask for the value itself, only whether it has climbed past some point. That's a cup. Two writers running side by side can push the value up in different orders, and a point once passed stays passed. Their programs come out the same every time partly because of which questions the language lets you ask: a program allowed to ask whether the cup is up yet could still see the other writer's timing.

Amin Timany and Lars Birkedal, coming at it from proofs about programs, show in their 2021 paper, as I read it, that any order at all can be carried this way in a logic for proving programs correct, as a value that only climbs and that the running program never reads. That's the closest I've found to somebody else cutting a direction into lines.

The two of them rest on the same shape, a value that only climbs and a question that only asks whether it's past a point, and they use it for opposite jobs. In an LVar the question steers the program. In Timany and Birkedal's logic, the value lives in the proof and the program never looks at it. The shape is theirs first. What I can't find in either is a price on crossing a line.

5.6 Why crossing a line has to cost

Chapters two and three got the toll out of the chairs: on a finite machine, a move that settles something for good lands two states on one. Does that survive a whole row of cups?

The chairs need one yes-or-no question to count with, and now there are many cups. So which cup? The first one I'd reach for is the cup at the place where the record starts. That goes nowhere: the starting state already has that cup up, so there's no newcomer coming in from outside. The cup that works is the one the step turns up, the cup at the place where the record lands. It's up afterwards and it was down before. So put everybody who has that cup up in the done room, and the state the step starts from is the person by the door.

Now it's the chairs again, once for that cup. If the move keeps everybody in the room inside and never lands two of them on one chair, they fill every chair, and there's none left for the newcomer. So one of two things happened: two arrows met, or somebody inside got moved out, which means that cup went back down. If cups never come back down, it has to be the first one. Two arrows meet.

And then Bennett's rule from chapter three does the rest. A move where two arrows meet costs at least one, so a move that turns up a cup costs at least one. The toll on the whole direction rests on the same three things as the toll on one cup: finitely many states with each cup up, cups that never come back down, and a price on two arrows meeting.

Each of those can be knocked out, and each one goes the same way it did for one cup. Let the states be all the whole numbers and let the record count up by one on every step, free: nothing ever merges, the price list has nothing to charge, and every step turns up a new cup. That's the endless strip, paying in fresh paper. Let a two-state record swap back and forth: nothing merges, and the cup it turns up gets knocked back down. That's the coin. Let a two-state record stamp every state up and charge nothing: the arrows meet and nobody asked them to pay. That's the stamp at zero.

So nothing new got smuggled in when one cup became a row, and nothing the one cup needed got left behind. You might expect the diamond to need some extra rule for the two middle cups that don't come before or after each other. It doesn't. Every cup gets its own room of chairs, and the argument in each room is the old one.

CHAPTER 6

Looking isn't keeping

The toll charges the move that settles something for good. But there's another thing a machine does that feels like it ought to cost at least as much, maybe more, and that's finding out. Every time a machine rules out a possibility, it knows more than it did. If two arrows meeting costs something, does learning cost something too?

Most people's gut says yes, and finding out is the expensive part. Most physicists felt that way too, for a long time. It's why one little imaginary creature kept them busy for over a century. This chapter is about where that feeling goes wrong, and where it's right. The short version: looking doesn't have to make two arrows meet. It uses up a blank page instead, and you can hand the page back. Wiping the page to use it again is where two arrows meet. And making a record that stays, one nobody can hand back, is paying for good.

6.1 A small being at a small door

In December 1867 James Clerk Maxwell, who had published his equations for electricity and magnetism two years before, wrote a letter to his friend Peter Tait about a problem he couldn't get rid of. Take a box of gas at one temperature, with a wall down the middle and a tiny door in the wall. Now imagine, he said, a "very observant and neat-fingered being" sitting at the door, who can see the molecules coming. When a fast one comes from the left, it opens the door and lets it through. When a slow one comes from the right, same. Otherwise it keeps the door shut. It never pushes anything. It only opens and closes a frictionless door at the right moments.

After a while the right side is hot and the left side is cold, and you've got a difference you could run an engine off, made out of nothing but looking. That's supposed to be impossible. It's exactly what the second law of thermodynamics says can't happen, and the second law is the one law physicists are least willing to give up. Arthur Eddington wrote in a 1928 book of popular lectures that a theory found to contradict it has nothing for it but to collapse in deepest humiliation.

Maxwell put the being in his textbook in 1871, and it got named Maxwell's demon, though not by Maxwell. William Thomson, later Lord Kelvin, called it a

demon in the journal Nature in 1874. Maxwell, in an undated note to Tait headed "Concerning Demons," tried to take the name back: "Call him no more a demon but a valve." Nobody listened, which is probably for the best, because "Maxwell's valve" wouldn't have lasted a century. But his correction is exactly right, and it's the whole problem: the being does no work. It opens and shuts a door.

So physicists spent the next century trying to find where the being was cheating. The obvious answer pulls hard: the being has to see the molecules, and seeing costs. In 1929 Leo Szilard cut the box down to a single molecule and did the arithmetic. The second law survives, he said, if finding out which side the molecule is on costs at least as much as the being gains. In 1951 the French physicist Léon Brillouin made it concrete: to see a molecule you have to bounce light off it, and, the way the philosopher of physics Jeffrey Bub sums it up, finding out that way costs at least one halving's worth of entropy for every bit. For thirty years that was the answer. Information costs because getting it costs.

But that's a price on one way of looking. It doesn't show that every way of looking has to pay it.

Szilard is worth one more minute, for a reason that has nothing to do with the demon. Four years after the one-molecule engine, in September 1933, by his own later telling, he was waiting at a traffic light on Southampton Row in London. He had just read a newspaper report in which Ernest Rutherford, the physicist who announced the atomic nucleus in 1911, called the hope of getting power out of atoms moonshine. As the light changed, it occurred to him that an element which gave off two neutrons when it swallowed one could keep a reaction going by itself. That's the nuclear chain reaction, thought up crossing a street. It's his memory of himself, and Richard Rhodes's history of the bomb opens with it, in a chapter called "Moonshine," so hold it as the man's own story, not a record made at the time. It changes nothing about the demon. What it adds is a picture of the man: the same mind looked at one molecule in 1929 and one neutron in 1933, and both times the question was what a single small step forces downstream.

6.2 Bennett moves where the cost is

In 1982 Charles Bennett, who had already shown in 1973 that computing can be done with no two arrows meeting if you keep the whole history, took the demon apart one step at a time.

The being looks at a molecule and writes down which side it's on. Draw it. Before the look, there are two ways the world could be: molecule on the left with a blank notepad, or molecule on the right with a blank notepad. After the look, there are two ways: molecule left with the pad saying "left," or molecule right with the pad

saying “right.” Two went in, two came out, and you can read the before off the after. No two arrows met.

But something got used up: the blank page. Before the look, the pad was blank and ready. After it, the pad is full. That’s the fresh paper from chapter one. The look didn’t merge anything. It borrowed a page.

So where’s the cost? The being has one notepad and a lot of molecules. After the first molecule the page is full, and to do the next one it has to wipe it. Draw the wipe. Pad says “left” goes to blank. Pad says “right” goes to blank. Two arrows, one landing place. There it is. Looking feels like the active part, the thing the being does, and wiping feels like housekeeping. The housekeeping is where two arrows meet.

That’s Bennett’s, and it’s the same rule chapter three wrote on the move: what has to cost is “the erasure of a bit or the merging of two computation paths.” In the same 2003 paper he lists copying onto a blank place among the operations that can, in principle, be done without any such cost.

6.3 The demon on a sheet of paper

Build it, because the whole thing is four dots, and you should see the arrows meet or not meet for yourself.

The machine has a hidden bit, which is the molecule’s side, and a display, which is the notepad. Write its four states as pairs, hidden bit first and display second: 00, 01, 10, 11. It has two moves.

MEASURE copies the hidden bit onto the display the way a light switch adds: if the hidden bit is 1, flip the display; if it’s 0, leave it alone. Draw the arrows. 00 goes to 00. 01 goes to 01. 10 goes to 11. 11 goes to 10. Every state has exactly one arrow landing on it. Nothing met.

Look at the two states where the display starts blank, 00 and 10. They go to 00 and 11: the hidden bit got copied onto the blank display. That’s the look.

WIPE sets the display to 0 no matter what it held. 00 goes to 00, and 01 goes to 00 as well. 10 goes to 10, and 11 goes to 10 as well. Two arrows land on 00 and two land on 10. That’s the wipe, and arrows meet twice.

Hidden bit, display before	After MEASURE	After WIPE
00	00	00
01	01	00
10	11	10
11	10	10

So Bennett’s rule charges WIPE at least one, and it’s allowed to charge MEASURE nothing. Zero on the price list doesn’t mean zero spent in a real device; a real MEASURE takes time and energy, and it uses up the blank display. What it doesn’t do is make two arrows meet.

One detail worth seeing. Follow only the two cases you actually set up, the ones with a blank display: after MEASURE they're 00 and 11. WIPE sends those to 00 and 10. Still two different states, because the hidden bit still tells them apart. So on those two cases, WIPE didn't lose anything. The rule charges it anyway, because the rule has to answer for every state it might be handed, and on the full table the arrows meet. The price is the worst case on purpose.

Now be the one watching. You can see the display and not the hidden bit. The display starts blank, so you've got two candidates for what's going on: 00 or 10. One bit you don't know. The machine runs MEASURE, and you look. If the display shows 1, only 10 could have done that. If it shows 0, only 00. Your two candidates went down to one. You learned the hidden bit.

The price was zero. And the machine's own list of where it could be went from 00 and 10 to 00 and 11. Still two. Nothing it could have been got merged with anything else.

You narrowed. It didn't.

6.4 Two rows of dots

That last line is the thing I had to get straight before any of this made sense, so here it is as a picture.

Draw the machine's possible states as a row of dots. Now draw a second row above it: the dots that you, watching from outside, still think it might be on. The first row is a fact about the machine. The second is a fact about you, about what you've seen and what you've crossed off. Both rows can shrink, and they shrink for different reasons.

The machine's row shrinks only when two of its arrows meet. That's what the price charges. Run the halving rule from chapter three along a whole run and the counts just multiply: if the first move costs one, the row can at most halve; if the next costs two, it can shrink to a quarter of that; and so on. So a run can't shrink the machine's own row by more halvings than it paid for, all told. (Count with whole questions instead and you lose a little: four places and three places both take two yes-or-no questions to pin down, so the rounded count can't see that squeeze. The plain count can.)

Your row shrinks when you see something some of your candidates couldn't have shown you, and you cross them off. Nothing happened to the machine. It didn't merge anything. It doesn't know you're watching. It would have done the same thing with nobody there.

The physicist Edwin Jaynes had a name for mixing those two rows up, and it's one of the most useful names I know: the mind projection fallacy, treating what you know about a thing as if it were a property of the thing. Your list of candidates is in

your head. The machine's row is in the machine. The price is about the machine's row, and crossing a candidate off yours doesn't move it.

I have to be fair to Jaynes, though, because he'd argue with where I just took his phrase. In a 1965 paper he wrote that entropy is "an anthropomorphic concept," a phrase he credited to the physicist Eugene Wigner, meaning it belongs to the particular experiments you or I choose to do on a system, not to the system alone. For Jaynes, the second law is about what's known too, and the line I'd like to draw, that physics cares about the machine's row and not yours, is exactly the line he'd refuse. That fight, Jaynes against Boltzmann, is one of the oldest in the subject, and I can't settle it and don't have to. What I need is only this: the two rows are different rows, and only one of them belongs to the machine.

6.5 Who holds the pen

Here's the most beautiful thing in this chapter, and you find it by playing with the four dots, not by thinking about them.

Run MEASURE twice. The first time copies the hidden bit onto the display. The second time adds the hidden bit in again, and adding it twice the light-switch way gets you back where you started. Check the arrows: 10 goes to 11, and 11 goes back to 10. The display went full and then blank again, no two arrows met, and the price was zero both times.

So a display that was blank before the first MEASURE can be put back to blank with no arrows meeting, by a second MEASURE. Not by WIPE. This is the borrowed page being handed back: while the original is still there to check against, the copy can be un-copied.

What's the difference? MEASURE looks at the hidden bit while it clears the display. It has the thing the copy came from, so it can undo the copy. WIPE doesn't look. It sets the display to 0 regardless, and "regardless" is exactly where the arrows meet. Bennett has this in his 2003 paper too: he lists erasing "one of two copies of data known to be identical" as a step that loses nothing, because you're only removing a duplicate. And Lídia del Rio and her colleagues argued in 2011 that the cost of erasing a system depends on how much the eraser already knows about it, dropping to zero, and in the quantum case below zero. The cost of clearing a record falls on whoever clears it, and it's lower the more they know.

Try it with a card under a cup, red or blue, and a note that's blank or marked. Start with the note blank. Lift the cup: for red, flip the note, blank to marked or marked to blank; for blue, leave it. Now the note tells you the colour. Do the same thing again while the card is still there, and the note goes back to blank. Check all four card-and-note combinations and you'll see this rule undoes itself, so no two combinations land together. Now erase the note regardless of the card: for either

card, blank and marked both land on blank. Arrows meet. The cards don't measure anything about heat. What they show is which information each rule gets to use.

6.6 Learning after the first look

A fair objection: the demon's watcher learned something by looking, but any watcher learns something on the very first look, before the machine does anything. So the better question is, after that first look, can a step teach you more, at no cost?

Yes. The smallest machine that does it has three states in a ring: U goes to V, V goes to W, W goes back to U. Say all you can see is one thing: whether it's on W. Start with two candidates, U or V. Neither is on W, so the first look tells you nothing. Take one step. From U you'd be on V, not W; from V you'd be on W. One step told them apart, and a ring never has two arrows meeting, so the price can be zero.

Two states can't do it, ever, with any arrows. If you can't tell two states apart at the first look, what you can see is the same on both, and those two are all the states there are. So what you can see is the same everywhere, forever. Try to draw a counterexample with two dots. You'll draw the same two dots a few times and give up, and giving up is the proof. Three is the smallest number of states where two that look alike at first can come apart later.

6.7 Finishing together

One more count before the big picture, because it's about how much room learning needs.

Run a machine from several different starts, and photograph its whole state at every tick. Lay the photographs out in rows, one row per start, one column per tick. Say every run finishes for the first time at the same tick. And say that in every column up to the one just before the finish, no two states step to the same state. That includes the last step, the one onto the finish: if two runs sat on two different states just before the finish, they can't both step onto one finished state.

Could two photographs on that sheet show the same state? Two in the same column: walk both back a tick at a time, and since no arrows met, they were equal the tick before, all the way back to the starts. But the starts were different. Two in different columns: give them both the same remaining steps. The one that's ahead would finish first, and it was supposed to finish at the same tick as everybody else. So every photograph shows a different state. With N starts and T ticks, the machine needs room for N times (T + 1) states. Running in step and finishing together has a size.

6.8 The certainty count

Now I can put the whole path from not knowing to knowing on one page.

You don't know which state the machine started in. There are a few it could have been, each with its chance. Run the machine. Take any two of those possible starts and watch where they go. Only two things can happen. They land on the same state, and the difference between them is gone from the machine: two arrows met, and under the halving rule from chapter three the price covered it, at most one halving per unit. Or they land on two different states, and the difference is still sitting in the machine, kept as two states held apart. There's no third thing a pair of starts can do.

So everything you didn't know about the start splits cleanly in two: the part the run destroyed, which was paid for, and the part the end state still holds.

Now put a watcher outside, who can see some part of the machine and not the rest. Whatever they see depends only on where the machine started, so whatever they learn is a fact about the start. And the same split holds for it: what they learned is either still held in the machine's end state, or it's gone from the machine and two arrows meeting paid for losing it. A run that costs nothing leaves everything the watcher learned still held in the machine.

That's the exact sense of "looking borrows." The watcher's copy is a copy, and the machine still holds the original. When the machine lets go of the original, two arrows meet, and the bill comes due.

What that doesn't say: how much heat. That's for the heat chapter. It doesn't say the watcher's own notes are free; they need a blank page too. And if the start is known for certain, there's nothing to learn and nothing destroyed, so there's no floor from any of this at all.

6.9 Who gets charged is partly bookkeeping

The physicists had this freedom on the books before me. Paul Fahn in 1996, doing the accounting of Szilard's one-molecule engine step by step, argued that whether you charge the looking step or the wiping step is partly a choice of how you keep the books, and that the total is what's fixed. Takahiro Sagawa and Masahito Ueda in 2009 worked out a bound on that total, with the split depending on the memory. So "a price list could charge MEASURE instead" is a known freedom, not a loophole I found.

The other side is alive too. Ruth Kastner argued in 2025 that Brillouin was essentially right after all: put the real physics of measuring back in, and it's the measuring that pays and stops the demon, and Bennett's free copy fails as physics. And John Norton argued in 2013 that at the scale of molecules, jiggling from heat keeps any process that's supposed to be perfectly reversible from ever finishing, and

finishing it costs more than Landauer's amount. If either of them is right, a real demon's look has a cost after all.

What's on the page here survives that, because it's about price lists: a price list that follows Bennett's rule isn't forced to charge the copy, whatever a real device spends. What wouldn't survive is the physics I've been hanging on it, that a real demon can look for free. If Norton is right, the line between looking and keeping keeps its counting and loses its physics. I can't referee that one.

And in 2010 a group in Tokyo, Shoichi Toyabe, Takahiro Sagawa, Masahito Ueda, Eiro Muneyuki and Masaki Sano, ran a real demon: a tiny particle walked up a staircase by feedback on what a camera saw. The energy it gained was paid for by the information the camera took, and nothing was violated. The demon is real, it works, and it pays. Where it pays is the part that took a century.

6.10 Somebody got there first, goddamn it

I'd been sitting on this idea, that making a result stay so it can't be handed back is what pays, and finding out only borrows, and I thought: this one's mine. Then I found Carlo Rovelli's 2024 paper, "The Thermodynamic Cost of Choosing." A real physicist, with a real paper, saying that making a choice that sticks gives off at least the heat Landauer charges for erasing a bit. He even quotes a German saying, Wahl ist Leiden, to choose is to suffer, and at that point what are you going to do but laugh.

It pissed me off. It's hard to have an original thought in this world. But it didn't last long, because once I actually looked, his was a sentence on the surface, and the machinery under it was different.

On its face he contradicts Bennett. Bennett says measuring doesn't have to cost; Rovelli says any measurement costs. So look at what Rovelli calls a choice. A coin in the air hasn't chosen anything; it has to hit the ground and stop. And then the sentence I'd put on my wall: "stabilization must be irreversible, otherwise it is not stabilization." Bennett's free look is one that can still be undone, which is why it's free. Rovelli's choice is one that's been made to stay, which is why it costs. The way I read them, they're standing on opposite sides of the same line: looking, which borrows, on Bennett's side, and keeping, which pays, on Rovelli's.

That reading is mine, and I owe you exactly how much of it. Push on it with his paper open and it gives in four places. He saves the word "choice" for an outcome that's stable and not predictable, and rules out a thermostat because it runs on a predictable mechanism; the settling move in this book is fixed in advance by a rule, so by his own definition his result doesn't cover it. His choice is one possibility branching into two, which he calls in a sense the opposite of erasing; the toll comes from two arrows meeting, the opposite shape. His stability lasts a while, not forever;

mine is forever, by definition. And he doesn't stand beside Bennett at all: he holds that any measurement costs, credits that to the philosopher Hans Reichenbach, and doesn't cite Bennett. So what Rovelli and I share is a sentence: a choice that sticks costs something. Push on it and his coin hitting the ground, a fact nobody could predict, sits closer to looking than to keeping. I still think it's the same instinct. You've got everything you need to disagree.

And on copying, the nearest physics isn't Rovelli's. It's in biology. Thomas Ouldridge and Pieter Rein ten Wolde worked out in 2017 the basic costs of making a copy of a molecule that stays matched to its template after the two separate, and of destroying one. A companion paper the same year, with Christopher Govern, does the same kind of accounting for how a cell reads out its own sensors, which is a measurement in all but name. The way I read them, the copy that has to stay is the one that pays. Then Jenny Poulton, ten Wolde and Ouldridge in 2019 found the sharp form: in their simple models, when a copy grows with the weakest possible push, its advantage over random copying, and the efficiency of storing that information, both vanish. Pushing harder, with chemical work, makes copies that stay matched after the two let go. That's finding out against keeping, in chemistry, with the cost attached, and biology had the distinction before I did. Theirs is a correlation between molecules, with heat in it. Mine is a reading in a model of computation, with no temperature anywhere.

Two more neighbours have "learning" and "commit" in their titles, so the line here shouldn't be read against them by accident. Sebastian Goldt and Udo Seifert in 2017 did the physics of a neural network learning a rule, and argued that learning has a cost: the entropy the network produces limits how much it can learn. That doesn't contradict this chapter. Their learner isn't copying onto a blank page; it's a driven, noisy physical system, and that's what they price. This chapter only says one way of finding out, a copy onto a blank, is allowed to be free. Nothing here says every way of learning is cheap, and theirs isn't. And from statistics, Aurélien Garivier, Tor Lattimore and Emilie Kaufmann in 2016 compared strategies that explore for a while and then commit to one choice against strategies that keep adjusting as they go. In their setting, over a long run, the best strategies that commit lose about twice as much reward as the best ones that keep adjusting. That's the closest published sentence I've found to "committing costs something" with a number on it. It's lost reward, though, not a count on a ledger, so it's a cousin, not a twin.

6.11 What the toll is for, said at last

Now I can say exactly what the toll charges. Not looking. Not copying. Not finding out. A look can be done with no arrows meeting, on a price list that charges nothing for it; the demon is a machine where that happens. Looking uses up a blank page,

and while the original is still there, the page can be handed back. What gets charged is the one thing chapter two showed can't be done on a finite machine without two arrows meeting: settling something so it stays, a record nothing hands back.

That's what "No Free Insight," the name I gave the toll before I understood it, really covers, and I don't want the name doing work it hasn't earned. Insight in the sense of finding out can come free; it borrows a page. The insight the toll prices is the kind made to stay. Whether that narrower thing deserves the name is fair to argue about. I kept the name because it's the question I started with, and because narrowing it down is the finding.

One more thing this tells you, for later. A later chapter counts what it takes to narrow a list of candidates down to a shorter one, and puts a cost on it. Narrowing alone doesn't force anybody to pay anything; the demon just narrowed two candidates to one for free. So wherever this book puts a price on narrowing, the payment goes in as an assumption, out in the open, and I'll keep calling it one.

CHAPTER 7

Looking through a window

Page one ended on two sheets of paper. One person sorted a list and checked it. The other copied a sorted list off somebody else. Same numbers, same order, and nothing on either sheet tells you which one was checked.

The obvious answer is: look harder. This chapter is about why looking harder doesn't help, exactly when it doesn't, and what does.

7.1 Two boxes under a camera

Put two boxes on a table. Each one has tiles on top, and the tiles spell out the same answer. On the side of one box, a stamp is pressed. On the other, it isn't. Hang a camera straight above them and take a photograph. The two photographs are identical.

Now hand me the photographs and ask which box was stamped. Take a better photograph. Blow it up. Count every tile. None of that gets the side of the box into the picture. The difference is real; it's on the box. It just isn't in what you handed me.

That's the thing I want a word for, and the word is window. A window doesn't see anything. It's a viewing port, an opening that faces one way and narrows what you can take in from where you're standing. The camera above the boxes is one: it faces down, so the tops come through and the sides don't. A display is one. So is a printout, or anything else that lets part of a machine through to you. And behind any one picture there's a pile: every full state of the boxes that would give exactly that picture. The tiles can agree while the boxes differ in whether they were stamped, in what checks were run, in what was erased along the way. All of that sits in the pile behind one photograph. There could be two states in the pile, or a million, or endlessly many.

7.2 A cave, and the people who got good at it

This one is Plato's, from the seventh book of the Republic. Socrates describes prisoners chained in a cave since childhood, facing a wall, with a fire behind them

and people carrying objects along a walkway between the fire and their backs. All the prisoners have ever seen is shadows on the wall. To them, he says, the truth would be nothing but the shadows. Later in the story Socrates reads it out himself: the cave is the world we see, and the climb out is the soul's climb toward what's really there.

The part I need comes a little before that. One prisoner gets freed and taken up into daylight, and he thinks back to the cave. The prisoners, Socrates says, hand out honours to whoever is quickest at spotting the shadows as they pass and best at guessing which comes next. Read that twice. The prisoners get good at it. They build a whole science of shadows, with experts and prizes, and it works, on shadows. What they can't do is say anything about a carried object that leaves no mark on the wall. That isn't because they're stupid. The information isn't on the wall. Skill inside a window doesn't reach past the window, no matter how much skill.

7.3 The shadow of a cube

Try this one tonight with a lamp and a wall. Keep the lamp well back. Hold a cube face-on to the wall and mark the outline of its shadow. Then hold up a square card in its place and move it until the outlines match. Show a friend only the outline. They can't tell you whether the thing had thickness. Ask them, and they'll tell you, truthfully, that they can't know, and that staring at the square won't change it.

Now turn each one a quarter turn. The card's shadow becomes a thin line. The cube's shadow stays a square. Now your friend knows. What changed wasn't how clever they were. It was the window. A second look from a different angle is a bigger window, and what paid for it was the turn.

Edwin Abbott wrote a whole novel about a window like this in 1884: Flatland. The people of Flatland live in a flat world and never see a shape. Put a coin flat on a table and lower your eye to the edge, and the coin turns into a line. That's all the view there is. What gives a Flatlander anything more is fog: a near line is bright, and a line running off into the fog goes dim, so a trained eye can read a triangle or a pentagon off how fast the brightness fades. The training takes time and money, and in the story only people with money get it. The window got bigger, and it was paid for.

Then a sphere passes through the narrator's flat world. What he sees is a line whose ends fade wrong, changing size in a way no flat shape could. He only works out it's a circle by feeling it, and the circle grows from a point to its widest and shrinks back to nothing as the sphere passes through. From one slice he can't put a sphere together. From the slices one after another, seen and felt and argued over, he starts to. The way out of a window is never "think harder." It's always a different window, one you can tell two things apart through when you couldn't through the

old one. Every time, in every story here, that cost something: a turn, a second look, training, time.

Who was Edwin Abbott? Not a mathematician. He was headmaster of the City of London School for twenty-four years, a clergyman, and the author of more than forty books, a Shakespeare grammar among them. He put Flatland out under the name "A Square." His full name was Edwin Abbott Abbott, two A's, and the pen name may be a joke on that, A squared; nice if true, but take it as a lead. The part I like best comes from a sketch of his life on the geometer Thomas Banchoff's pages: Flatland, by far the most read thing he wrote, isn't mentioned at all in his obituary in the London Times. The official record of the man was a window too. Through it you got the headmaster and the grammarian, and not the one book people still read.

A video about that book is where the hunch behind this one started. That story is in the afterword.

7.4 The whole fact

Here's the whole thing, and it's small.

You've got the states, the real things. You've got a window: for each state, it lets through some part of it, and that part is all you get. And you've got a question about the state: give it a state and it hands back an answer. What you want is a reader: something that takes what you can see and gives the right answer to the question, every time.

7.5 When does one exist?

Take two states that give the same picture. The reader sees the same picture for both, so it gives the same answer for both. If the question has different answers on those two states, the reader is wrong on one of them, and there's nothing it can do about it, because it never saw the difference. So a reader can exist only if the question gives the same answer on every state in the same pile.

And the other way round: if the question does give one answer per pile, a reader exists. For each picture, answer whatever the question says about any state behind it; since they all agree, it doesn't matter which one. Existing isn't the same as being easy to find, though. To work out the answer for a picture you still have to find a state behind it, and on some windows that search is hard, or endless.

Try the smallest case by hand. A state is a position, 0 or 1, plus a hidden flag, up or down. Through the window you get only the position. Start at position 0 with the flag down. One move goes to position 1 and puts the flag up. Another move goes to position 1 and leaves the flag down. Through the window, both look like position 1. Now write down every possible reader from "what I see" to "is the flag

up." There are exactly four: from two positions to two answers. Check each one against both results. None gets both right, because both show position 1 and one has the flag up and one doesn't. There's no fifth reader hiding anywhere.

7.6 People who found this on their own

The fact is so plain that it got found separately in every field that has to deal with seeing part of something. Laying them side by side is the kind of evidence I trust: people who never met kept walking into the same wall.

In 1956 Edward Moore, working on the small switching machines that became computer hardware, asked when somebody who can only feed a machine inputs and watch its outputs can tell two of its inside states apart. He made "can't tell apart from outside" precise, and found a sting: even when every pair of states can be told apart by some experiment, there are machines where no single experiment tells you which state you started in. What you can always do is find out which state you're in now.

In 1960 Rudolf Kalman asked the same question for systems that change smoothly, called it observability, and gave a test you can compute. His unobservable states are exactly a pile: states whose difference never reaches the output, however long you watch. Try it with two toy cars on identical roads, a mile apart. Give them the same speed at every moment, and let nothing about the road depend on the mile marker. Their speedometers tell the same story forever. No amount of rereading that story gives you the starting position. A landmark, or a hill at a known spot, would.

Where did Kalman's filter go first? Toward the Moon. In the autumn of 1960 Kalman visited Stanley Schmidt at NASA's Ames Research Center, not knowing what Schmidt's group was working on. They were stuck on how a spacecraft going around the Moon could work out where it was from a few angles its crew could measure through a sight. That's a window problem with a capsule in it: the state is position and speed, the view is a handful of angles, and the job is a running best guess from a string of views. With real work, his method fit, and it went into the studies for navigating Apollo. Leonard McGee and Schmidt, telling the story in 1985, called it a fortunate sequence of events. The filter didn't make the angles show more. It used how the spacecraft moves, plus a string of noisy views, to estimate what the views alone couldn't pin down.

In 1977 Patrick and Radhia Cousot built a whole way of analysing programs on throwing information away on purpose: run the program on rough descriptions of its values instead of the values themselves, and only accept answers that are true for everything the rough description could stand for. If all you know is that a number is between minus one and one, you know its square is between zero and one. You don't know which square. In 1970 Edgar Codd, inventing the relational database,

gave users views, deliberately partial windows onto the stored data, and in 1981 François Bancilhon and Nicolas Spyratos worked out the problem from the other side: if a user changes what they see through a view, which of the many stored states behind it should the database move to? Several are legitimate. That's the pile, seen from the writing side.

Two more fields have it as a founding idea. In the logic of knowledge, what an agent knows is modelled by sorting the possible states into piles it can't tell apart, and it knows a fact exactly when the fact holds everywhere in its pile. The economist Robert Aumann used that in 1976 to prove that two people who start from the same beliefs, and whose estimates are known to each other, can't agree to disagree, and Ronald Fagin, Joseph Halpern, Yoram Moses and Moshe Vardi built a whole logic on the same piles in 1995. And in computer security, Joseph Goguen and José Meseguer defined noninterference in 1982: one group of users doesn't interfere with another when nothing the first group does changes what the second can see. David Parnas, whose cut of a program into modules comes up later, hid things on purpose: a module is defined by the decision it hides from whoever uses it.

There are more. Oliver Kosut and Jörg Kliewer asked in 2018 when a receiver, with a possible attacker on the line and no shared key, can always either get the message right or notice the attacker, and the answer turns on whether the attacker can make what the receiver sees look exactly like an honest message. That's the pile with the probabilities taken out. And in 2022 Anvith Thudi, Hengrui Jia, Ilia Shumailov and Nicolas Papernot asked how a company could prove a trained model had forgotten someone's data, and found the same finished model can come out of different training sets, so the model alone can't show whether a point was used. The only thing you can check is the procedure that was run. Same wall: the finished state is a window, and what happened is in the run.

7.7 Notes passed across a room

Philosophers who study being told things hit the same wall from the other side. Read one after another they blurred together for me, until I built a picture of where each one stands.

Somebody passes you a note across a room. It says the answer to question seven is 42. What crossed the room? Ink. Whatever made the writer believe 42 stayed on their side: the working-out, how careful they were, whether they knew or guessed. Pull that apart and there are five things in play: the claim on the paper, the writer's head, whatever got the writer there, the trip across the room, and your head at the other end. Every position I found puts its weight on one of those five.

The two ends are John Locke and Tyler Burge. Locke, in 1690, says believing something because someone with authority said it isn't knowing it: if the reason

didn't cross the room, you don't have it. Burge, in 1993, says you're entitled to accept what you're told, if it's presented as true and makes sense to you, unless you have a reason not to. I get the pull of Burge; nobody could get through a day checking every note. But a default anybody can trigger by writing something sensible is a record nobody paid for, the thing this book keeps refusing. So on that dial this book sits at Locke's end, and staying there means carrying the check across the room every time.

Jennifer Lackey, in 2008, cuts the wire between the writer's head and what arrives: a teacher who doesn't believe the theory she's required to teach, and students who come away knowing it anyway. Sanford Goldberg, in 2010, puts the weight on the process behind the writer's word, machinery the reader can't see. I'm not comfortable with that, and that discomfort is why this book exists: machinery you can't see is machinery you can't check. Christoph Jäger and Nicholas Shackel built a model in 2025 where the writer knowing is neither needed nor enough for the reader to know, landing where Lackey's teacher did from a completely different direction. Wesley Holliday, in 2009, put trust into a logic of changing your mind, with a graph of whose word each person takes. I've only read a summary of Holliday's paper, so I'm taking that on somebody's word, which is a funny thing to admit in this paragraph. And Michael Burrows, Martín Abadi and Roger Needham, in their 1990 logic of authentication, said: if I believe an authority has the say over a statement, and I believe the authority holds it, I believe it. That's Burge's default with one change: you named the authority before the note arrived.

Kenny Easwaran, in 2009, went after what crosses the room itself. Why do mathematicians accept a long proof, even one partly done by a computer, and turn down a proof based on chance that can be far more certain? His answer is a property he calls transferability. A step-by-step proof puts the reason on the paper: every step crosses the room, so you can learn from the writer without trusting the writer. A proof based on chance leaves the reason behind and sends only the writer's word about what the dice did. That's the cleanest statement I found of what a checked record should do: make the reason cross the room, and run the check on your own side. That reading is mine.

7.8 Slips handed out at desks

People who build systems out of many computers hit the wall too, and one pair measured how far it goes. Picture people at desks, working from one rulebook and passing slips to each other. Nobody sees into anybody's head; all anyone gets is the slips. Somebody may be cheating. What can the honest ones do about it with nothing but slips?

Andreas Haeberlen and Petr Kuznetsov asked in 2009 which kinds of cheating an outside observer can catch at all. As I read them, it depends on whether the cheating ever shows up in a slip. A cheat who breaks the rulebook without changing any slip, or who writes a slip an honest player could have written too, can't be caught by anyone holding the slips. A cheat who fails to send a slip that was due, or sends one the rulebook would never produce, can. The first two are a pile: an honest room and a crooked one handing out the same slips.

Their group's PeerReview system, from 2007, is the ladder up to that wall: every desk keeps a secure log of its slips, and a checker replays the log against a copy of the rulebook. That's close to "show me everything," built. Close, because a log of slips only stands in for a whole desk if the rulebook can rebuild what the desk did from them, and only if the checker's rulebook is the right one.

Then the people asking what you can say once you're at the wall. Ralf Küsters, Tomasz Truderung and Andreas Vogt, in 2010, defined a verdict that names who misbehaved, and showed that being able to check a result is a weaker form of it: one says something went wrong, the other points at a desk. Radha Jagadeesan, Alan Jeffrey, Corin Pitcher and James Riely, in 2009, looked at systems that enforce their rules after the fact, by audit. Robert Künnemann, Ilkan Esiyok and Michael Backes, in 2019, built a way to check it automatically by making it a property of the whole run. And in September 2026 Timo Treitz and Künnemann gave the public logs of web certificates the full treatment, and found that the plain version names a culprit only if the log itself is assumed honest. Somebody always has to be trusted somewhere.

Turn that on me and it bites twice. A raise of the stamp that never reaches the window is the first kind of cheating, the kind nobody outside can catch. And every verdict in this book is "yes" or "no" about one claim; none of them points at a desk.

7.9 Moves that keep to the tabletop

Back to the two boxes. Give them two kinds of move.

Tile moves move the tiles, and only by looking at the tiles. They cost nothing and never touch the stamp. Stamp moves leave the tiles alone and can only press the stamp or leave it as it is. Once pressed, it stays pressed, and pressing it pays the toll.

Now take the stamped box and the unstamped box, with the same tiles, and give them the same tile moves. After one move the tiles still agree, because each next picture depends only on the picture before and the move. After two moves, they still agree. After a hundred. Meanwhile one stamp stays pressed and the other doesn't. You can watch the tabletop for the whole run and never see the difference you started with.

There's one way to break that, and it's worth seeing. Let somebody read the stamp and choose a different tile move because of it. Now the stamp can signal through the tiles. Here both boxes get the same moves, on purpose, to keep the question fixed: what can come through the window, if nothing that reads the stamp is allowed to steer the tiles?

7.10 The clerk with two photographs

Now point the camera at the price, because that's what this chapter is really for.

Say you want to charge for pressing the stamp, exactly: at least one for every move that presses it, nothing for a move that doesn't. And say the person charging is a clerk who only looks through the window: a photograph before each move and a photograph after. The clerk's charge can only depend on those two photographs.

Can the clerk charge exactly? Not if there's a pair like this: two moves that give the clerk the same before-photograph and the same after-photograph, where one pressed the stamp and the other didn't. A stamp pressed inside a closed drawer, next to somebody opening and shutting the drawer. The clerk has one charge for that pair of photographs. If it's at least one, the move that pressed nothing gets overcharged. If it's zero, the press gets undercharged. Nothing read off the photographs gets both right.

The fix is the plain one: put the stamp in the photograph. Then "one when the stamp goes from not pressed to pressed, zero otherwise" works. Showing the stamp isn't the only fix. Any window with no such pair works too; a later chapter has one that never shows where the record is but always shows when it moves.

You can even find the clerk's best careful prices. Put all the moves that give one pair of photographs in a tray. If you never want to miss a press, charge one whenever the tray has even one press in it. If you never want to overcharge, charge one only when every move in the tray is a press. When the tray is all presses or no presses, those two agree. When it's mixed, they can't, and that's the bad pair again. Those are "some member" and "every member" applied to the same tray, which is exactly how the mathematician F. William Lawvere described logic's two quantifiers in 1969, and Robin Cockett, Isabelle Geng, Carlo Maria Scandolo and Priyaa Srinivasan used the same idea in 2023 to stretch measures of resources along a map.

Control engineers have hunted pairs like that for a long time. In 1995 Meera Sampath, Raja Sengupta, Stéphane Lafortune, Demosthenis Teneketzis and Kasim Sinnamohideen asked when an observer can always work out, from what it sees, that a hidden fault has happened. Anooshiravan Saboori and Christoforos Hadjicostis asked in 2007 the question the other way round, for secrets: when can an observer never be sure the system is in a secret state? A colliding pair like the clerk's is one

place where the observer can't tell right away. Their versions are richer than mine. What this chapter adds is the price.

Why can't the clerk just watch the move itself? A clerk who sees the whole state and the move charges exactly with no trouble. So the problem bites an observer who doesn't see the move, or who's handed an account of it they can't trust. If the names of the moves come from a log the program wrote, the program can write whatever names it likes, and a clerk reading that log is reading one more window, one the program painted.

7.11 Look before the concrete covers it

An inspector arrives after the concrete is poured. The question is about the steel underneath. On one job it was sound. On the other it wasn't. Give both jobs exactly the same finished surface, and give the inspector only that.

There's no amount of staring that tells those jobs apart. Hunting for a sound arrangement that could have made this surface doesn't do it either; a sound arrangement was possible on the bad job too. What matters is which one was there. It's the four readers again: one input, two answers needed.

While the steel is still showing, you can inspect it and keep the answer. Or keep enough evidence to inspect later. Either way, something that tells the good job from the bad one has to survive. Wiping some unrelated detail does no harm. Wiping the last difference your question depends on leaves the next inspector with the same surface and two possible pasts.

For one question asked again and again along a run, what you have to keep can be tiny. Start a bit at yes, and at each check, set it to yes only if it was yes and this check passed too. It says exactly whether every check so far passed. A later failure turns it down for good. Put that next to a stamp that says "some check passed at some point" and stays up, and you can see they answer different questions.

7.12 Three ways past the window

Say you're outside, and you want to be convinced a run paid. The window you have doesn't show the count. How bad is that?

Take the smallest setting. A run that presses the stamp and counts one, and a run that only moves the tiles and counts zero, from the same start to the same picture. Call the before-and-after photographs the transcript. Both runs leave the same transcript. Now try to build a checker that reads the transcript and says yes exactly when the count is one. It has to say yes to the first run's transcript. The second run left the very same transcript, so it says yes there too, and the count there is zero. No checker of that transcript is both never fooled and never unfair. Dress

the transcript up with extra parts, and the same thing happens, as long as the extra parts are the same on both.

Economists hit the same wall: Jerry Green and Jean-Jacques Laffont in 1986, and Jesse Bull and Joel Watson in 2007, worked out what a system can decide when people can only claim about themselves what their evidence backs up. An audit can only tell apart the cases the evidence on offer tells apart.

So what works? I know three things, and cryptography seems to use all three. Each gets past the window, not past the cost: the run that pressed the stamp paid either way. And each rests on trusting something.

You can be shown the whole state. Then there's nothing to work out; you read the count straight off. What you trust is that the state you were shown is the state there was.

You can be shown a promise: the ordinary window plus one extra mark that's set when, and only when, the claim holds. Reading that mark works exactly when it can't be set falsely and is always set when it should be. What you trust is whoever can set the mark. A mark anybody can set is just another window. Bruce Schneier and John Kelsey worked out in 1999 how to make a log that an intruder who takes over the machine can't quietly rewrite, and their honest limit is the honest limit here: it protects what was written before the break-in, and nothing after. Mihir Bellare and Bennet Yee had named that property forward integrity in 1997. Point it at an AI agent keeping its own log, and the line moves to the very start: an agent that writes the wrong thing from its first step is past the break-in for the whole log. A 2026 preprint by Laurent Bindschaedler, Quentin Botha and Christoph Siebenbrunner builds exactly such tamper-evident trails for agents and calls it an Agent Flight Recorder.

Or you can ask. The outside party puts a question and gets an answer that pins it down. What you trust is the honesty of whoever answers, because a liar's answer is another shadow. The refined form is Shafi Goldwasser, Silvio Micali and Charles Rackoff's zero-knowledge proofs, from 1985, where a checker is convinced by a conversation built to separate true from false with a guarantee against a cheating prover. Joseph Halpern, Yoram Moses and Mark Tuttle gave them an analysis in terms of knowledge in 1988. My asking, in this chapter, is one round with one fixed question, and nothing in it handles a liar. It's a toy of the real thing, and I'm not going to let the real thing's reputation stand behind the toy.

The real thing even has a cave in it. In a 1989 paper called "How to Explain Zero-Knowledge Protocols to Your Children," Jean-Jacques Quisquater and Louis Guillou, with their families and Tom Berson, told it as Ali Baba's cave: a ring of tunnel with a magic door at the back that opens only for a secret word. Peggy goes in down one side while Victor waits outside; then Victor walks to the fork and shouts which side she should come out of. If she knows the word, she always can. If she

doesn't, she gets away with it only when he happens to call the side she went in. Play it with a coin for Victor's shout. One round, a cheat gets through half the time. Twenty rounds, about one time in a million. And here's the part that belongs in this chapter: Victor ends up convinced, but a video of the whole thing convinces nobody else, because a staged run, where the two agreed beforehand which sides he'd call, makes exactly the same video. The video is a window, and the real run and the fake one cast the same shadow.

What would it take to make that promise-mark binding for real, against somebody who wants to cheat? Manuel Blum's 1981 paper "Coin Flipping by Telephone" is where I'd point. Alice and Bob have just divorced, they live in different cities, and they want to flip a coin over the phone for who gets the car, with nobody both trust to flip it. If Alice flips and announces, Bob has to take her word. If Bob calls first, Alice can lie about how it landed. Blum's way out is for one side to commit first, sending something that fixes the choice without showing it, and open it after the other side calls. What keeps that binding isn't a promise. It's a bet that a certain kind of big number is hard to factor. So in Blum's protocol, binding is earned, on that condition. In this chapter it's only assumed. That's why I won't say these escapes beat a cheater. Andreas Winter, Anderson Nascimento and Hideki Imai worked out in 2003 how fast a noisy channel can carry promises like that, and Claude Crépeau, Rafael Dowsley and Nascimento in 2020 how that rate drops when the one promising can steer the noise.

One more relative: verifiable computation. A box hands back its answer with a short proof that the answer is right, which a checker can verify without redoing the work (Pinocchio, built in 2013 by Bryan Parno, Jon Howell, Craig Gentry and Mariana Raykova), and proof-carrying data passes proofs like that along a whole chain of parties who don't trust each other (Alessandro Chiesa and Eran Tromer, 2010). That's the strongest way out there is, and what it carries out is the claim. The history of which machine ran it, and when, stays behind unless the proof is built to carry it.

Now point all three back at a stamp kept in a machine's own state. That stamp is bits too. Hand somebody a copy of it with a note saying "it's pressed," and they're reading a window: the run that really pressed it and a box built to print that it did cast the same shadow. So keeping the record inside the machine answers "was this earned?" for whoever can see the machine's state and trusts the step that wrote it. For anybody else, it moves the question from "did it run?" to "do I trust the box that says it ran?" Nothing in this book supplies any of the three ways out. The record here lives in one machine, with nobody's signature on it.

7.13 What gets through

Put the two boxes back under the camera. Their tops agree and their sides differ. The photograph answers questions about the tiles. To answer the stamp question, turn the camera far enough to see the side. The whole state was there all along. What changed was which part reached you.

Taking more photographs from the same spot only helps if something different eventually comes into view. When the two runs give the same picture at every step, a longer album keeps the same blind spot. That's Kalman's unobservable states over a whole run.

And none of this is the famous kind of impossibility. Henry Rice proved in 1953 that no program can sort every program by a question about what it does, unless the question gives every program the same answer. That's a deep fact about computing, and a later chapter takes it up. What this chapter shows is smaller and comes first: hand a reader one picture for two states that need two answers, and it's finished before any question of computing comes up. One input, two answers needed.

Which leaves the question I've been pushing ahead of me since the two sheets on page one. Fine: the stamp got pressed, it got paid for, and you can see it through the right window. What does a pressed stamp actually tell you? Anybody can write "CHECKED" on a whiteboard. What would make the stamp mean the list was checked? That's the next chapter.

CHAPTER 8

Established

The last chapter ended at a whiteboard. Give a worker a whiteboard and a pen, and ask for an answer with a line underneath that says “CHECKED = YES.” The worker can write both. So can somebody who copied the answer and checked nothing. The extra line tells you nothing about which one happened.

This chapter is about what it takes for that line to mean something. It’s the case page one called the sharpest, and it’s sharp because the two answers are the same bits, and everything that differs is in what was kept.

8.1 Two computers that end up the same

Put two computers on a table and hand each one the same list of a thousand numbers.

The first one sorts it. It compares, swaps, walks the list again and again until nothing is out of order. Then it does one more thing: it walks the whole list front to back and checks that every number is no bigger than the one after it. That check could have failed. It passes. Then the computer tidies up after itself, wipes its scratch work, and leaves the sorted list in memory. (That check is about order. Whether the list still holds your thousand numbers and no others is a second question, and it would take a second check.)

The second computer never looks at your list. Somebody typed the sorted version in earlier, the same thousand numbers in the same order, and it puts them in memory.

Now wait until both are done and read what’s left: the list, the working registers, where each one is in its program. Arrange for both to stop at the same point in the same program and clear their scratch work, and all of that can agree. The first compared the numbers and checked their order. The second loaded the answer and did neither. Wheel the boxes out, swap them, and bring them back. From what’s left, neither of us can tell which run happened.

Be careful about what that does and doesn’t say. It doesn’t say every atom of two real computers matches; the room, the heat, a clock or a saved log might still carry evidence. And an ordinary computer can keep a “checked” flag or a log that can’t

be changed. These two were set up so the difference got wiped from what you can read. That choice, and what it stops the next reader from answering, is the example.

Who cares? Anybody who's been handed a sheet that says "all tests passed." Anybody who's looked at an inspection sticker on a windshield: the sticker looks the same whether somebody checked the brakes or somebody sold the sticker. An audit, a proof, a ledger, an AI that sounds very sure of itself: each one is a pile of bits that could have come out of either computer, and the question that matters is the one the bits can't answer. Did something establish this, or did it just print it? I'll come back to every one of those at the end of this chapter and say which ones this book actually helps with, and it's fewer than you'd hope.

8.2 Look harder

The obvious answer is: look at everything, not just the end. So make the computers as easy to look at as computers get. Scale them up to the size of a warehouse, every wire a lamp that's lit or dark, every switch something you can walk up to and touch. Nothing is hidden in a chip too small to see. Put them side by side and walk the floor.

While they run, you'd see the difference: one has a whole wing lighting up, comparing and swapping, and the other just copies. But they finish, and the comparing wing goes dark. Walk the floor now and look at every lamp. The sorted list is spelled out in lit and dark lamps in both, the same pattern, and everything else is dark in both, because both cleaned up.

There's no lamp anywhere that says "a check ran here and passed." It isn't hidden, and it isn't scrambled. It just isn't there. Looking harder doesn't help, because the problem was never that you couldn't see enough. Nothing was kept.

8.3 So keep the trace

Then don't let the run go. Keep the trace: every comparison, every swap, the check and its answer, written down as it happens. Wheel the computers out and back, read the traces, and you'll know which one sorted.

It's the right instinct, and it gets most of the way. But it has three problems.

First, a trace is bits, and bits can be typed in. Whoever typed the sorted list into the second computer can type a trace in too: "compared, swapped, checked, passed," a few thousand lines of it, every one of them believable. A trace a program writes is something the program produced. Keeping it doesn't answer the question. It moves it: did something establish this trace, or did it just print it?

Second, the way out of that is to take the pen away from the program. Don't let the program write its own trace. Have something underneath it write the trace, one

step at a time, as the steps actually happen, so nothing a program does can add a line that didn't happen or erase one that did. Airplanes do a version of this. Flight recorders go back to a prototype David Warren built in Melbourne in the 1950s; he was an Australian scientist at the Aeronautical Research Laboratories, working on jet fuel, and after a string of unexplained airliner crashes he wanted a box that would hold the instrument readings and the voices in the cockpit for whoever came looking afterwards. I know his story from museum and encyclopedia accounts, not his own report. What comes out of a recorder is evidence of what its channels carried, as long as the recorder and its sensors worked. A recorded reading can still be wrong about the airplane. The recorder has the right shape, though, and that shape isn't mine. In computer security it's called a reference monitor, and I'll come back to it.

Third, you can't keep everything. If every tick of a run can go either of two ways, then after n ticks there are 2 to the n possible histories, and a fixed number of bits eventually can't tell them all apart. Flight recorders make a practical choice here: a voice recorder loops, replacing the oldest sound when it's full. Thirty minutes, two hours, twenty-five hours: the window depends on the aircraft and the rule, and the FAA's 2026 rule stretches it to twenty-five hours for certain new aircraft. Whatever the length, it's a window.

The fingerprint is the sharpest answer to that, so take it seriously. Fold every new line of the trace into a short scramble of everything before it, and you've got a hash chain. Haber and Stornetta's timestamps work that way, from chapter four; so do the audit logs Bruce Schneier and John Kelsey designed in 1999, and the measurement registers in the security chip a lot of computers carry. That really does keep something about the whole trace in a few dozen bytes. But look at what it keeps: a binding, not the content. To read the history you still need the trace stored somewhere, and the fingerprint only tells you whether a copy somebody hands you is the one that went in. It holds only because nobody knows how to find two traces with the same fingerprint, which is a bet on how hard a search is, not a law of nature. Moni Naor and Guy Rothblum proved a sharp version of that in 2005, and even called it a fingerprint: checking a big file somebody else stores, against an opponent with unlimited computing, costs on the order of the whole file, and doing even slightly better against a weaker opponent needs exactly that kind of bet. And squeezing endlessly growing traces into a fixed number of bits has to land many traces on one fingerprint. That's two arrows meeting, hidden behind how hard it is to find which ones. It's a very clever way of forgetting, not a way out of it.

So the real question was never "save everything." It's: what's the smallest thing worth keeping for good, and what does keeping it take? Not every comparison the sort made. Just the one fact that a check of this very claim passed and the question got closed. And keeping even that one fact for good takes something, on every machine: two arrows meeting, or fresh paper forever. That's chapters one and two.

So it was never true that computers can't keep records. They keep records all day. What nothing in an ordinary computer's rules does is make a record paid for, permanent, and earned. Those come in at different places. Paid for is the toll. Permanent is in the definition: a Thiele machine's settled answer never goes back. Earned, meaning a check of this very claim passed first, isn't in the definition at all. It's what this chapter adds.

8.4 More boxes to tick

Go back to the sorter. It checked the order of the list. Did it also check that none of the numbers went missing? The list can be the same whether that second check ran or not. One lamp saying "checked" won't tell you which work was done.

So put a sheet beside the scratch work, with a box for each check you care about: one for order, one for "nothing missing," and so on. When you come back you want to know which boxes got ticked.

Count what that takes. Say four checks would pass on this list. Each one can be run or skipped. That's two choices, four times over: sixteen different sheets, all sitting behind the same list. Two of them that differ on even one box can't be the same state of the machine, because asking about that box would need two answers. So the machine needs room for all sixteen behind that one list. In general, it's two to the number of checks that would pass, for every list, added up over the lists. And you can build a machine with exactly that much room and no more: keep the list and the sheet, clear the sheet whenever a new list is written, and tick a box only when its check passes.

Look at one small piece of it, because the arrows meet there too. A list that's true or false, and one check. Three states: false; true and unchecked; true and checked. The check takes "true and unchecked" to "true and checked," and leaves "true and checked" where it is. Two arrows, one landing place, right above the same list.

Now keep the old sheets when the list gets wiped and reused. Think of carbon copies filed before each sheet goes in the bin. On each of n rounds you can choose to check or not. Bring every round back to the same final list. Then ask about any round you like: was that one checked? Two histories that disagree on any round need two different states, however alike the final list looks. That's 2 to the n states for n rounds. Keep asking for one more round, with the same freedom and the right to ask about every one, and no finite amount of room is enough. That's where the endless paper comes from: not from keeping one answer, but from keeping a file of answers that never stops growing.

Ask the file a smaller question, "was anything ever checked?", and it becomes a single box that only goes up. That's the kind of record a Thiele machine keeps.

8.5 Computed, stored, established

Three words get used like they're one.

Computed is about the steps. The first computer computed the sorted list. The second didn't.

Stored is about the bits. Both stored it.

Established is neither, and that's the whole problem. It isn't the steps, because they're gone. It isn't the bits, because they're the same. It's this: whoever reads the first computer has a right to lean on "this list is in order," because a test that could have failed was run on this very list, it passed, and then the question got closed. The second computer has the same sentence and no right to it. That's a true fact about the two of them, and it's written down nowhere, in either one.

Four words, and I'll hold them still from here on. A check is the test, the thing that could say no. A commit is the closing: the question shut for good, about that version of that thing. The record is whatever stays behind to say so. And established, in the sense a machine can carry, is all three, about one thing.

What if the second computer, handed the typed-in list, runs the check on it? Then it has what the first had, and if it closes the question and keeps the record, it has established the list is in order. It didn't sort and doesn't need to. The line here is between checked and unchecked, not between worked hard and didn't. Whatever this book counts, it isn't effort.

8.6 The clerk who forgets everything

The picture of a computer in chapter one was a person with a pencil, a strip of paper and a short list of rules, and Turing's 1936 paper takes it a long way. He's careful with the clerk's mind: whatever the clerk knows, beyond a handful of states, has to go down on paper. Then he goes all the way. He imagines the clerk getting up, walking off, forgetting everything, and coming back later. To pick up where he left off, the clerk leaves himself "a note of instructions," and Turing says that note is the clerk's state of mind, as far as the computing goes. Marks on paper and a note are the whole state.

The two computers are sitting right inside that. Suppose the clerk didn't write the note. Somebody slipped it under the door, in the right form, with the right instructions. He comes back, reads it, carries on, and every mark he makes from then on is exactly what he'd have made if he'd written it himself. Turing's model can't tell the clerk who did the work from the clerk who was handed a note saying he did. It was never asked to. Turing wanted something you could check without crawling into anybody's head, and a note on a desk is as checkable as it gets. That's

why the paper still works ninety years later. A Turing machine could keep a note of who wrote the note, and rules about who's allowed to. This one just doesn't.

So the founding picture of computing is a man who leaves his desk, forgets everything he was doing, and trusts a sticky note. I relate to him deeply.

8.7 Information doesn't have it either

The next guess is information: one computer got some, the other didn't. That one dies fast. Claude Shannon, who turned information into a number in 1948, says on his first page that the meaning of a message is "irrelevant to the engineering problem." His measure depends on the chances of the possible messages, and the same message from the same set of chances gets the same number whichever computer it came from, or if one of them got hit by lightning and came out right by luck.

Fine, forget the set of messages and look at the one string. In the 1960s three people separately worked out how to measure the information in one string: Ray Solomonoff, who was after a theory of guessing what comes next; Andrei Kolmogorov, the great Soviet mathematician; and Gregory Chaitin. Their measure is the length of the shortest program that prints the string. The two computers hold the same string and get the same number. And running a program can't add much of that kind of information: what comes out can't hold more than what went in plus the program, give or take a little. Leonid Levin, who studied under Kolmogorov, proved a general version, and has a line for it that's too good not to repeat: you can torture an uninformed witness all you want, you won't get information about the crime.

That doesn't say sorting tells a person nothing. A person in a hurry can read the smallest number straight off the first slot of a sorted list. Short descriptions, useful access, and a record that a particular check ran are three different things.

Logic says the same thing in a different coat. The standard logic of knowing, the one Jaakko Hintikka worked out in 1962, has a property called logical omniscience: if you know some facts, you automatically know everything that follows from them. So by that logic, once you knew every number on your list, you already knew whether it was in order. Hintikka flagged the problem himself, and people have been trying to fix it ever since. The fixes read like a pile until I asked each one the same question: when it lets you write "known" next to "in order," what does it make you write in the margin? Hintikka's margin is empty. Each fix writes one thing there.

The closest one to me is Sergei Artemov's, from 1995. He wanted a logic that could name the proof itself: you only get to know a thing if you've built an actual reason for it, and the reason travels along with the claim. That's almost a record. Almost, because nothing in his margin says the reason was ever run, or when. Others

taught it to count. Ho Ngoc Duc, in 1997, made knowing take steps of reasoning, and Artemov and Roman Kuznets, in 2014, asked how big a reason can get before knowing by it stops being practical. Francisco Bavera and Eduardo Bonelli, in 2018, turned it into a programming language whose terms carry the history of the computation that made them, and Samuel Bucheli, Meghdad Ghari and Thomas Studer, in 2017, put reasons on a time line where they last and change.

Line those up against what this chapter wants in the margin: earned, paid for, permanent, and placed at a step in the run. Artemov's reason is nearly earned. The counted versions put prices on reasoning. The time line gives dates. What none of them has is the closing of a question as a single step with a price on it. Push on mine, though, and it gives twice. Artemov's margin holds the reason itself, and mine only says a check passed. And their prices grow with how hard the thing was to know; this book charges the step that closes the question, and how long the checking took never enters it. For a measure of effort, read theirs.

And yet you know something after the sort that you didn't before. You can stop worrying. Something happened, and it's the first solid clue: whatever established is, it isn't new bits. It's the same bits, settled: a question sorted into yes and no, with yes made to stay.

8.8 Statues that walk off

If it isn't in the information, maybe it's in the knowing, so I went and read the philosophers. They'd been arguing about the difference between having the right answer and knowing it for a couple of thousand years.

Plato's version is a joke about statues. There was a story that the sculptor Daedalus made statues so lifelike they'd walk off if you didn't tie them down. True beliefs, Plato says in the Meno, are like that: they wander off, and they're not worth much until you tie them down with the reason why. Once they're tied down, they're knowledge, and they stay put. Knowledge tied to a reason, and knowledge that stays.

Then in 1963 Edmund Gettier broke the tidy version in three pages. Smith and Jones are up for the same job. Smith has good reason to think Jones will get it, and he's counted the coins in Jones's pocket: ten. So Smith figures, with good reason, that the man who gets the job has ten coins in his pocket. Then Smith gets the job, and, without knowing it, Smith also has ten coins in his pocket. Smith had a belief that was true and backed by a reason, and it still wasn't knowledge. He was right by luck, because his reason was about the wrong pocket.

That's the push that matters for a machine. Checking one pocket and closing the question about another is right by luck. So whatever a machine closes, the check

behind it has to be about this very claim, and here's exactly what that means. Four things.

It checks the same property the commit names, not a cousin of it.

It checks that property on the same thing the commit names, not the thing next to it.

Nothing has written to that thing between the check and the commit, so it's still the same version the check looked at.

And the check is honest: it says yes when the property holds and no when it doesn't, on anything it could ever be handed.

Break the first three and you're Smith with his ten coins. Break the fourth and the check is a rubber stamp. Two notes on those. The fourth asks a little more than it needs: to rule out luck, a check only has to never say yes when the property is false. And the third is a very old bug in computing wearing Gettier's coat. Security people call it time-of-check to time-of-use: check a file, use it, and something swaps the file in between. Matt Bishop and Michael Dilger wrote up how to hunt for it in 1996. Databases guard against the same thing by making sure, just before a commit, that nothing they read has been written since; H. T. Kung and John Robinson called that optimistic concurrency control in 1981. A philosopher's pocket and a programmer's race condition are the same bug. One limit, said now: a machine can only promise "nothing has written to that thing" about its own memory. For brake pads or a voter roll, that condition becomes an assumption about whoever measured.

Does anything fix Gettier for good? People tried for years. The first patch was "no false steps in the reasoning." Then the philosopher Alvin Goldman came back with barns, crediting them to Carl Ginet. You're driving through a county that, without your knowing, is full of fake barns, just painted fronts propped up to look real. You look at the one real barn and think "that's a barn." It's true, you believe it, you have a good reason, your own eyes, and there's no false step anywhere. And you were still lucky: if you'd glanced at the next one over you'd have believed the same thing and been wrong. Every patch gets a new barn. What I keep from that: whether a check was good isn't only about this one case. It's about what the check would have done on the cases next door.

8.9 What "could have failed" means

I keep saying the first computer's check could have failed. The computer is deterministic. Given your list, the check was always going to pass. So in what sense could it have failed?

Here's the sense. Hand the checking step a list that's out of order, and it says no. That's all. Chance has nothing to do with it. A check splits every possible input into a pile it says yes to and a pile it says no to, and "could have failed" means the no

pile isn't empty. A check that says yes to everything has an empty no pile. It tells you nothing, however long it runs. The philosopher Robert Nozick made a version of this a condition for knowing, in 1981: you know a thing only if, had it been false, you wouldn't believe it.

Notice that "could have failed" is about what the check would have done on other lists. It's not a fact about the finished state at all. Look at the first computer's memory: a sorted list. Nothing in those bits says "and whatever made me would have caught a mistake." That's exactly why the only way a machine can own this is to write something down at the moment the check passes, and have that something mean what it says. Even then the record only carries half of it. It can say a check ran on this version and passed. That the check would have said no on a bad list is a fact about the checker, and that half rests on a proof about the checker, made once, outside any run.

Rig a check to pass only on your one list. Is that cheating? No. It's an honest check of a different claim: "this is that exact list." Its no pile is huge: every other list. It establishes exactly that and nothing more. A check establishes what it's honest about.

And one thing I'll say plainly now: nothing here stops you from asking a question that can't come out no, like "is this number at least zero." Ask one, pay for the check and the close, and nothing is established but the record of asking.

8.10 Computing already had a word for it

Computing already has a word for this and uses it every day. Anyone who's used a database has seen it: COMMIT. A database does a pile of tentative work, and then there's one moment, the commit, written to a log, after which the change counts.

Jim Gray, who more or less built the theory behind that, opens his 1981 paper on it by saying the idea comes from contract law, and his example is a wedding. People negotiate, somebody asks, two people say "I do," and from then on it's binding. If it was a mistake, you don't erase it. You do another transaction to fix it, the way a bad contract gets a second contract.

The wedding was a philosopher's example first. J. L. Austin, lecturing at Harvard in 1955, said "I do" makes a marriage; it doesn't describe one, and sayings like that aren't true or false. They can misfire: the wrong person says it, a step gets skipped. A commit works the same way. Writing the commit record doesn't report that a question got closed. Writing it is the closing. It can't be false; it can misfire, by closing without the check, or about the wrong pocket. Then somebody else reads the record, and for them it turns into a report, "this was checked and closed," and a report can be false. That's how a commit gets tied to the truth: through a check, and no further than the check reaches.

Robin Milner, building a proof checker in the 1970s, ran out of memory, so he decided to keep only the finished theorems and throw the proofs away, like a teacher rubbing out the blackboard to make room. The obvious worry is forgery: if the proofs are gone, what stops someone from just writing a theorem down? His answer was beautiful. Theorems got their own type in the programming language, and the only way to make one was through the rules of logic. You couldn't forge one because the language wouldn't let you build it any other way. He named that language ML, for Meta Language. I wanted to call my machine the meta machine. I'm choosing to treat that as a coincidence.

Bitcoin keeps a history that's expensive to replace. A node that comes back online checks the candidate chains against its rules and follows the valid one with the most work behind it. Replacing an old block means redoing the work that ties every later block to it. That's protection under assumptions about the attacker and the network, and a recent block can still be undone. And the work is tied to something scarce, electricity and hardware. A unit of cost in this book is tied to nothing. The economist Eric Budish showed in 2025 what that tie has to cost: for Bitcoin's kind of trust to hold, the honest work has to stay expensive at every moment compared with what an attacker could win. A flat charge of one unit can't do that, and isn't trying to. It's a count, not a lock. The lock is the rule that the close waits for the check, and it would work at a price of zero. The price is there because, on a finite machine, keeping the answer makes two arrows meet.

And Bitcoin wasn't first. Pull on its footnotes and three of the eight references in the paper that started it point to Stuart Haber and Scott Stornetta, who in 1991 worked out how to stamp a document with a time nobody could fake later, by chaining each stamp to the ones before. A company built on their idea, Surety, printed a short fingerprint of its chain in the classified ads of the New York Times once a week from 1995, and was still doing it in 2018, under "Notices & Lost and Found," according to a piece the journalist Daniel Oberhaus wrote that year. To make a record nobody could quietly rewrite, they put it somewhere a million people would keep a copy without meaning to.

All of those work. I lean on all of them. The next thing isn't "they're broken." It's where they sit. The database's commit is bits in a log, kept by a database program, on a machine whose own step will overwrite those bits the moment some instruction says to. Milner's guarantee holds inside programs written in his language and nowhere else. Bitcoin is a set of rules running on ordinary computers that do whatever they're told. Each one is a program or a rule sitting on top of a machine that can't tell the two computers apart.

The obvious answer: put the rule one level lower, in the piece of the system every action has to pass through. Security people call that a reference monitor. James Anderson described it in a 1972 planning study for the US Air Force: a piece every

access passes through, that can't be tampered with, and small enough to check. Fred Schneider argued in 2000 that a monitor that only watches a run as it goes can enforce exactly the rules whose breaking shows up at some point you can name, and "no close without an earlier passing check of the same claim" is one of those. And Arthur Azevedo de Amorim and six coauthors built monitors in 2015 that check a rule at every single instruction, and proved them correct. Putting a rule under every program, so nothing can get around it, is their move, not mine. A monitor that wouldn't let any program raise its record without paying and without the check first would be one way to build what this book describes. That's an implementation, not the thing. What I'm putting under that move is narrower: a record that has to be paid for, that has to be earned, and a reason from counting why the price of a record that never comes down can't be zero on a finite machine.

8.11 The two computers, put back together

Here's the two-computer story again, rebuilt as Thiele machines, with the record and a running count written out. The record here is established: it goes up only right after a check of the claim passes. The prices are one to check, one to raise the record, and nothing for ordinary work.

Why does the check cost anything, if effort never counts? Not for the work. The check leaves a mark saying it passed, so the raise can see it, and you've already drawn what that mark does: "true and unchecked" and "true and checked" both land on "true and checked." Two arrows meet, so Bennett's rule charges it. The raise costs one because that's the toll. And the copier's raise costs one even though it's refused, because the price is on the button, the same as the stamp pressed on YES in chapter four.

step	sorter does	record	count	copier does	record	count
start	nothing yet	no	0	nothing yet	no	0
1	sorts (ordinary steps)	no	0	copies in the typed list	no	0
2	checks "in order" on this list: passes	no	1	(no check)	no	0
3	raises the record	yes	2	tries to raise the record: refused	no	1

At the end, the lists are still the same. Everything else differs. The sorter's record is up and its count reads 2. The copier's record is down, and it paid 1 for the attempt, because an attempt costs what it costs. Nothing the copier can do raises its record without first running a passing check of the claim. And anything that lets through only the list can't tell you which is which. Walk the warehouse now and there's finally a lamp that's lit in one and dark in the other.

How literally should you take that table? It's a picture of the condition, and that condition is the whole of what established means. What it needs from the question is a checker that says yes exactly when the answer is yes. Without one, the record is honest about whatever the checker reads, and only by convention about your list. That's Gettier's pocket again.

And a lamp for whom? The record is bits too. It's a lamp in the machine, for anybody walking that warehouse. Print it out and hand it across a desk, and you're back at the sticker on the windshield: the person reading it has to trust whoever handed it over, or have some way to check the box that says it ran. That's the last chapter's problem, and this book doesn't solve it. What changes is the question, from "did it run?" to "do I trust the box that says it ran?" That's a smaller question. It isn't no question.

So, back to the list from the start of the chapter. The windshield sticker is about whether a piece of paper is genuine, and nothing here touches it. The "all tests passed" sheet fits only if the test is a checker proved to mean what it says; a set of tests that can miss a bug establishes "this procedure said yes," which is true and smaller. An audit or a ledger kept by a program on an ordinary computer is bits the program wrote; it fits only where the step underneath keeps the record and refuses to raise it without the check. A proof is one example: checking its steps is a check that can say no, though whether its statement says what you meant is still yours to read. The AI that sounds sure of itself isn't touched at all.

Now the honest half. Take away the rule that the raise waits for the check, and keep only the bare toll from page one, and the copier has an easier move: pay one, raise the record, done. The toll is satisfied and the receipt is real, but it's a receipt for nothing. The bare toll prices the closing of the door. The check behind it goes unpriced and unrequired. That's why established is a condition you add, not something the definition hands you.

8.12 What putting it in the step buys

The hardest objection is one I'd rather make myself. You defined a mark and a charge, then showed the charge gets paid when the mark goes up. That's your definitions agreeing with each other.

For the bare machine, that's fair, and the copier's receipt for nothing is the proof. What makes the record mean established is the condition that it goes up only after a check of that very claim passed, meeting the first three of the four conditions, with the fourth, an honest checker, a fact about the checker. And the condition isn't about one program. On every run from a clean start, if the record is up, then somewhere earlier a check of the claim passed, with nothing touching the thing between the check and the raise.

So a raised record, from a clean start, tells you a claim went through a passing check before the raise. That piece of history is what it keeps, and only that. It doesn't say the claim still holds at the end; the thing it's about can change after the raise. The record stays up about the version that was checked.

That "every run" is what a rule on every step buys. A log written by one program is only as good as that program. A rule the step itself enforces covers every run, hostile programs included. There are two limits on that "every," and I'll say them now so you don't have to find them later. It's every run, not every everything: from a clean start, one step at a time, on a step that does what its definition says. A hostile start, a soldering iron on the record's bit, or a hostile reader is outside it. And on their own, the record and the count are what people who prove programs correct call ghost state: erase them and every program computes exactly what it did before. What isn't ghost is whatever the step reads before it lets the record rise: the check in front of it.

A computer that runs a Thiele machine, your laptop say, isn't that machine. It implements it, inside bounds, the way a laptop implements a Turing machine. Turing's tape never runs out, and a laptop's memory does; strictly, a laptop is a very big finite machine, and it acts like a Turing machine only on the computations that fit. Nobody minds, because the bound is known and most of what people compute fits. That's the same deal any device running a Thiele machine gets. The bounds aren't fine print. They're the whole difference between running a model and being one.

How much can a difference in size matter? On 4 June 1996 the first Ariane 5 rocket lifted off and broke apart about forty seconds later. The inquiry board, chaired by the mathematician Jacques-Louis Lions, traced it to one conversion in the guidance software: a 64-bit number, a value tied to horizontal speed, squeezed into a 16-bit whole number. On Ariane 4, where the software came from, that value could never get big enough to overflow, because of how that rocket flew. Ariane 5 flew differently. The value got too big, the backup unit failed on it, the main unit failed the same way a fraction of a second later, and the code doing the converting didn't even serve a purpose once the rocket was off the ground; it was kept so the software would match the old rocket's. The board's report says the assumption that made the conversion safe had been agreed on and then, without anyone meaning to, kept out of sight of any outside review. An assumption about size that holds on one machine is a fact about how that machine gets used. The code doesn't carry it, and yet it travels with the code without anybody noticing. The only defence I know is the dull one: write the assumption into every statement that needs it, where the next person will trip over it.

CHAPTER 9

Settled

The last chapter tied a record to a check: the record goes up only right after a check of that very claim passed. But I kept saying two things in one breath, the check and the closing, and they're not the same thing. This chapter pulls them apart, and then asks a question that bothered me the whole time I was writing the last one: is "established" really a switch, on or off? Or is it something with more than two places?

9.1 Settled, whether it's true or not

I ran into Charles Sanders Peirce again here, the American scientist and philosopher, in an 1877 essay called "The Fixation of Belief." He was asking what inquiry is for, and his answer named the thing I'd been running together without noticing. The point of inquiry, he says, is to get out of doubt and settle on a belief. And when someone objects that surely what people want is a true belief, he answers like a guy who's watched a lot of arguments end: "as soon as a firm belief is reached we are entirely satisfied, whether the belief be true or false."

That's funny, and then it isn't, because he's right. Settling and being right are two different events. A question can be closed on a wrong answer, and once it's closed it acts closed. Peirce cared a great deal about which ways of settling hold up; science wins, for him, because it has to answer to something outside any one person. But the settling itself, the question snapping shut, happens every time, and it never comes with a guarantee.

Karl Popper came at it from the other side. He was a philosopher from Vienna asking what separates science from everything else, and in his 1934 book, the one that came into English as The Logic of Scientific Discovery, he says every test has to stop somewhere, at some statement somebody decides to accept, and nothing in the evidence forces you to stop there instead of one step further. His picture is a jury: the verdict is a decision about what happened, and the jury deciding it doesn't make it true. Science, for him, is a building on piles driven into a swamp, and the piles stop where somebody's satisfied they'll hold. They never reach solid ground.

A real jury has a problem Popper doesn't mention, because a real jury is twelve people. Lewis Kornhauser and Lawrence Sager, two law professors, noticed in 1986 that a court that votes by majority on each part of a case, and also on the verdict, can end up contradicting itself, even when every judge is consistent. Christian List and Philip Pettit turned that into a theorem in 2002: no way of combining people's yes-or-no judgments on connected questions meets a short list of fair-looking conditions and always hands back a consistent answer for the group.

Try it with three ballots. The rule is: a payment is owed only if a contract was signed and a deadline was missed. One voter says yes to both. One says signed but not missed. One says missed but not signed. Each voter applies the rule consistently: one says owed, two say not owed. Now count the questions separately. A majority says signed. A majority says missed. And a majority says not owed. You can point at the contradiction on the paper without blaming any one voter's logic. A group's settling is a rule's settling, and which rule you pick decides which honest settling sticks. I haven't tried to put a group on the machine, and that's where I'd expect the first wall.

Popper's swamp goes further down, because it's the oldest problem there is. Ask why you believe something and you get a reason. Ask why you believe the reason and you get another one. Sextus Empiricus, a skeptic and doctor who probably lived in the second or third century, wrote down the ancient version, credited to someone called Agrippa, and in 1968 the German philosopher Hans Albert gave it a name with a joke in it: the Münchhausen trilemma, after the baron who claimed he pulled himself out of a swamp by his own hair. Every chain of reasons does one of three things. It goes on forever. It loops back on itself. Or it stops somewhere, because somebody decided it stops.

It's the same shape as chapter one's step law, with reasons in place of states. Follow a chain: it keeps finding new reasons, or it comes back to one it already used, or it stops. There's no fourth thing. So "when am I done checking?" doesn't get a clean answer from anybody. What you can do is be honest about where you stopped and why, and that's a lot of what this book is: every place it stops is written down, with its name on it, so you can decide whether you'd have stopped there too.

Lay Peirce and Popper on top of each other and what comes through is that the check is one act and the stopping is another. The check can run as long as you like and it never makes you stop. At some point something decides, and the deciding is its own event. That's the copier's whole trick from the last chapter: it skipped the check and kept the answer.

9.2 A proof is an act

Then a logician said the part I didn't expect a logician to say. Per Martin-Löf, a Swedish logician, in lectures he gave in Siena in 1983: "a proof is, not an object, but an act ... Only secondarily, and irrevocably, does it become the act that has been performed." That took me a few reads, and the tenses opened it up: first the act being done, then, for good, the act that has been done.

You could say that's true of everything that ever happened. It's the two computers again: the sorter's check happened, for good, and the machine right now carries no trace of it. The past can't be undone. The record of it can be lost, unless something keeps one. That gap is where this book lives.

Then I found the people who had already built a record like that, on paper, and it came clear only when I sorted them by three words: the act, the thing it produces, and the trace the act leaves behind.

The Dutch mathematician L. E. J. Brouwer held that a mathematical truth is something a mind makes evident, and from 1948 on he argued about an idealised mathematician working in stages, the creating subject. The logician Georg Kreisel, in 1967, wrote it as a formula: the subject has evidence for a statement at stage n. That's a trace, something you can come back and ask about after the act is over. Anne Troelstra, a Dutch logician, added in 1969 that the subject never forgets what it has made evident, so the trace never fades. That's how Mark van Atten credits it in his 2018 history; the Stanford Encyclopedia's entry lists the never-forgetting rule among Kreisel's own, and I haven't settled which is right. Van Atten reads the formula as "by stage n the creating subject has made it evident." Look at the tense: has made. That's Martin-Löf's "the act that has been performed," reached from the other end. Van Atten even says the subject can't just accept a proof handed to it from outside; it has to go through the steps and rebuild the evidence itself, which is the copier who runs the check. And Göran Sundholm, a philosopher of logic at Leiden, has long kept the proof as an act apart from the proof as a thing, and in 2014 worked through how Brouwer's creating subject sits beside the idea that every procedure is a program. I know that paper only by what it sets out to do, so I lean on it lightly.

Set their trace next to the record this book wants. Theirs happens at a stage; mine goes up at a definite step in the run. Theirs never forgets; mine never goes back. Theirs is never wrong; mine is earned. Nearly every column fills. The one that stays empty is the price, and I think I see why. An idealised mind with unlimited memory pays for never forgetting with fresh paper, and an idealised mind has endless fresh paper, so nobody charges it. On a finite machine the fresh paper runs out, and keeping the record makes two arrows meet. That's chapter two. So theirs is the closest thing to established I've found, and it's been around for sixty years.

Push on mine from their side and it gives twice. Van Atten's subject won't take a proof from outside, and my machine takes exactly one: its checker is trusted on a proof about the checker, done once, outside every run. And their evidence is never wrong because, for Brouwer, made evident is what true means. Mine is right only as far as the checker is honest.

9.3 Not a switch

So far I've talked about established as if it were a light switch: on or off, the sorter on and the copier off. That bothered me the whole time I was writing it, and for a while I couldn't say why. It felt too small. I don't trust that feeling on its own, because I've had it plenty of times about things that turned out to be nothing. So I went looking for something outside my own head that would either give the feeling something to stand on or show me I was imagining it.

I didn't know what I was looking for at first, beyond a rough shape: two things that look exactly the same and differ in how far something about them has been settled. So I started collecting anything that seemed to have that shape. Most of it fell apart as soon as I looked closely, and how each one fell apart taught me more than the thing itself did.

The first batch was all hidden things. A pea under one of three cups. A tree, whose age you can't see from outside. A forged painting that looks right until somebody tests the paint. A glass of gas, where the thermometer gives one reading and underneath there are endlessly many arrangements of molecules that would all give it. Plato's prisoners. Every one looked promising for about a minute. Then I noticed they all fail the same way: look harder and the hidden thing shows up. Lift the cup. Cut the trunk and count the rings. Test the paint. Turn around in the cave. The hidden part is a fact about the object that better eyes could find. That isn't what the two computers have. You can see every lamp in the warehouse and not find the check.

That gave me a crude test. Give someone perfect vision of the thing in front of them, as much detail as they want. Could they read the difference off it? If yes, it's just hidden. If no, it might be what I'm after. The test is a judgment, and I'm the one making it, so it's a way of sorting my own examples, not a verdict.

The second batch passed that test and failed differently. A clock face reads three o'clock and doesn't show how many days have gone by. And a computer can keep a complete history of what it did: that's Bennett's machine from chapter two, which writes down every step, copies out its answer, then runs itself backwards, wiping the history as it goes, until it holds only the question and the answer, with no two arrows ever meeting. The history it kept along the way was right out in the open. But it could be run back down. If the thing I was after could be put up and taken

down with nothing paid either way, it would just be bookkeeping. So I needed something that only moves one way, and costs something to move.

The third batch came closer. In 1782 James Six, a Canterbury man from a family of silk weavers who had taken up watching the weather, described a thermometer for the Royal Society. He wanted the coldest moment of every night, and he'd found that with an ordinary thermometer you had to be looking when it happened, because, in his words, "if it be not immediately noticed, it can never after be known." His fix was two little steel markers in the tube that the column pushes along and that stay where they're left. A record that only moves one way. Then he spoils it, for my purposes, in the same paper: every evening he wrote the numbers down and slid the markers back with a magnet, "without a touch."

Since 1996, starting with polio vaccine, vials bought through UNICEF have carried a little sticker, a square inside a circle. The square darkens as the vial takes on heat, and it doesn't lighten again when the vial cools. When the square matches the circle, don't use the vaccine. It only moves one way, and it can't be reset. But it's chemistry: the reaction in the square is chosen to run about as fast as the vaccine itself spoils. Nobody ran a check. The heat did.

Closer still: the international vocabulary that measurement labs work from says a result is traceable only if it can be related to a reference "through a documented unbroken chain of calibrations." Standing earned by checks, step by step. The same document adds that traceability "does not ensure . . . that there is an absence of mistakes," which is "settled isn't the same as true," written into an international standard. And the top of the chain wasn't fixed either. When the official copies of the kilogram were compared with the original in 1889, 1946 and around 1991, they had drifted apart by tens of micrograms over the century, and nobody can say which one moved, because the original was the kilogram. In 2019 the kilogram was redefined by fixing a constant of nature instead.

The rules of chess have the shape too. The laws of chess say two positions are the same only if the same player has the move, the same pieces are on the same squares, and the possible moves of all the pieces are the same. They spell out that a king that could still castle and one that can't are in different positions, even on an identical board. That passes the perfect-vision test. And the rules say why: the castling right changes which moves are legal. The hidden part steers the game.

Each of those got something right and missed something else.

9.4 How a statement gets established

The one that fit best, among the ones I found, was the standing of a mathematical statement. Here's how it went for one statement, as far as I could follow it.

In 1611 Johannes Kepler, the emperor's mathematician in Prague, owed his friend and patron Johann Matthäus Wackher von Wackenfels a New Year's present and, he said, had nothing to give. Wackher, Kepler wrote, loved nothing, for the games you can play with the word, so Kepler wrote him a little pamphlet about the nearest thing to nothing he could find, a snowflake, which had landed on his coat as he crossed a bridge. Why six corners? Kepler couldn't say in the end. But along the way he stacked equal balls in his head and said that one way of stacking them, the way grocers stack oranges, packs them so tightly that no other arrangement could fit more into the same box. He didn't prove it. He said it.

Then the sentence sat. In 1831 Carl Friedrich Gauss proved a special case. In 1900 David Hilbert put the general question on his famous list of problems for the new century, without mentioning Kepler. In 1953 the Hungarian geometer László Fejes Tóth showed how the whole thing might be cut down to a problem with finitely many numbers in it, and guessed that computers might one day finish it. Thomas Hales started on it in earnest in 1994, and in August 1998 he and his student Samuel Ferguson announced a proof that leaned on a great deal of computer work. The journal gave it to a panel of twelve referees. They ran seminars on it for years. A journalist, George Szpiro, later reported the editor's letter as Hales described it: the referees had "not been able to certify the correctness of the proof, and will not be able to certify it in the future, because they have run out of energy." The journal printed the human part in 2005 and the whole proof in 2006. Hales started a project in 2003 to check every single step by machine. It finished on 10 August 2014.

Lay those dates out on the table and look at what moved. The sentence never changed. Whether it's true never changed, as far as anyone can tell. What changed was how far it had been established, and it changed in stages: a guess, a special case, a place on a list, a plan, an announcement, a panel that ran out of energy, a printing, a machine check.

Then I tried to break that one too, because by now I'd watched a lot of good examples fall over, and there's an obvious place to push. Standing can fall.

In 1852 a student named Francis Guthrie, colouring a map of the counties of England, noticed that four colours always seemed to be enough to keep neighbouring counties different. The question went to his old teacher Augustus De Morgan, who wrote to the Irish mathematician William Rowan Hamilton that a student had asked him for "a reason for a fact which I did not know was a fact," and that he didn't know it yet either. In 1879 Alfred Kempe, a lawyer who did mathematics on the side and sang counter-tenor, published a proof, and his description of finding it is wonderful: "After a somewhat arduous search, I have succeeded, suddenly, as might be expected, in hitting upon the weak point." The journal's associate editor, William Story, added a note saying Kempe had "substantially proved" it. Eleven years later Percy Heawood, a lecturer at Durham who by one account brought his dog to his

lectures, showed that one step of "the now apparently recognised proof" doesn't work.

So standing can fall. And then the part I didn't expect. The four-colour statement was true all along. Kenneth Appel and Wolfgang Haken proved it in 1976 with a computer, and the proof was met with suspicion for years; a cleaner version came in the 1990s; and in 2005 Georges Gonthier finished a proof checked step by step by a small proof-checking program. So one statement stayed true the entire time while its standing went up in 1879, came down in 1890, went up again in 1976 against a lot of doubt, and went up further in 2005. The fall wasn't a fall of the truth. It was a fall of how far the truth had been established, and it was the right fall, because what had been established was less than anyone thought.

Laying all of that out together is what changed my picture. If there are stages, then at each one there's a line, with two sides: not this far yet, and at least this far. Each line is its own yes-or-no question. But pick the question carefully. "Do I still trust this proof?" can turn back to no, the way Kempe's did. "Did I carry out this particular check?" is about something that happened. Cross out the trust and the check still happened. To keep that answer, the machine has to keep a record of it and have no move that knocks the record down.

So maybe the switch wasn't wrong. Maybe it was just the first line, and the real shape is a direction with lines drawn across it, which is the row of cups from chapter five. I've gone around many times on whether the lines make the direction or the direction comes first and the lines are just where you cut it. I don't have an answer. The part I could actually settle is the part chapter five did: one switch isn't enough once there are three or more stages.

9.5 What settling looks like

Here's the picture, put together.

Draw every state the machine could be in as a field of dots. A yes-or-no question about the state splits the field in two: yes and no. That's a sorting, the smallest one there is. If one side is empty, there's no sorting at all, just a field, which is the empty no pile from the last chapter. Now the extra thing that makes it settling: once the machine is on the yes side, no step it can take leads back out. The door only swings in. A settled question is a sorting plus a door that only swings one way.

At its best, what goes through the door is a relation: these numbers are in this order, these two things belong together. But the door doesn't care. Hook it to a check that says yes to everything and the machine walks through even when the claim is false. The question is settled either way, true or false, like Peirce said. A door that only swings one way tells you a question was closed. It doesn't tell you

it was closed right. Price and correctness are two different books, and I keep both, separately.

And one more edge, the one Gray's wedding already hinted at. What stays shut for good is that the question got closed, about that version of that thing. Whether the claim is still true is another matter. Change the list after the commit and the record still says, truthfully, that the old version was checked and closed. If the claim turns out bad, the fix is another act on top, a second record that says "withdrawn." The first one never quietly comes back down. So "I can lean on it right now" is three things at once: closed, not withdrawn since, and the thing unchanged.

CHAPTER 10

What structure is

The chapters so far were about one thing settling: a coin going into YES, a cup turning up, a box getting ticked. This chapter goes after the bigger word behind all of it, and I'll give it to you now, so you can decide whether to stay: structure.

I used that word for most of my life without once looking at it, which I suspect is true of most people and most words, and it's fine, right up until you do something stupid. The stupid thing I did was build a machine that charges for keeping something. You can arrange things however you like, and the machine lets you put any price on the arranging, including none. But the moment an arrangement has to stay, the moment you ask the machine to keep it, or to vouch for it, to put its own name on the claim that these things belong together, there's a charge, and it isn't zero. Chapters two and three showed why. What was in front of me, standing there with the thing built, was a model that charges for keeping structure, and I could not have told you what structure was. It didn't care. It can't nod along the way a person does. It needed the word to hold still, and the word had never held still for anyone, least of all me.

So I went looking at the people who'd wrestled it before me. Some of them have been dead two thousand years and are still, I'd bet money, better at this than I am. I used everything I had: language models, more than one, books, libraries, my own gut, anything in reach. I don't know if I found enough. I never do. There's always another shelf behind the one I was reading. And being dead and famous is not the same as being right, so I pushed on each of them, and what follows is what I could reach, pushed on as hard as I know how. The one thing I'm asking is that you push on it with me, out loud, at your own kitchen table, because the first time through I was the only one checking, and that's not enough people.

The order I ran into them was a mess. Category theory came first, because I'd built one by feel before I knew it had a name, and everything else came in whatever order the next question dragged it in. I'm going to put them in the order where each one leans on the last, which is mostly the order they happened. And none of it is an oracle. History is where I start digging. It gets retold, smoothed over, and now and

then made up outright (there's a drowning coming up that nobody can place), so every piece of it is a lead to check, and I check the parts I lean on.

10.1 A string, a square, and a drowning nobody can place

Start with the oldest one, and start by doing it. Take a string. A guitar string, a rubber band over an open box, anything that twangs. Pluck it. Keep its tension as nearly unchanged as you can when you stop it at a new point; stretching it tighter changes the pitch for a second reason. Now stop it exactly at the middle and pluck half. With the tension held steady, that's the same note an octave up, the one that sounds like the same note sung higher. Press at a third of the way along and pluck the long side: that's the fifth, one of the handful of intervals a bugle can play at all, which is why you hear it all through bugle calls. Half, two thirds: small whole numbers, and your ear says yes before your brain does. About twenty-five hundred years ago, in the south of Italy, the followers of a man named Pythagoras are credited with finding exactly that, and it's as true in your kitchen tonight as it was on their hillside.

I'll tell you what they did with it, as best anyone can tell. Nothing survives that's surely from Pythagoras's own hand. The earliest fragments are from a later member, Philolaus, and scholars argue over those. Mostly what's left is Aristotle, one to two centuries later, writing in his Metaphysics about them the way you'd write about the neighbours' religion, and he says they took the elements of numbers to be the elements of all things: everything is number. That doesn't mean everything can be counted; it means the parts of numbers are the parts of all things.

Stop and ask what I asked. What does that even mean? Is a chair a number? Does a chair have a number tucked inside it somewhere? Aristotle doesn't say, and he's reporting a belief he doesn't hold. So don't take it on his word, or theirs, or mine. But before you push on it, know what it was holding up, because you can't test a load-bearing wall without knowing what sits on it. These weren't people doing math for fun on weekends. They were a brotherhood, with rules about what you could eat and when you had to keep quiet, and that plucked string was their scripture. Halve the string, hear the octave: to them that was proof that under the mess of the world there were whole numbers in clean ratios, that the cosmos was a chord and they'd heard it. "Everything is number" wasn't a slogan; it was what they'd organized their lives around. And that's exactly the kind of sentence you should be suspicious of, mine included.

Now push, and push where they loved most: the square. Draw one, with a side one unit long, any unit you like, and draw the line from one corner to the opposite corner. Call that the diagonal. If everything is number, then the diagonal is some whole number of parts long when the side is some whole number of parts, so the

diagonal divided by the side is a fraction, a whole number over a whole number. Cancel anything the top and bottom share, the way four-sixths becomes two-thirds, until they have no common factor left. I'll call the top p and the bottom q, and the letters are purely so I don't have to keep writing "the top" and "the bottom."

You need one fact about squares, and it's the one Pythagoras is famous for. The square built on the diagonal has exactly twice the area of the square built on the side. If you want to see it for yourself, cut out two identical squares, cut the second one along both diagonals into four triangles, and lay one triangle against each side of the first, its long edge on that side. The outline is the square on the first one's diagonal, and it's made of exactly two of the small squares; it's a five-minute job with scissors. So p times p is twice q times q. People write "times itself" as a small raised two, so that's $p^2 = 2q^2$, and that is the only piece of notation in this argument.

Twice anything is even. So p times p is even. An odd number times itself is always odd: draw that many rows of that many dots, pair off the columns, then pair the dots in the one column left over, and one dot remains. The same pairing works for any odd side, so if p were odd, p times p would be odd too. It isn't, so p itself must be even: it's two times something, and I'll call that something r. Put it back in. Two r times two r is four times r times r, and that equals twice q times q. Halve both sides: two times r times r equals q times q. So q times q is even, so q is even too, by the same step as before. But you already cancelled everything the top and bottom had in common, so they can't both be even. The argument has eaten itself, and when an argument eats itself the thing you supposed at the start is false. There is no such fraction. The diagonal of a square can't be named by any ratio of whole numbers, ever. Aristotle reaches for this very argument in his Prior Analytics as his example of proof by contradiction (suppose the diagonal can be measured, and odd numbers come out equal to even ones), and you just did it with a pencil.

There's a story that a member of the brotherhood named Hippasus let this secret out, and his brothers took him to sea and drowned him for it. I went and checked, because it's a good story and good stories are the ones to check. The oldest drownings I could find are in Iamblichus, writing around AD 300, seven or eight hundred years after Hippasus. In Iamblichus the man drowned by name is Hippasus, and what he gave away was how to fit twelve pentagons together into a ball, the dodecahedron. A few pages on, Iamblichus tells it again about whoever first let out the incommensurable, and that time he names nobody. Carl Huffman, who wrote the Stanford Encyclopedia of Philosophy's entry on Pythagoreanism, follows the historian Walter Burkert there and finds no ancient writer who connects Hippasus to this discovery at all. So the version with Hippasus drowned for the diagonal looks like two old stories stitched together later. It's a legend, but notice that the legend exists. Somebody, long after, felt that a man should have been drowned for

this. That's how much was riding on it. To whoever stitched the story together, the diagonal wasn't a math problem; it was the end of the world.

So the claim is dead, and you killed it. I only handed you the pencil. Now the question that matters: what's left? Look at your square again. The diagonal is still there. It still stands in a perfectly definite relation to the side; you drew it, and nobody drowned. That relation was real before anyone tried to express it as a fraction, and it stayed real after that attempt failed. The relation came first. The number, when there is one, is a name for the relation, and sometimes there isn't a name. I'm going to keep that, and I want you to keep it too, because you're going to watch the same move get made again and again as this book goes on: somebody says "everything is X," and it's true enough to do real work with, and it sounds right when you pluck it, and then somebody draws the diagonal. Whether the thing I built survives its own diagonal is a real question, and a later chapter answers it.

10.2 Shapes poured into nothing

About a century and a half after the brotherhood, Plato, in a dialogue called the Timaeus, tried to build the whole physical world out of geometry. Fire, air, water and earth were four regular solids (the pyramid, the eight-faced one, the twenty-faced one, and the cube), and the faces of those solids were built from exactly two kinds of right triangle. You can make one of each in a minute: cut a square from corner to corner for the first, and cut an equilateral triangle from a corner to the middle of the opposite side for the second. Everything else, in Plato's story, was a receptacle, a featureless medium that takes on shape the way wax takes a seal. Structure, for Plato, is shape laid into something that has no shape of its own.

Why did he care? He wanted a world made on purpose, by a craftsman working from a plan, and triangles were the plan. Is it true? No, but not for the cheap reason. You can't refute a claim about what things are made of at the smallest scale by picking one up; the smallest scale is exactly where your hands don't reach, and Plato knew that. What sank it is that it made no prediction anyone could check, and the people who eventually did reach that scale, two thousand years on, found atoms, and atoms aren't triangles. The idea was a guess with no way to lose, until it lost. But push on it and two things survive. One: structure is something laid in, and the stuff it's laid into doesn't carry it by itself. That's a real idea, and I'd ask you to keep it loosely for now; when you meet a record that's established, you can ask whether the state it holds is the wax or the seal. Two: Plato put a crack in his own forms himself, in another dialogue, the Parmenides, with an objection that Aristotle later nicknamed the third man, and it's the most useful mistake in this section. Take all the large things. Say they share one thing, Largeness. But Largeness is itself large, so now there's a further thing that all the large things and Largeness share, and

that one is large too, and it never stops. The extra move is treating Largeness itself as one more large thing. Grant that again at each stage and it never stops; refuse it and this particular regress never starts. The lesson, and I think it's right though Plato never spells it out: the structure of a collection is not one more member of the collection. The plan isn't a brick. Hold on to that one.

10.3 Prove it from what you and I both accept

Around 300 BC, in Alexandria, Euclid wrote the Elements, and the thing to take from it is not the geometry. It's the method, and the method is the whole reason I trust anything in this book, so give me a paragraph on it. He starts with definitions ("a point is that which has no part"), five postulates (things he asks you to grant, like "to draw a straight line from any point to any point"), and five common notions (things nobody argues with, like "things which are equal to the same thing are also equal to one another"). Then he derives everything else, four hundred and sixty-five propositions, from only those, in order, each meant to use nothing but what came before (he sometimes leans on things he never stated, and later geometers filled those in). He never says "trust me" or "it's well known"; he says, in effect, here are the rules you and I both accept, now watch. If you've ever been in an argument that ended with "well, that's just your opinion," Euclid is the man who found a way out of that room, and I'm not embarrassed to say I learned the way out from someone dead for two thousand three hundred years. Four hundred and sixty-five times he does the same trick, and the trick is everything.

And then there's the one rule Euclid seems almost embarrassed by, and it's a hole worth falling into. His fifth postulate, the one about parallel lines, is long and awkward next to the others, and for two thousand years people tried to prove it from the rest so it wouldn't have to be assumed. Proclus, the fifth-century philosopher who wrote a commentary on Euclid's first book, tried. Mathematicians in Persia and the Arab world tried. In 1733 an Italian priest named Girolamo Saccheri spent a whole book assuming it was false so he could watch the contradiction arrive, and he got pages deep into what is now a perfectly good geometry before declaring that it was repugnant and stopping. A century later three people, separately, took the step he wouldn't: drop the postulate, and what you get is other geometries that hold up just as well, and nonsense never arrives. Farkas Bolyai begged his son János to stay away from the problem, warning that it had swallowed years of his own life; the son went in anyway and came out with one of those geometries. The part I keep is that they found something real by refusing to believe one of the rules everyone accepted, and checking what happened without it. Refusing a rule and checking what happens without it is the push, and this one went on for two thousand years.

What is structure, to Euclid? Whatever the starting rules force. And when are two shapes the same? You can do this one yourself: cut out two paper triangles and lay one on the other. If they coincide, they're equal. That's his fourth common notion, and it's how he proves his fourth proposition, by picking up one triangle and setting it down on another. Sameness is coinciding under a move. Keep that; it's about to get very general, and I didn't see that coming either.

10.4 The statue and the bronze

Aristotle, Plato's student, pulled the forms down out of heaven and put them inside things, which was a bit rude to his teacher and also, I think, correct. His example, from the Metaphysics: a bronze statue is bronze plus a shape. And a circle can be made in bronze or stone or wood, so, he says, the bronze is no part of what it is to be a circle. The same shape in marble makes a different statue with the same form. The bronze is the matter and the shape is the form, and the form is what the thing is.

Push on it with two things from the kitchen. A cup can be ceramic or metal and still hold your coffee. A sandwich carved in wood can keep the outline and fail to be lunch. The two swaps preserve different things. If the question is shape, the carving may pass; if the question is whether you can eat it, it does not.

Aristotle already makes a distinction of this kind. In Book VII, chapter 11, he warns against treating a living animal just like the circle whose bronze can be changed: its parts must be in a condition to do their work. The sandwich is a way to make that difficulty visible, not a correction he never considered.

That is the bet I need for the record. Change the material, then check the operations and the answers again. If they still agree, the record has survived the change in the respects I promised. A later chapter states those promises over a chosen base. Neither theorem says that every material can do every job, or that the price in one material is the price in another.

10.5 Where things are, without numbers

In 1679 Gottfried Leibniz, who co-invented calculus and had opinions about everything, wrote to the physicist Christiaan Huygens asking for something that didn't exist: a way to reason about where things are directly, the way algebra reasons about how much, without ever laying down coordinates. He called it the analysis of situation. Try it yourself. Describe where the furniture is in the room you're sitting in without using a single number. "The couch is against the wall. The table is between the couch and the window. The lamp is next to the chair." You just did it, and what you used was relations between places and nothing else. Huygens wrote back that

November and was not impressed. It took roughly two hundred years for Leibniz's idea to become a field, and the field is called topology, and Huygens is remembered for other things.

What survives the push is the same thing that survived the square, coming at you from the other side. Position is a relation among places. The numbers you lay on top are a convenience, and they're a convenience you can take away and still have the room. I'll be honest about why this one landed for me: the thing I built keeps track of which things belong together, and I spent a long time thinking that was a list of numbers before I understood it was a relation that the numbers were merely naming. Leibniz had that in 1679. Huygens told him no. I'd like to think I'd have sided with Leibniz. I probably would have sided with the famous guy.

10.6 Naming a thing by saying when two are the same

In 1884, in a short book called The Foundations of Arithmetic, Gottlob Frege noticed something you can check in your head right now. "Line a is parallel to line b" and "the direction of a is the direction of b" say exactly the same thing. There is no fact in one that isn't in the other. But the second sentence has a new object in it, a direction, that the first one doesn't. Where did it come from? From the sameness. Once you can say when two things count as the same, you've earned the right to name the thing they share. Frege used exactly this move to say what a number is: the number of things in a pile is whatever every pile you can match up with it, one to one, has in common. The residue has teeth: a sameness relation lets you name a new object.

Now push, because Frege pushed too hard, and it cost him. In his formal system, Basic Law V treated value-ranges as objects and identified them exactly when the corresponding functions agreed everywhere. Together with his unrestricted comprehension principles, that let the self-membership construction in. The familiar set version is easier to tell: In June of 1902, a young British philosopher and logician, Bertrand Russell, wrote him a short letter. Consider the collection of all collections that are not members of themselves. Is it a member of itself? If it is, then by its own rule it isn't. If it isn't, then it qualifies, so it is. Do it in your head; it flips every time you look at it, like one of those drawings that's a vase and then two faces and then a vase. Frege's floor had a hole in it, and he knew it the day he opened the letter, and his second volume was already at the printer. One repair was Zermelo's 1908 axiom of separation: a property selects a subset of a set already given. His system also had other existence axioms, including power set and infinity; separation was not its only way to obtain a set. What it removed was unrestricted sorting of "everything." You sort a pile. I'll tell you now that this rule shows up again, later, in a place where I had to obey it or fall through the same hole, and I'll point at it when the book gets there.

Frege's reply is worth hearing, because it's the best example I know of how to lose. He wrote back within days. He didn't hide it or argue around it. He said, more or less, that arithmetic was tottering, and in an appendix to the volume that was about to come out he wrote that hardly anything worse can happen to a scientific writer than having a foundation give way right after the work is finished. Then he tried to fix it. If I'm wrong about something in this book, and I'd bet I am somewhere, that's how I'd like to take it.

10.7 Choose the changes, keep what survives

By 1872 there were too many geometries. Euclid's, and ones where parallel lines meet, and ones with no such thing as distance, and nobody could say what made them all geometry and kept them from being a pile of unrelated hobbies. Felix Klein, twenty-three years old and taking up a professorship at Erlangen, published an answer in a program paper that people still call the Erlangen program. Pick a set of allowed changes: rotations, stretches, whatever you decide. Then a geometry is the study of the properties that stay the same under those changes. Pick a different set of changes and you get a different geometry on the very same page. Euclid's, in Klein's reading, is the one where you may slide, turn and flip a shape and scale it evenly, and stretching it more one way than another is off limits. And I'd underline, if this were my book, which it is, that in Klein's scheme any group of changes can define a geometry. The choice is yours. Nobody tells you which changes to allow.

You can do this one at your desk. Take any drawing and ask what's still true after you turn it upside down (lengths, angles), after you stretch it (lengths are gone by then; which lines cross which is still there), after you bend and crumple the sheet without tearing it (which drawn paths remain connected survives). Two distant ink marks may touch across a fold; that new physical contact is not a new connection in the drawing. You have to say which relation you are following. The three changes give three answers and three structures, all from one drawing. The residue is that structure is what survives a chosen set of changes, and "chosen" is doing real work in that sentence. There's no structure of a thing from nowhere. You have to say what you're allowed to do to it first. I didn't know that when I started, and I'd been saying "the structure of" things for years.

Klein's move has a descendant in logic, and it goes all the way. In a lecture in London in 1966, published only in 1986, three years after his death, the Polish-American logician Alfred Tarski asked what makes a notion logical, and answered with Klein's recipe at its far end: allow every one-to-one transformation of the world onto itself, and call logical the notions that survive all of them. The more changes you allow, the fewer notions are left standing, and with every change allowed what's

left is logic. Tarski's answer is the sentence above, "structure is what survives a chosen set of changes," with the choice made as large as it can be.

10.8 An invariance comes with a conserved quantity

In 1918 Emmy Noether, who at the time wasn't allowed to qualify for a university teaching post because she was a woman, and for a while taught courses advertised under David Hilbert's name, proved something that physicists still treat as close to holy. Take a physical system whose laws can be written the way most of physics is written, as "of all the paths the system could take, it takes one that makes a certain total stationary" (the total is called the action, and stationary means a small change to the path doesn't change the total to first order; the path needn't make it smallest, and that's all I'll say about it here). For such a system, every continuous symmetry that leaves the action unchanged comes with a quantity that is conserved. If the action doesn't change when you shift everything in space, momentum is conserved. If it doesn't change when you shift everything in time, energy is. A symmetry isn't decoration. It has a consequence downstream that you can measure with instruments. I can't hand you this one to do at the table; the proof needs calculus I'd have to build first and I'd lose you and probably me, and the "written as an action" condition is doing real work that I'm not going to unpack here. What I can hand you is the shape. Structure, once it's really there and the setting is the right kind, has consequences you can measure. That's the sentence that made me think a machine could carry a cost for structure in the first place. Noether makes it reasonable to ask. She doesn't make it true, and nothing I build in this book is a system with an action.

10.9 All of mathematics as structures

Starting in the 1930s, a group of young French mathematicians began publishing a vast treatise under one made-up name, Nicolas Bourbaki, as if he were a single person. He wasn't. In 1948 he also applied for membership in the American Mathematical Society, and the application was quietly turned down, which I find comforting. (The historian Michael Barany has gone through that application, a second one a year later, and the letters they set off.) Their goal was to rebuild all of mathematics on one idea: a structure is a set of elements "whose nature has not been specified," together with one or more relations on them that satisfy stated axioms. They believed three kinds of structure were the mothers of all the rest: algebraic, order, and topological.

Push on it. The historian Leo Corry went through the entire treatise looking for the place where this formal definition of structure actually does any work, and what

he found is that it "plays no actual role in the presentation of theories within the treatise." Outside the chapter that defines it, it does work almost nowhere. And I found no argument in their writing for why the mother structures number three. And the chapter on category theory, which they promised their readers, never appeared; Corry is where I found that too, and Ralf Krömer's 2006 history of Bourbaki and categories in the 1950s follows the attempt. That does not mean Bourbaki left maps out. Their Theory of Sets has sections on morphisms, derived structures and universal mappings (Chapter IV, sections 2 and 3). Corry's criticism concerns how much their general formal apparatus was actually used. The lesson I can take for my own work is smaller: naming a structure is not enough; show which constructions and comparisons it supports. I learned that one the slow way, and the afterword tells it.

Philosophers of mathematics took the same idea further and gave it a name, structuralism. In 1965 the philosopher Paul Benacerraf pointed out that the whole numbers can be built out of sets in two equally good ways that disagree about which set the number three is, and concluded that numbers aren't objects at all: what arithmetic is about is the pattern, and anything standing in the right relations to everything else can play the part of three. The philosophers Michael Resnik (Mathematics as a Science of Patterns) and Stewart Shapiro (Philosophy of Mathematics: Structure and Ontology), both in 1997, made the patterns themselves what mathematics is about. The philosopher Geoffrey Hellman (Mathematics without Numbers, 1989) kept the talk of patterns and did without the patterns, reading each statement as a claim about what would hold in any system with the right structure, if there were one. The logician Steve Awodey (1996) argued that category theory says it best, a structure being what its maps keep, which is where this section goes next. And the philosopher of science James Ladyman (1998) carried the idea into physics: what a successful theory gets right about the world is its structure. This section's thesis, that structure is the relations and the things in them don't matter, is theirs. None of the formal results takes their arguments as a premise, and the bibliography lists them under the same heading as Bourbaki.

10.10 A set with some named relations

What mathematical logic eventually settled on, with the logician Alfred Tarski giving the field its name, "the theory of models," in 1954, is almost boring, and the boring ones are the ones that last. A structure is a pile of things plus whatever relations a chosen vocabulary names on it. Pick the vocabulary (Klein again: you choose), and a structure is anything that gives those names a meaning.

Now I want to say "the smallest one," and "smallest" means nothing until I say what I'm measuring. So here's my ruler: count the names first; when that ties, count

how many things each name looks at at once; when that ties too, count how many answers each name can give. The bottom of that ruler, for anything that sorts at all, is one name, looking at one thing at a time, with two possible answers: one yes-or-no question, put to each thing in the pile.

A relation like "these two go together" can sort a pile too, but it looks at two things at once. A label for each thing can sort a pile, but it can give more than two answers. So the ruler puts both above one question. Two limits, so you don't have to find them. A single named thing, "this one, here," sits even lower on the ruler, but by itself it doesn't sort anything; ask "is it this one?" and you've got a split in two again, one question whose yes is a single thing. And a question only sorts when some things say yes and some say no. If they all agree, there's one group and nothing got sorted.

With those said: by this ruler, the smallest structure that sorts is one question. A different ruler could say something different. This is the one I chose, and I'm telling you so.

And notice one more thing about it. A yes-or-no question carries one bit more than a plain split into two groups: which group gets called yes. That bit is a direction across the split, a door that only swings one way. Keep it. It's about to turn into the whole chapter.

10.11 The structure is in the arrows

I met this one first, and backwards. I'd built it by feel, in a rendering engine, as "digital DNA," and only went looking for its name after the thing already ran. In 1945 two mathematicians, Samuel Eilenberg and Saunders Mac Lane, published a paper that founded a field now called category theory, and they said in plain words that it was a continuation of Klein, which I'd have loved to know at the time. The move they made was to stop asking what a thing is, and ask how it's carried. A category is some objects and some arrows between them, where arrows can be followed one after another and every object has a do-nothing arrow to itself, plus two rules that make the following dependable: following three arrows in a row gives the same result whether you pair up the first two or the last two, and the do-nothing arrow really does nothing when you tack it onto either end, and that's the whole definition. On the page the two rules look like nothing, and they're the part people skip, so slow down on them for a second, because each one is there for a reason you can see.

Picture three dots in a row, A, B and C, an arrow f hopping from the first to the second and an arrow g from the second to the third. Composition says there's an arrow taking the whole stride from A to C in one, g after f. Without it, every multi-hop path has to be named on its own, and the structure falls apart into a pile

of unconnected pairs, a phone book without a search function. Associativity says that with a third arrow h on the end, pairing up the first two and then adding h gives the same arrow as adding h to the last two first. Both spell the same three-hop path. With your paper triangle, bracket "slide, then turn" before "flip," or bracket "turn, then flip" after "slide": you still carry out the same three moves in the same order. Swapping the turn and the flip would be a different question. If the two bracketings came out different, two people who built the same chain in different orders would hold two different arrows, which is a bug, because the chain is the thing and how you assembled it isn't. The identity laws say that tacking the do-nothing loop onto either end of an arrow gives back the arrow. If the loop changed things, it wouldn't be the loop. Those six terms, objects, arrows, composition, identities, associativity and identity laws, are all of Eilenberg and Mac Lane's definition.

Composing relations is older than the definition, too. Who put a name on that move first? I went looking. Augustus De Morgan, writing on the logic of relations around 1860, took the "is" out of the old syllogism, put a general relation in its place, and composed two relations to get the conclusion. The example that gets quoted from that tradition, Peirce's a decade later, is "x is the lover of a servant of y," which is relational composition with better gossip. When exactly De Morgan read that paper to the Cambridge Philosophical Society, and on which pages it was printed, the sources disagree, which is a small reminder that history is where you start digging. Then in 1870 Charles Sanders Peirce, an American scientist and philosopher you met in chapter nine, carried George Boole's algebra of logic (the arithmetic of true and false, and Boole was a largely self-taught schoolmaster) over to relations, with composition as one of its basic operations, and he's usually counted as the real founder of the theory. So following one arrow after another is older than computers, older than categories, older than Turing's machine: it is a nineteenth-century logician's product of relations.

And where did they get the word? They didn't get it from mathematics. They borrowed category from the philosophers, from Aristotle and Kant, and functor (a map from one whole category to another) from the philosopher Rudolf Carnap, and then gave both words exact meanings none of those philosophers had in mind. So Aristotle walks back into this chapter without being asked: the man who put the form inside the bronze lent his word to the field that keeps track of forms by their arrows. That doesn't change the laws. It's a reminder that a word can start in philosophy and arrive in mathematics with a precise meaning bolted on, which is the thing I've been trying to do with established since the start of the book.

What a thing is is how it maps. For now the residue is the emphasis I needed: structure lives in the maps. Nothing later in this book needs more category theory than that sentence, and the chapter on a box over any machine, where a record is

pinned down by the maps between a machine and the base it extends, is where the sentence does its work.

10.12 Structure is the part that compresses

Play this one with me. I write "th" and you guess the next letter. You said "e," and you were right. Now "the q": you said "u." Congratulations, you just measured the structure of English. In 1948 Claude Shannon measured it properly, in a paper that founded information theory, and found that about half of written English is "determined by the structure of the language." It's the reason a zip file of a long letter is so much smaller than the letter. Be careful what that says. It does not say that every other letter is fixed by the ones before it; you just guessed "e" after "th" and you could have been wrong (think "than"). It says that, on average and over long stretches, about half of what's written could be rebuilt from the rest, so that the text is roughly twice as long as the choices in it. And he said why that matters in one line that I'd frame if I framed things: the structure "allows one to make a saving in time" (he added "or channel capacity"). The more you can predict, the less you have to read.

Thirty years later Jorma Rissanen, a Finnish information theorist at IBM's research lab in San Jose, turned that saving into a price. His idea is called the minimum description length principle, and it says that the best model of some data is the one that makes the total description shortest, where the total is the length of the model plus the length of the data once the model has explained what it can. A model is paid for in bits, and it's worth stating only if it saves more than it costs. Peter Grünwald, a statistician who leads a machine learning group at CWI, the Dutch national institute for mathematics and computer science in Amsterdam, in a tutorial you can find online, adds the footnote that keeps this honest: the price of a model depends on the code you chose to write it in, so the price itself needs a principle behind it, and that principle is something you have to argue for, not something you're handed.

That's a lot of "length of" to swallow in one paragraph, so do one with me, with numbers small enough to check on the back of an envelope. Every number in it comes from a code I'm about to make up on the spot, and I'll say every rule of the code out loud, because Grünwald's footnote is the whole point: change the rules and every number moves.

The data is a row of thirty-two digits, each a 0 or a 1, and you and I agree in advance that the row is thirty-two long, so nobody has to spend anything saying so:

```
01010101010101010101010101010101
```

(The space is only there to rest your eyes.) Every description in the code starts with one digit that says which kind of description follows: 0 means "here's the row,

written out," 1 means "here's a pattern, then the exceptions to it." Each digit is one yes-or-no answer, one bit, which is what Shannon's unit is. Written out, the row costs that first digit plus the thirty-two digits themselves, thirty- three bits, and there's no model in it at all. A pattern description goes like this. First come five digits giving the block length minus one: 00000 means one digit, and 11111 means thirty-two. That covers every allowed length. Then comes the block itself. The rule is "repeat the block, then stop at exactly thirty-two digits, cutting off the last repetition if necessary." Then come five digits for how many places the row disagrees with the pattern (up to thirty-one, which is plenty here), and five more digits for each of those places, numbered zero through thirty-one and listed in increasing order without repeats. At every listed place, flip the predicted digit.

Do it for the row above. The block is 01, two digits long. So the model costs 1 (which kind) plus 5 (the block's length) plus 2 (the block), which makes eight bits. Then comes what's left to say about the data once the model has said its piece: the row disagrees with "repeat 01" nowhere, so that's five digits saying "zero exceptions" and nothing else, five bits. The total is thirteen, against thirty-three for writing it out. The pattern pays for itself, with twenty bits to spare, and Rissanen's sentence has just happened on paper: a model is worth stating when it saves more than it costs.

Now spoil the row. Flip two of its digits anywhere you like. The model doesn't change, eight bits, but now the leftover says "two exceptions, here and here": five plus five plus five, fifteen, for a total of twenty-three. It's still cheaper than thirty-three; the pattern still pays, it just pays less, because the data has more to say that the model can't. Flip four, and the total is $8 + 5 + 4 \times 5 = 33$, a dead tie, and a pattern that saves nothing isn't worth stating. Flip five and it's thirty-eight, and now writing the row out is cheaper than explaining it, and this particular pattern has lost to writing the row out. Another pattern might still win. Flip places 5, 11, 17, 23 and 29 of the original row: now the block 010100, repeated and cut off at thirty-two digits, fits exactly. Its description costs $1 + 5 + 6 + 5 = 17$ bits. Five changes defeated "repeat 01" and produced a better description of their own. The comparison is between descriptions in the stated code, not a count of spoiled digits alone.

Message	Original row	Two flips	Five specified flips
Write all 32 digits	33	33	33
Repeat 01, list exceptions	13	23	38
Repeat 010100, no exceptions	does not fit	depends	17

For the last column, flip positions 5, 11, 17, 23 and 29. That defeats one short description and supplies another. Pattern descriptions include the type bit, length field and exception-count field. The raw row uses one type bit followed by its thirty-two digits.

So "is there structure here?" has turned into a question with a number on each side, and a break-even point you can find yourself: four spoiled digits, in this code.

And now change the code, because I promised you'd see the numbers move. Say I'd used seven digits for each exception's place (wasteful, but nobody stopped me). Then three spoiled digits cost $8 + 5 + 3 \times 7 = 34$, one more than writing the row out, where the five-digit code charged twenty-eight and the pattern paid. The pattern stopped paying one spoiled digit sooner. The row is the same, the pattern is the same, the verdict is different, and the only thing that changed was a convention I chose. That's Grünwald's footnote, and it's why I won't let you take any of those numbers anywhere else. They're the price of a description in one made-up code, a length of a written-down message, and nothing more. They are not the charge I opened this book with. That charge, as page one states it, is on one step a machine takes. How long anybody's description is doesn't enter it, and if the two ever turn out to be related, that has to be shown. Both of them being counted in whole numbers doesn't get to stand in for the showing.

Now push, because this is where "structure means information" gets born, and it's wrong, and it's an easy thing to believe. Flip a fair coin two hundred times and write down the string. Usually there is no substantially shorter way to say that string than to say the whole thing. You can watch it happen in the little code above, cut to thirty-two digits: a coin-flip row disagrees with "repeat 01" in about sixteen places, which would cost eighty-odd bits to list, and even the one block guaranteed to fit with no exceptions, the whole row used as its own block, costs $1 + 5 + 32 + 5 = 43$. The written-out row at thirty-three beats those two descriptions. Another block could do better; a lucky run of coins can make a pattern, so check it. The general reason long random rows usually resist substantial compression is a count: put each short description beside the one row it prints, and there are too few short descriptions to stand beside more than a small fraction of all the long rows. Nikolai Vereshchagin, a mathematician at Moscow State University, and Paul Vitányi, a computer scientist at CWI in Amsterdam, in a 2004 paper on exactly this question, put it without mercy: every bit of a string like that is "meaningless information, random noise." Such a string carries maximum information and zero structure. So in this sense structure is the part that compresses, the part where the "th" tells you about the "e," and most information and most structure are different things. Hold that one tightly, because if you built something that charged for information, you'd be charging for the wrong thing, and for a while I wasn't sure which one I'd built.

And now notice that the word just changed meaning on you, and on me. I'm pointing at it so it doesn't slide past. Shannon and Rissanen and Vereshchagin mean regularity: a pattern that lets you say something shorter. Tarski, a few pages back, and Paul Halmos, a Hungarian-American mathematician whose short textbook on sets is coming up, mean a set with named relations on it, and those two are not the same thing. You can run the test yourself: number a thousand objects, flip a coin for each, heads goes in group one, tails in group two. The result is a perfectly

good sorting in the set theorist's sense, every object in exactly one group. And the list of who went where is a thousand coin flips, which is to say it compresses about as well as a thousand coin flips: typically very little under a code fixed before the tosses. Feed it to a code built like the little one, with the fields widened to fit a thousand, and you get the coin-flip row again, only longer: typically most of the assignment still has to be written down. A lucky pattern can save some of it; the coin tosses alone do not guarantee there is none. By the set theorist's rule, the sorting is the structure, and it's every bit as much a sorting as "evens in one group, odds in the other," which compresses to almost nothing. The two are the same kind of object with wildly different description lengths. So the two meanings don't even rank things the same way: one asks "is there a sorting?" and the answer is yes for both; the other asks "does a short rule say who went where?" and the answer is yes for evens and odds and typically no for a substantial saving on the coins. The coin sorting has structure in the set theorist's sense without necessarily offering a shorter description. I'll keep both meanings, and I'll try to say which one I'm using, and when I slip you'll know what to look for.

10.13 No place for a grouping

You built Turing's machine on a table in chapter one: a tape, a head, a state, and a card of rules. Chapter one also kept three claims apart: what's proved about it, the Church-Turing thesis, and the claim about physics. Every laptop and phone and server computes nothing a Turing machine can't. What differs between them is speed, space, energy and cost. Hold on to that last word. Cost is where this book lives.

Now do the thing I did, and do it honestly. Look through those four parts for a place that says "these squares belong together." There isn't one, and that's not me failing to look. Everything the machine is at any moment is which state it's in, where the head is, and what's on the tape, and the table says what happens next, and that's the full list. So any grouping has to live in the states and the table, as a rule, or on the tape, as symbols, which is to say as bytes like any other bytes. The machine can hold a grouping. It just has no place where holding one is different from holding anything else.

In 1972 David Parnas, a software engineer, wrote a short paper on how to cut a program into modules. He took one small program and cut it two ways. Both worked. But one was easy to change later and the other wasn't, and the easy one could be "much less efficient" if talking between modules cost anything. His point was that a module is "a responsibility assignment," a decision about who answers for what, and how you cut decides what every later change costs. If you've ever maintained someone else's code, you've paid for their cut the way Parnas said you

would, probably at two in the morning. What survives the push is that a division is a commitment: it changes what the work costs, and it is no caption, no label stuck on afterwards. I'd like to say the thing I built understood that from the beginning. It didn't. I did, eventually.

10.14 Laying them all down

Now lay every one of those on top of the others and see what shows through. I see seven things. They're my overlay, the places where I see the sources agreeing. The sources never proved it together; some of them never met, and some of them disagree (Eilenberg and Mac Lane put maps and their composition at the centre; Shannon's structure and Tarski's are two different words, as I just showed you). You've pushed on each source yourself by now, so you're in a position to tell me if I'm seeing things.

A list of seven slogans would be easy to nod at and impossible to check, so I'm going to carry one thing through all seven and let you watch each one add to the one before it. The thing is the square you drew for the Pythagoreans, side, diagonal and all. Draw a few more on the same sheet: one small, one big, the small one again but tilted, and one stretched out into a long rectangle. You'll want a pile.

Structure is about relations among things, and the things themselves aren't it. You see that in the diagonal, the bronze, Leibniz's room, Bourbaki's unspecified elements, Tarski's named relations and the arrows. On your sheet, what makes a square a square was never the ink. It's that the four sides match and the corners are square, and that the diagonal stands to the side the way it does whether or not any number names it. Every one of those is a relation among parts. But on its own this first thing is too loose to use, because a sheet of paper is drowning in relations: this square is left of that one, this one is nearer the coffee ring. Which relations count?

Structure is what stays the same under a chosen set of changes. You see that in Euclid's triangles laid together, Klein's allowed moves, Noether's symmetry, and Frege's direction that every parallel line shares. This second thing gives the answer the first thing was missing: the relations that count are the ones that survive the moves you've decided to allow. Slide the small square across the sheet, turn it, and the diagonal still stands to the side exactly as before; "left of the coffee ring" doesn't make it across. The second thing is a filter on the first.

The same structure can be carried by different stuff: the statue in marble, or two sets that answer the same vocabulary the same way. The third thing is the second again with one more change allowed, which is to swap the material. Cut the square out of cardboard, or lay it out in string on the table, and if the relations survive, it's the same square in every way you agreed to care about. Aristotle's bronze is one

more thing you agreed not to look at, and the wooden sandwich is what happens when the material was never on that list.

The first act of structure is a sameness that sorts a pile you were already given. It's there in Euclid's first common notion, Frege's direction, Tarski's one question, and Russell's letter, with Zermelo's repair that you must start from a pile. At this point the overlay stopped being a list for me and turned into a shape, so slow down here with me. Look at your pile of drawings and say: two of them count as the same when an allowed move, for now a slide or a turn, carries one exactly onto the other. Watch what happens to the pile. It falls into groups. The small square and its tilted twin go together; the big one sits on its own; the rectangle sits on its own. And it falls cleanly, nothing half in one group and half in another, for a reason you can check. Klein's allowed moves always come with three guarantees: doing nothing counts as a move, every move can be undone, and two moves in a row is again a move. Those three are, in different clothes, "everything is the same as itself," "if this matches that, then that matches this," and "if this matches that and that matches a third, then this matches the third." Put the last two together and you get Euclid's first common notion, "things which are equal to the same thing are also equal to one another." So the second thing, sameness under chosen moves, is a sorting of the pile the moment you ask which drawings it calls the same. Frege did it with lines and parallels. Tarski's one question does it in the crudest way there is, by cutting the pile in two. And the pile has to be there first, because a sameness has to be a sameness among something; that's Russell's letter, and Zermelo's repair.

Adding structure rules possibilities out. Every one of Euclid's postulates rules one out. Half of English is already decided. A conserved quantity limits where a system can go. On your sheet: once the small square and its tilted twin are in one group, you've given up the right to answer any question the moves respect differently for the two of them. Tell me the side of one and the diagonal of the other isn't an open question any more. A sorting rules out a whole bundle of possibilities at once.

Held structure gets used without re-deriving it. You see that in Shannon's saving in time, Rissanen's compression, Parnas's one-module-at-a-time, and Leibniz's analysis of situation (reasoning about where things are straight from their relations, with no coordinates laid down first). The fifth thing pays off here: the possibilities you ruled out are ones you never have to check again. Know which group a drawing is in and you know everything the group shares without getting the ruler out, the same way the row of 01 never needed writing out. There's one catch, and I'll need a whole section for it: it only pays if you kept the sorting somewhere you can read it back.

And structure is always relative to a chosen frame, whether that frame is Klein's group, the vocabulary, the code you compress against, or Parnas's criteria. Nobody gets to say "the" structure of a thing without saying from where, and I do it constantly,

so watch me for it. On your sheet: allow stretching as one of the moves and the rectangle walks over and joins the squares; allow only slides and turns and it doesn't. Same pile, same ink, different groups, because I chose different moves. The groups were never in the paper. They were in the paper and the choice, together.

I think the seven hang together this way. Relations are what structure is made of. The chosen moves pick which relations count, and a change of material is just one more move. Asking which things the moves call the same sorts the pile. The sorting rules possibilities out, and what's ruled out is what you get to skip later. And because the moves were chosen, all of it is relative. Read that way, the fourth thing sits in the middle: it's where the first three turn into something you can write down, and where the last three get their start.

That's my reason for starting the next subsection with a sorting and a door across it, and I want to be exact about what kind of reason it is, because it's mine. None of these people said "start with a partition." Klein would tell you to start with the moves, Eilenberg and Mac Lane with the arrows, Shannon with the probabilities. What I can actually point at is narrower. In the framings about when two things are the same shape or the same kind (Euclid laying one triangle on another, Klein's moves, Frege's directions, Tarski's question) there's a moment where somebody decides what counts as the same, and that decision, written down, is a sorting of a pile. In Shannon, Rissanen and Noether I can't find that moment as cleanly, and I'm not going to bend them until I do. So the overlay doesn't prove that a sorting comes first. It shows me where the framings I trust most keep putting their first foot down, and I'm putting mine there too, with one thing added that none of them needed and the toll does: a direction across the sorting.

I looked for a framing that denies any of the seven and didn't find one, which could mean there isn't one or could mean I didn't look where it lives. If you know of one, I'd like to see it.

10.15 The smallest piece

Now the sentence I wanted to say at the top and had no right to until here, and even here I have to split it in two, because half of it is a textbook fact and the other half is me. The piece of structure I'm going to start from is a cut: a pile sorted in two, with a door between the halves that swings one way. A partition is the cut with the door taken off, and the partition comes with a textbook fact, so I'll do that half first.

The textbook half: a partition and an equivalence relation are the same object. That's Paul Halmos, Naive Set Theory, 1960, section seven, and you could read that section before dinner. I'll say it in words first. Take any pile of things. An equivalence relation on it is a way of saying "these two count as the same" that behaves the way sameness should: everything is the same as itself; if a is the same as b then b is the

same as a; and if a is the same as b and b the same as c, then a is the same as c. Every such relation sorts the pile into groups, the things that count as the same go together, and every sorting into groups hands you back a relation, "same group." Do one and then the other and you land where you started, in both directions. So a partition and an equivalence relation are one object seen from two sides, and that's the fourth thing on the overlay, the sorting of a given pile, now with a name and a page number.

You can check that with the buttons from chapter four, or anything else in a pile. Say two things count as the same when they're in the same group. Is everything the same as itself? Yes, it's in its own group. If a is with b, is b with a? Yes. If a is with b and b is with c, is a with c? They're all in b's group, and b is only in one group, so yes. Now go the other way: start with any "counts as the same" that follows those three rules, and gather everything the same as a into one group. Could two of those groups overlap without being the same group? If something is the same as a and the same as b, then by the rules a is the same as b, so their groups are one group. So the groups never half-overlap: you get a clean sorting. And doing one and then the other lands you back where you started. That's the whole proof, and it's textbook mathematics. I'm claiming nothing new in it.

And the smallest structure that sorts, by my ruler, the one question, is the same object a third time. Put a yes-or-no question to every thing in the pile. The ones that say yes are one group and the ones that say no are the other. Going the other way, a sorting into two groups becomes a question the moment you point at one group and call it "yes." The only thing a question has that the sorting doesn't is which group got the label. So the set theorist's first step and the logician's first step are the same step, and nobody has to pick a side, which is a relief, because it's an argument you can lose with yourself for a long time.

Now put the door back. In chapter five you met "at or below," with its two rules: every place is at or below itself, and at or below carries across a third place. It doesn't have to go both ways. Take the places that are each at or below the other, and put them in one group. That "each at or below the other" follows all three of sameness's rules, so it's a clean sorting, the buckets. And between buckets, "at or below" only goes one way: if two buckets were each at or below the other, they'd be one bucket. So an order is exactly a sorting with doors between the groups, and every door swings one way. Take the doors off and a sorting is what's left.

A yes-or-no question is the smallest case of that: two buckets, no and yes, and one door from no to yes. The thing settling on one side of a line, from the first page of this book, is a move through that one door. What chapters one through three found keeping takes is what it costs to go through it and not come back.

So much for the textbook half. The other half is mine, so hold it looser, because Halmos says nothing about "smallest" or "first," and I'd be lying if I let his page

number carry my sentence. In what I build, the order goes: a pile, then a split of the pile with a door across it, then more splits at once, which is an order. The groups come first because a question needs something to be about, and a door needs two sides to stand between. That's an order for my construction. It isn't a law that every account of structure has to start by sorting. You could start with arrows between the things themselves and no groups at all, and people do, and nothing I've shown you rules that out. So the honest sentence is: a split with a door across it is where I chose to start. It's the smallest by the ruler I stated. If you have a smaller start, by a ruler you can state, I want to see it. It would change this book.

10.16 Buckets

So I'll do the symmetric half once, for real, because it's the half you can do with your hands. Take everything in front of you and sort it into buckets, with nothing in two buckets and nothing left on the floor, and that's the whole of the definition. It's the thing you do with a kindergartner and a pile of blocks, and I am aware that I've just marched you past Pythagoras, Plato, Euclid, Frege and Noether to arrive at a pile of blocks. Keep the blocks anyway. They're about to cost you something.

Take ten blocks, and put the red ones in one bucket, the square ones in another and everything else in a third. Now a red square turns up, as they do, and you've got your first crisis: it belongs in two buckets and the rule says one. So this isn't a partition yet, and you have to decide which bucket wins. That decision is the whole thing, because a partition is a pile of small rulings like that one, and once you've made them all, the pile is holding a fact it wasn't holding a minute ago.

The next part isn't kindergarten. Why would that fact cost anything? Watch what it does to one question. Before the buckets, "are these two in the same group?" means working it out: go find the first one, go find the second one, apply the rule to each, and compare. After the buckets, it's a glance: read the first one's bucket, read the second one's bucket, and you're done. With each block's address already in hand, and a table that retrieves a label in one operation, that is two lookups whether the pile has ten blocks or a billion. Those are access and cost assumptions. A pile with no addresses, or an index that must be searched, has not been given that same two-lookup rule. What you're looking at is Shannon's saving in time and Parnas's one-module-at-a-time, the sixth thing on the overlay, showing up in a kindergarten.

Look at what makes that glance possible: a table, with a written-down line for each block saying which bucket it went in. The partition is the fact; the table is one way of holding the fact so it can be read back fast, and you had to build the table, and you have to store it, and it has to be the kind of thing you can read in one look (a bucket number, say; a list of every block in the bucket would have you searching through it again). Anybody who's used a database has met this table; it's what an

index is, and an index is a thing you build, store, and keep up to date, and every part of that takes something. Take the table away and keep the partition and you're back to working it out. And sometimes there was never a search to save: if the rule is "evens here, odds there," you can answer "same group?" by looking at the last digit of each, no buckets needed. What survives is that holding a sorting in a form you can read back is worth something, and building that form cost something, and those are already two different events. Building the table, storing it, checking that it's right, and deciding to rely on it are four different things. The first cost question I want to put to that structure lives in the gap between the third and the fourth. Nothing in the kindergarten has shown a charge in that gap. It has separated the acts well enough to ask.

So who, in everything I read, says what has to be paid? I found three answers that come close, and I'll be exact about how close, because this is the edge of what I know. Rolf Landauer, in 1961, and Charles Bennett after him, say that any step which takes two possible situations and makes them one, erasing a bit or merging two paths a computation could have taken, has a heat floor when it removes uncertainty under their thermodynamic assumptions. That is a physical claim about how the device and its surroundings are prepared, Landauer's principle, and in this book it is always a premise, never something I prove. It's the place, of all I've found, where a fact about information turns into a fact about heat, though at room temperature the amount for one bit is far too small to show up on anybody's electricity bill. In 2012 a lab in Lyon erased one bit, held in a single tiny bead, and measured the average heat it gave off coming down toward Landauer's floor as the erasing was slowed, and other labs have found the same since. Not everyone accepts the principle in general, and I'll come back to the people who don't. Is sending blocks to buckets a step of that kind? Push on it, because the answer depends on what you keep. If you keep each block and write its bucket next to it on a spot you knew was blank, you've merged nothing; you can read the block right back off the pair. You have spent something, though: the blank spot the label now sits on, which the machine hadn't used before, and any heat from the writing depends on its implementation. Bennett's whole point in his 1973 paper was that a computation which keeps its inputs, and keeps enough of its working to retrace its steps, can be run without any merging step at all, and it pays for that by taking fresh states. If instead you throw the block away and keep only the bucket label, then two different blocks can leave the same label behind, and that is a many-to-one step, the kind Landauer charges for. Both are things people call "sorting into buckets." Only the second loses a distinction. So the bucket by itself doesn't hand you Landauer's heat. The forgetting does, when there is forgetting, and keeping instead is paid for with fresh states. Whether a given machine forgets is a question about that machine. It has to be answered on the page, and I won't wave at it here. Rissanen says a model is paid

for in the bits it takes to state, and Grünwald says the price depends on the code. Parnas says a bad cut is paid for later, at two in the morning. And that's where it stops. Rovelli, back in chapter six, prices a choice that sticks, and his choice is one nobody could predict, which that section showed is a different act. I couldn't find anyone who says that the moment you vouch for a grouping, the moment you stand behind "these belong together," has a price of its own. Just arranging them that way is a different act. That could mean nobody says it. It could mean I didn't find the person who does. I honestly can't tell you which.

So that's where I am at the end of one word. A grouping, the act of making it, the information kept about it, and a commitment to rely on it are different things. The blocks let you separate them with your hands. Now take them down to the smallest case there is, a bucket with two sides, one block, and a ruling that sticks, which is a thing settling on one side of a line and staying there, the case the first chapters started from and showed what keeping it takes. The construction I will study keeps selected distinctions in a state. Some are there from the start. Some are newly recorded. Those are different cases: a box that already says YES can sit there forever under a do-nothing rule, with neither a merge nor fresh states. The count in chapters one through three starts with NO and follows a new YES that stays under a fixed total rule. That is the case to which its merge-or-fresh conclusion applies. A grouping by itself does not supply that run or its price.

CHAPTER 11

The counter

Back in chapter four you kept a tally beside the two boxes, the way you'd keep score beside a board game. The tally lived in your hand. The machine never saw it. That's fine for a game on a table. For anything that's going to run on its own, with nobody keeping score, the total has to live somewhere the machine can reach, which means the machine can, in principle, write it.

So this chapter is about what a running total inside a machine has to be before you're allowed to call it a receipt. Here's the answer first, because it's short, and then the chapter is about why each word is there: a receipt is a total that lives in the state, that only climbs, that equals the sum of what was charged, and that nothing else could have been.

11.1 A stick, split down the middle

The oldest receipt I could find that you can still hold is a piece of wood. From about 1100, under Henry the First, until 1826, the English treasury recorded what it was owed on tally sticks: a strip of hazel, notched across with cuts of set widths, then split lengthwise so each half carried the same notches. The treasury's own handbook from around 1180, the Dialogue of the Exchequer, gives the widths: a thumb's width for a hundred pounds, a swollen barleycorn for a pound, a bare cut for a penny. The payer kept one half, called the stock, which is one story for where the word stockholder comes from. The treasury kept the other. When the time came to settle, you laid the two halves together, and only the two original halves fit, because wood splits along a grain nobody can copy. Add a notch to your half and the other half shows a gap where the partner should be. Try to take a notch away and you find that wood can't be uncut.

When they finally stopped using them, the clerks burned centuries of old tallies in the furnaces under the House of Lords, overheated the flue, and burned down the Palace of Westminster. I mention that because it's the best fact I picked up for this book, and because it tells you how many of them there were.

Do it yourself. Draw a border around a strip of card, notch it once for every coin you hand a friend, then split it lengthwise through the notches and keep half each.

Don't trim the edge or swap in a fresh strip. Now add a notch to one half and put them back together. You'll see the mismatch straight away. And you can't put a cut back: a patch is just one more thing to look at. The total is the notches, one per coin, and the count only climbs. Both halves started with the same count, so if they disagree now, somebody changed one. The mismatch won't tell you who. What the stick gives you is a comparison you can make, not an object nobody can forge.

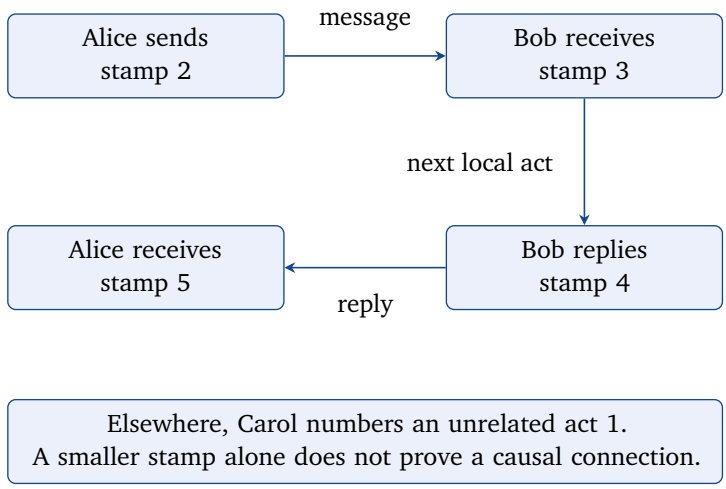
Economists read the same stick another way, and they're not wrong either. Alfred Mitchell-Innes made the treasury tally his star witness in 1913 that money is credit: the crown paid the people it owed by "raising a tally" on some branch of its income, a wooden promise to pay. A receipt to me, money to him, and it's the same piece of wood. Narayana Kocherlakota proved a version of his point in 1998: in a wide range of economies where nobody can be held to a promise, anything money lets people do, a memory of everyone's past dealings lets them do too. His title says it in three words: money is memory.

11.2 Why it has to climb

Page one already showed why there are no refunds: if a later move could pay you back, the move that settles charges you one, the next move hands it back, and the whole run comes out free. A total that can come back down is an accountant you can lean on to revise the books. "This run cost at least one" would be true of a run that ended having cost nothing. So the count only climbing isn't decoration. It's what makes "at least" mean something.

The same rule turns up in two places that have nothing to do with each other, pointing in opposite directions, and that's a big part of why I trust it.

In 1978 Leslie Lamport asked how a set of computers passing each other messages, with no shared clock, could ever agree on what happened before what. His answer is a counter on each machine with two rules: add one before you do anything, and when a message arrives carrying the sender's count, jump your own counter above it, never below your own. Those counters, he showed, respect cause and effect: if this caused that, this has the smaller number. And they do it, in his words, "with no actual timing mechanism." They're clocks only in name.



Start Bob at zero and Alice just after her local act 1. On receipt, each raises the local count above both its old value and the incoming stamp.

Try it with three people passing notes and you'll see it work in two minutes. Then let one of them reset their counter to zero after reading a note, and watch a reply come out numbered before the note it replies to. A counter that goes down can't be leaned on for order. The other direction doesn't hold, though: two people can number two unrelated acts 1 and 2, and the numbers alone don't say the first caused the second.

Where did Lamport get it? In the notes he keeps on his own papers, he says the timestamps came from an earlier note by Paul Johnson and Bob Thomas on keeping copies of a database in step, whose method wasn't quite right: it let things happen in an order that broke cause and effect. What let him see the fix, he says, was special relativity, which teaches that the one order of events everybody agrees on is the order of what could have caused what. And he passes on a line from Jim Gray, the database man whose wedding you met in chapter eight: Gray had heard two opinions of the paper, that it was trivial and that it was brilliant. Lamport says he can't argue with the first and isn't inclined to argue with the second.

Now the other direction. In 1949 Alan Turing, and in 1967 Robert Floyd, each asked how you prove a program stops. Both times the answer was to find a number that goes down on every step and can't go down forever, like a whole number that can't go below zero. If every step lowers it, the steps can't go on forever. Up-only gives you order, and down-only gives you stopping. They're the same shape, seen from the two ends: a number that only goes one way is something you can build on.

And there's a real case where a counter was allowed to go down and got gamed exactly there. Ethereum, one of the largest blockchains, charges "gas" for every step of a program, and for years it gave gas back when a program freed up storage. Someone noticed you could fill storage when gas was cheap, hold it, and cash the

refunds when gas was dear, and a token was built to do exactly that. The real work in a block could reach nearly twice the block's stated limit. In 2021 a change called EIP-3529 cut the refunds sharply and capped them at one fifth of the gas used. The written reasons are the token and those overfull blocks. I read both as the same lesson: a counter that goes down was being leaned on at the place where it went down. Gas comes back in a later chapter, as the one real system of five that charges at the moment of the write.

11.3 The total is the charges and nothing else

The next property is the plainest. The counter after a step is the counter before, plus what that step was charged. Add up a whole run, and all the middle values cancel out, the way a sum of differences collapses to last minus first. The total is the sum of the charges.

Why say it? Because it's the property that fails quietly. Say a machine charges one for raising its record, and somewhere in its error handling forgets to add the charge when the step fails halfway through. Or it charges two on one path and one on another for the same move. Or it has one step that writes the counter directly. Then the counter is mostly the sum of the charges, and "mostly" is the whole gap between a receipt and a note that says "trust me, about this much." The rule is that there's no "mostly": every step, every outcome, every error, adds exactly its charge, and a step that fails is charged the same as one that succeeds, because the charge is on the attempt. For a machine built that way, the rule can't fail. What can fail is the claim that a real machine is built that way, and checking that means going through every move it has, the failure branches included, one at a time. It's dull, and it's the whole job.

One thing it doesn't say is that every unit bought something. The total is exact about what was spent. What the spending was worth is a later chapter's question.

11.4 Only one honest count

The next property surprised me by being provable at all. Fix the price list: this move costs one, that one costs zero. Now say somebody else keeps their own books: their own total for every state, starting at zero, and going up on every step by exactly what the price list says. Could their books disagree with the machine's counter, on some state you can actually get to?

No, and the proof is the tally stick. Both totals start at zero. On every step, both go up by the same amount. So on every state you can reach from the start, they're equal. Agreement at the start survives the first move, then the second, then every

move after it. Given the price list, the count is forced. It's the arithmetic, not a second opinion.

It only says that about states you can reach. On a state nothing reaches, it says nothing, which is the right thing for a receipt to say.

And one thing follows that's easy to miss. If the counter is part of the state, then a run that comes back to a state it's already been in has the same counter both times, so the moves on that loop cost nothing in total. A machine that keeps its counter inside its state can only go round a loop that costs nothing. Every paid move lands it on a state it's never been in, because the counter is higher there than anywhere before. So a counter that only climbs is itself paid for in fresh paper. On a finite machine, that means no move can be priced above zero at all: play any move over and over, the states come back around, and a loop costs nothing. A machine that keeps its own receipt, with even one paid move, needs endlessly many states. That's chapter two again, coming up from underneath.

Here's a puzzle that shows where the "only one" stops. Take the one-switch machine whose set costs one and whose reset costs zero. Set it once and it's up. Reset it and set it again, and it's up again, the same state. Now picture a second machine that keeps the whole list of moves. To it, "set" and "reset, set" are different. So you can always walk a list of moves into the second machine, one way and one way only. But you can't always walk the first machine's states into it, because two lists that land the first machine on the same state can land the second on different ones. You can do it exactly when the second machine never tells apart two lists the first one treats the same. Joseph Goguen put that as a general rule in 1972, in a paper whose title says it outright, "Realization is universal," and Thomas Colcombet and Daniela Petrişan redid it in 2020. None of that uses the toll at all.

The nearest relative in physics is in thermodynamics. Elliott Lieb and Jakob Yngvason showed in 1999 that once you fix which states can be reached from which, and that follows a short list of rules, there's a quantity that goes up exactly along it, and it's the only one once you pick where zero is and what one unit is. Their order goes in and their count comes out forced. Here the price list goes in and the count comes out forced.

11.5 The toll is an old trick with a new name

It would be nice to have a new kind of cost argument here. This isn't one. The toll is a case of a technique computer scientists have used since 1985, and that a mathematician studying whether things settle down had, in another form, in 1892.

It's called the potential method. Give every state a number, the potential, that measures how much is stored up in it. Then look at each step and ask: does the step's cost, plus the change in potential, ever come out negative? If it never does,

then any run costs at least as much as the potential dropped from start to finish, because the drop had to be paid for along the way. Robert Tarjan set it out in 1985, crediting the potential view to Daniel Sleator. Tarjan used it the other way round, to show a run costs at most so much.

Do his coin version. A stack of coins. PUSH adds one and costs one. POP-ALL removes them all and costs as many as it removed. Let the potential be the number of coins. PUSH costs one and adds one to the potential. POP-ALL of k coins costs k and drops the potential by k. So any run from an empty stack costs at most twice the number of pushes, however many expensive pops are in it.

The decreasing number underneath it is Aleksandr Lyapunov's, from his 1892 doctoral thesis on whether a moving system settles down. It was in Russian. A French translation came out in 1907, made by a French naval engineer, Édouard Davaux, and corrected by Lyapunov himself. The English one came out in 1992, a hundred years after the original, translated from the French. A translation of a translation. Did the computer scientists of 1985 get it from him? I can't tell you, and nothing here needs them to have. What I have is a picture: the same idea, a number that only goes one way along a run, in a thesis about whether a moving system settles down and in a paper about data structures, close to a century apart.

Now point it at the record. Let the potential be one when the record says not done and zero when it says done. A step that takes not done to done drops the potential by one, so for the step's cost plus the change in potential not to come out negative, that step has to cost at least one. That's the toll, exactly. And "any run from not done to done costs at least one" is just the method's usual conclusion, the potential dropping from one to zero.

Ask for the other direction too, that a step's cost is never more than the potential it drops, and the two together pin every price exactly: one on a step that settles something, zero on every other step, and nothing ever going back to not done. One catch: that only works if the price can depend on where the step starts. With a single price per button, like the stamp, a button that settles once still costs one when it's pressed again on done, so no price list per button can meet both directions.

Two more tools sit next to this. Run a list of moves inside a wrapper that adds up a number as it goes, and you get back the final state and the run's total; Norman Danner and Daniel Licata built cost reasoning for programs on exactly that, in 2015 with Ramyaa Ramyaa and in 2022. And give each move its price ahead of time, as a tag that adds up as the moves do; that's Shin-ya Katsumata's grades from 2014, which Dominic Orchard, Vilem Liepelt and Harley Eades put into a working language in 2019. The potential method itself went into the checking of programs before any of those: Martin Hofmann and Steffen Jost in 2003, and Jan Hoffmann, Klaus Aehlig and Hofmann in 2011. The difference with this book is when the number gets written. A cost checked ahead of time gets checked before the program runs,

by whoever checks it. A cost kept in the state gets written while it runs, by the step itself. The number is the same. The moment of truth is different. For a program with loops whose cost depends on its input, a fixed tag can only bound the cost, while the counter holds the exact number; cost-aware type theories like calf, from Yue Niu, Jonathan Sterling, Harrison Grodin and Robert Harper in 2022, reason about such costs with proofs.

And there's one more neighbour, closer than any of those. A number the step keeps up to date and nothing else ever reads is what people who prove programs correct call ghost state. Susan Owicki and David Gries added it to programs in 1976, because some true facts about a program can't be proved without it, under the rule that it never steers the program. Jean-Christophe Filliâtre, Léon Gondelman and Andrei Paskevich worked out in 2014 when ghost code can be removed without changing what the program computes. By that test the counter is ghost whenever no step reads it. So what does putting it in the step buy? Two things. The number is there while the machine runs, where somebody looking through a window can see it, not only inside a proof written afterwards. And the rule that keeps it covers every move the machine has, so every run.

Two last relatives. Chapter four's floor, "any run from not done to done costs at least one," is the cheapest-path bound in a system with weights on its moves: every path from the not-done states to the done ones crosses the line between them, and every step across that line weighs at least one. Weighted machines like that go back to Marcel-Paul Schützenberger in 1961. And the physicists who study what you can turn into what, when some operations are free, have the same shape: Bob Coecke, Tobias Fritz and Robert Spekkens gave the general setup in 2016, Fritz worked out its algebra in 2017, and Tomáš Gonda and Spekkens catalogued the numbers that never go up under free operations in 2023. Read with "done" as the thing you're after and one unit of cost as the currency, the not-done potential is the simplest of those. It's a neighbour to know about, not a result of mine.

So what's new, if the method isn't? Only where it's pointed. Tarjan and Sleator's method proves things about costs. What this book adds is which potential, the one that drops when something settles, and what happens to that when you can only see part of the state through a window.

11.6 What the counter is, said once

A receipt is a total that lives in the state, only climbs, equals the sum of the charges on every path including the failures, and is the only such total, on every state a run from the start reaches, for its price list. Each of those had a home before this book: the stick for a total that lives in the state, Lamport's clocks for climbing, Turing's and Floyd's numbers that only go down for the same rule run backwards, the cancelling

sum for adding up, the step-by-step argument for the one honest count. None of them is physics, and none of them says what the charges should be. Once you've chosen the charges, the counter is the arithmetic.

There's one thing a counter can't do just by existing. The stick works because you can lay the two halves together. What if all you're ever shown is one half, through a window that doesn't show the notches? Then you're back in chapter seven. Take two states that differ only in their count, one at 1 and one at 2, and let nothing that comes through the window depend on the count. The window gives the same view for both, any reader gives the same answer for both, and one of the answers is wrong. So a window that leaves out the count can't give it back. To read the count, the count has to come through.

CHAPTER 12

What a ticked box lets you lean on

I keep writing "you can lean on it" and then backing off. Let me stop doing that and say what it would take. A box went from not ticked to ticked. The toll charged the move that ticked it. It stays ticked. Chapter eight made it established: it could only be ticked right after a check of that very claim passed. And the right window lets you see it.

Now: what is a later step allowed to do because of that? That's a different question from "is the claim true," and the gap between those two is what this chapter is about.

12.1 The coin that moved

Run this on a friend. The feeling of it is the point.

Put a coin under one of three cups while they watch; say cup one. Send them out of the room. Move the coin to cup three. Then, and this is the trick, put a second coin under cup one. Bring them back and ask: is there a coin under cup one?

Say they answer yes, because they trust the coin they saw has stayed put. There is a coin there, so they're right. And their reason was a good one: they watched it go in, and nobody usually moves coins around. But look at what happened. Their reason is about the first coin. The answer came out true because of the second coin. They were right by luck.

That's Gettier's trap again, from chapter eight, done with cups. Edmund Gettier's three pages in 1963 broke a definition of knowing: that you know something when it's true, you believe it, and you've got good reasons. Gettier took that definition from two philosophers of the 1950s, Roderick Chisholm and A. J. Ayer, and the only place he mentions Plato is a footnote saying Plato "seems to be considering" something like it. The Stanford Encyclopedia's entry on the analysis of knowledge, by Jonathan Ichikawa and Matthias Steup, says that definition gets called "traditional," and that the reason part of it is "often traced to Plato's Theaetetus." The same entry calls the

idea that it was ever widely held "a bit of a convenient fiction." And Julien Dutant, a philosopher at King's College London, wrote a 2015 paper called "The Legend of the Justified True Belief Analysis," arguing that the older tradition asked for something stronger: a mark on the belief that can't be wrong. I've read only summaries of Dutant's paper, so take that last part as his, secondhand.

Bertrand Russell had a related case in 1948: somebody looks at a stopped clock at the one moment it happens to be right. They're right about the time, and they know nothing. And 1948 isn't the bottom either. The Stanford Encyclopedia gives a case from the eighth-century Buddhist philosopher Dharmottara: you're looking for water on a hot day, you see water ahead, it's a mirage, and when you get there you happen to find water under a rock. And one from Peter of Mantua, a logician who died in 1400: Plato is running right next to you, you take him for Socrates, so you believe Socrates is running, and Socrates is in fact running, in Rome, where you can't see him. People in different centuries, languages and traditions kept building the same trap by hand: a reason that points at one thing, and a truth that happens to be sitting on another.

What would fix your friend's reason? Lift cup one again. Now there's a new look at the coin that's actually there. That gives them evidence about this coin, now, instead of a memory about the coin that left.

12.2 Plato nearly had it

The dialogue where Plato goes after knowing is the Theaetetus, and it's named for a real person, so I wanted to know who. He was a mathematician. In the dialogue he's a teenager, and he tells Socrates that his teacher had been showing them, one case at a time, that the side of a square of area three, or five, and so on up to seventeen, can't be written as any fraction. That's the Pythagoreans' diagonal from chapter ten, done case by case. The young man's idea was to stop going one case at a time and find one word for all of them. Euclid's tenth book, on lengths that share no common measure, is thought to be largely his work. The dialogue opens with him being carried home to Athens from the army at Corinth, badly wounded and sick, barely alive; the usual date is 369 BC. And the conversation itself survives as a record: Euclid of Megara, not the geometer, says he took notes as soon as he got home, filled them out later, and had Socrates correct them whenever he came to Athens. So the book about what turns a true belief into knowing survives as a record Socrates himself corrected.

In it, Socrates tries three different meanings for the "account" that should turn a true belief into knowledge: a statement you can say out loud; a list of the thing's parts; and, third, the mark that sets this thing apart from everything it could be confused with. Each one fails, and the dialogue ends with nobody knowing what

knowledge is, which is one of the more honest endings in philosophy. But the third one is the one I'd underline: the account as the thing that tells the real case apart from its rivals. He couldn't make it work because he had no way to say what that account had to be attached to.

Socrates gives his own reason for dropping it: if the account means knowing the mark, then knowledge has been defined as true belief plus knowledge of something, and the word you were defining is back inside its own definition. A circle. Does a check get out of it? It gets the word out: "a check that ran and found the mark" isn't defined in terms of knowing. Whether the check was any good is still a real question, and the rest of this chapter puts a price on it. So the circle moves rather than vanishing. It moves somewhere you can look at it.

12.3 Three things on hand

Here's what I mean by being entitled to lean on a claim. Three things have to be there.

An object, kept in the machine's state: a grouping, a list, a count, something you could print out. A belief doesn't count.

A checked relation between that object and the claim: a check that actually ran, with its result on the record. "A check exists somewhere" doesn't count.

A paid event: a step of the run where the object was put in place, and which the price list charged at least one. In a Thiele machine, the step that settles something is a paid step, because the toll charges it.

Keep the second and third apart, because they come apart. A note saying "coin under cup one, checked" can be written without anyone lifting a cup: an object, but no checked relation. Now actually lift the cup, find the coin, and write it down, but let that step be free, so the box never gets ticked and nothing gets charged. The object and the check are both there. The paid event is missing. And the other way round: ticking the box costs one, but the toll alone says nothing about what the step looked at. Payment by itself supplies no check.

And here's the sentence most likely to get lost: nothing in the bare machine makes a later step read the box. A move is handed the state and goes where its rule says, and its rule is allowed to ignore the box completely. So when I say a run is entitled to lean on a claim, I mean what somebody checking the run afterwards may conclude. The machine doesn't have to act on it, and unless somebody builds it to, it won't.

Line up the three things this book has called by related names, so they don't blur:

The bare toll: a paid move from not done to done. Nothing says a check happened at all.

Entitlement: an object, a check that ran with its result on the record, and a paid event. Nothing yet says the check was about this very claim.

An established record: from a clean start, it goes up only right after a check of the same property, on the same thing, with nothing changed in between, and the raise pays.

Even that last one is missing two things. Whether the check is honest is a fact about the checker, not something the record holds. And nothing asks whether the check could have failed: a check that says yes to everything is perfectly legal, true, and useless.

Why does the honest checker carry so much? The philosopher Linda Zagzebski argued in 1994 that any account of knowing that leaves even a small gap between having good reasons and being right lets a Gettier case through: take a case where bad luck makes the reasons point wrong, then add good luck that makes the belief true anyway. The three conditions plus an honest check close that gap, and only half of honesty is needed: the half that says no when the claim is false. A check that can be wrong, a set of tests that sometimes passes broken code, puts you right back in her gap, and then the record establishes something smaller and still true: this procedure said yes.

And who vouches that the check is honest? The record doesn't. Honesty is about what the check does on every possible input, and a finished state only holds this run. A proof about the checker can be written down and checked, but the checking is a check too, with its own honesty to vouch for, and somewhere the chain stops at a proof somebody accepts. In this book that proof is read once, outside every run, and nothing on the record pays for it. So an established claim is two things stuck together: something about the run, which the record carries, and something about the checker, which a proof carries.

The engineering version of this was built in 1996 and 1997. George Necula called it proof-carrying code: when you send a program to a computer that doesn't trust you, you send a proof with it that the program follows that computer's safety rules, and the computer checks the proof that arrived before it runs anything. Not your reputation, not "a proof exists somewhere," but the one in the box. Stephen Cook's certificates from 1971, in chapter four, have the same bones: a claim comes with something that's quick to check, and the checker checks that.

And David Hume, in 1748, arguing about miracles, laid out what a receipt replaces. He weighed testimony against evidence and wrote that "a wise man, therefore, proportions his belief to the evidence." The way I read him, his whole argument is a method for how much to trust a witness when you have no receipt. The move in this book is to stop weighing. A claim on the record is either backed by its object and its paid event, or it's testimony from outside, and a run that leans on testimony hasn't earned it itself.

Robert Brandom, a philosopher at the University of Pittsburgh, built a whole account of language on two words this book leans on, commitment and entitlement, in Making It Explicit in 1994. In his picture, as I read it and as his reviewers describe it, people talking keep score on each other: what each speaker is committed to, and what each is entitled to claim. Many claims count as entitled by default until somebody challenges them, and entitlement can be borrowed: I can claim a thing on your say-so and point to you if I'm challenged. Lay mine beside his and the differences are the content. His score is kept by the other people in the room; here it's in the machine's state. His entitlement often comes by default; here it's an event that has to happen and gets charged. His borrowing is my testimony, let back in on a leash: borrowed entitlement is only as good as the copy is genuine, which is chapter seven's problem.

12.4 Changing your mind about the cups

There's a whole field about how a reasonable mind should change what it believes when something new comes in, and I couldn't keep its people apart until I put them back on the table with the cups.

Your friend believes the coin is under cup one. They can learn something that doesn't clash with that, say cup two is empty, and just add it. They can lift cup one, find nothing, and have to change their mind about a table nobody touched. Or you can tell them you moved the coin. Those last two felt alike to me at first. They aren't. In one the friend was wrong. In the other the friend was right, and then the table changed.

Krister Segerberg, a Swedish logician, built a logic in 1998 for accepting something so it can never be given up. Isaac Levi split taking on a belief into a deliberate kind and a routine kind: deliberate is reasoning your way there, and routine is accepting whatever a procedure you trusted in advance hands you, the way you'd accept a reading off a thermometer you've already decided to trust, or what's under a cup when you lift it. The trust goes in once, before the answer comes. A passing check behind an established record is routine in exactly that sense.

Neil Tennant and Gösta Grahne I first took as two separate complaints. They're one move made from opposite sides. The target is the best-known impossibility result in that field, Peter Gärdenfors's theorem from 1986, that a natural rule for "if" statements can't live with reasonable mind-changing. Tennant argued in 2008 that it only works by treating some additions as changes of mind: the friend who learned cup two was empty gets counted as having changed their mind. Grahne argued in 1998 that it only bites real changes of mind, so the friend whose coin was moved is out of its reach. Side by side, they squeeze the theorem down to one case: being wrong about a table nobody touched.

The line this book leans on hardest is between those two kinds of change. Revising is changing what you believe about a world that stayed put. Updating is changing it because the world moved. Hirofumi Katsuno and Alberto Mendelzon drew that line for databases in 1991; Arthur Keller and Marianne Winslett had been working on updates since the mid-1980s. And look at what the friend sees: an empty cup one is the same empty cup whether they misremembered or you moved the coin. What they see can't tell revising from updating. That's a window with two states behind one view. What tells them apart is a count of how often the cups were touched. A machine can keep that count, a version number for each thing, and treat every write as an update: a write moves the version number, so every fact about the old version stops counting for the new one.

Alexandru Baltag, Bryan Renne and Sonja Smets gave a logic in 2014 that keeps apart three things Gettier's cases blur: good belief, knowing with the reason written out, and evidence that settles it. The friend who watched has good belief. The friend who lifted the cup has evidence that settles it.

Put that together and it says something exact about this book's record. A record that's never wrong and never corrected would be a stubborn fool's dream if its sentences were about the present. They aren't. Each one says a check passed on one version of one thing, and that sentence stays true after the version number moves on. The record can be never wrong and never corrected only because its sentences have the version stamped on them. Strip the stamps, and the first write would turn it into a liar that can't take anything back.

Push on that and it gives twice. The record is Segerberg's never-give-it-up acceptance every time, with no changing its mind in it. If a check turns out dishonest, the only fix that fits a record like this is a second record on top saying "withdrawn," the way a list of cancelled certificates works. And the record is never wrong partly because it only talks about past versions. Whoever reads it wants now, and getting from "held at that version" to "holds now" crosses every write since. The record doesn't make that step for you.

Chapter eight met the logicians who taught knowing to count, Ho Ngoc Duc and Artemov and Kuznets. One more thing about them, now that a record is on the table. Duc's knowing points forward, at what an agent could come to know with more steps of reasoning. A record points back, at what was checked. They're one arrow, read from opposite ends. A check is one of Duc's trains of thought, and what I've added is a direction and a charge on stopping. Where the charge sits follows from one choice: I want it to bind every program the machine runs, and a charge that does that has to sit inside the step, since a program can skip any charge kept somewhere it can write.

Three things entitlement isn't. It isn't permission: a system that says who may write to what grants that from outside and says nothing about truth. It isn't "it's true,

so I can use it": that's the cups. And it isn't "right by luck": a shortcut that gives the right answer because the inputs happened to cooperate is the stopped clock.

12.5 Count the rivals

Now the number, because a book about what keeping costs has to say how much, and the idea fits in a game with three playing cards.

Shuffle cards marked 1, 2 and 3 face down. Sort them using only questions of the form "is this one bigger than that one?" and count the questions. There are six orders the cards could be in. Each question has two answers, so after one question you've split the six into at most two piles, after two at most four, and you need to get down to one. Four is less than six, so two questions can't always do it, and three can. Draw the tree of questions and you'll see it: six leaves, three deep. The number of questions you need is at least the number of halvings it takes to get from all the possibilities down to one, rounded up. Donald Knuth gives this, in the sorting volume of The Art of Computer Programming, as the floor for all sorting by comparing: a tree so many questions deep has at most two to that many leaves.

Is the floor always the price? Try five cards. There are 120 orders, and seven halvings get you under that, so you need at least seven questions, and seven is enough, though not by the obvious method. Finding the way is a good evening with a deck of cards; Knuth has it, under the name merge insertion, from Lester Ford and Selmer Johnson in 1959. Now take twelve cards. They can be in 479,001,600 orders, so the count says at least 29. Ford and Johnson's method takes 30, and in 1965 Mark Wells showed by a computer search that nothing does better: twelve cards need 30. The floor was 29 and the price is 30. So the count is a floor and only a floor. It never promises the floor can be reached.

Ralph Hartley measured information exactly this way in 1928, before Shannon: by the number of possibilities, with no chances in it, just the count. That's the measure I use here, and it's Hartley's.

The same count works with something you can hold. Put three coins under three cups. Each coin is heads or tails, so there are eight possible rows. Somebody tells you heads under the first cup and tails under the second. Lift the first cup and compare: if it matches, four possible rows are left. Lift the second and compare: two left. The last coin could still be either. Eight, four, two.

This is a parlour game, and an old one. In A Christmas Carol, in 1843, Scrooge's nephew Fred plays a game called Yes and No at his Christmas party: he thinks of something, and everybody else asks questions he may only answer yes or no. He's thinking of an animal, a disagreeable, savage one that growls and grunts and lives in London, and the answer is Uncle Scrooge. Most people know it as twenty questions, and twenty is a good number for it: twenty questions that each split the field in half

can single out one of about a million things. But the halving only happens because Fred answers truthfully about what's actually in his head. Lie once and the count still shrinks, around the wrong animal. The cups showed the same thing: the four and the two are real because you lifted the cup and compared. Being told what was under it wouldn't have made them real.

Now put it on a run. Say a run comes with a list of possibilities still on the table, and a shorter list with some crossed off. Picture the crossing off as a tree of questions, like the card tree. If the tree is no deeper than the number of paid steps the run took, and it has enough leaves to cover the narrowing, then the run paid at least the number of halvings it takes to point at one thing on the long list, minus the same for the short list.

Chapter six is why the payment has to go in as an assumption. Looking can narrow a list without costing anything: the demon's watcher narrowed two candidates to one for free. So the narrowing alone doesn't force anybody to pay. Wherever this book puts a price on narrowing, the payment goes in out in the open: either a tree no deeper than the paid steps is handed in, or the run does the narrowing itself, with every test step charged at least one and every test keeping at least a set share of the list.

That last condition does real work, and you can see it fail. Take a list of eight things and one test: "is it this one?" If it passes, one thing is left. That's three halvings for one paid step. If every test has to keep at least half the list, that test isn't allowed, and with it allowed, the bound breaks. So the bound is: if every test keeps at least a share of one over two-to-the-k, the halvings dropped are at most k times what the run paid.

12.6 Narrower means a stronger test

One link here is pure logic, and Plato's third account is the whole of it. Take a list of candidates and something you can observe about each one. The list's test passes an observation when some candidate on the list would show it. Shrink the list, and the test can only get pickier: anything the short list's test passes, the long list's test passes too.

Pickier isn't the same as strictly pickier, though, and the cards show the gap. Take all six orders as the long list, and as the short list the three where card 1 sits left of card 2: 1-2-3, 1-3-2 and 3-1-2. Observe "is 1 left of 2?" Every survivor says yes, and the dropped 2-1-3 says no. So the long list's test passes yes and no, and the short list's test passes only yes: strictly pickier. Now, with the same two lists, observe "is the first card a 3?" The survivor 3-1-2 says yes and the other two say no. Among the dropped ones, 3-2-1 says yes and 2-1-3 says no. Both tests pass yes and no. Same strength, even though three orders are gone.

What made the difference is a dropped candidate whose observation no survivor shares. That candidate is Plato's mark, the thing that sets this case apart from everything it could be confused with, turned into something you can check. Narrow the list and keep one candidate like that, and the test is strictly stronger. Narrow it without one, and the test may not have moved at all.

12.7 What you've got

Put the pieces on one list. Narrow a list of candidates with a test that tells some dropped candidate apart from every survivor. Find the paid step. Count the charges against a tree the run paid for. Then three things follow, on any Thiele machine: the short list's test is strictly stronger than the long one's; some step of the run took the record from not done to done, and that step was charged at least one; and the run's charges add up to at least the halvings the list lost.

A statement like that is true by packaging. It takes the conditions in and hands the conclusions back. The right word for it is a checklist, a checklist for leaning on a claim. It doesn't discover anything about leaning.

And a checklist isn't nothing. On 30 October 1935, at Wright Field outside Dayton, Ohio, the Army was evaluating Boeing's new four-engine bomber, the Model 299. Shortly after take-off it plunged back onto the field and caught fire; that's how the wire report of that day has it, and later histories add that it pitched up and stalled first, which is what controls locked in place would do. Two men died of their injuries: the Army's pilot, Major Ployer Hill, and Boeing's chief test pilot, Leslie Tower. The airplane was fine. The crew had left the gust locks on, a mechanism worked from the cockpit that holds the control surfaces still while the plane sits on the ground. There's a famous line that a newspaper called it too much airplane for one man to fly. I couldn't find that line in any paper from 1935; the earliest place I could find it is a New Yorker article from December 2007 by Atul Gawande, a surgeon who writes about how people get complicated work right, and he names no newspaper, so take it as a lead. What I can find is what came after. The answer was a card: a list of things to check, in order, every time. The most careful history of it I found is by Walter Boyne, a retired Air Force colonel. By his account it wasn't the first written routine for a pilot; those go back before the First World War. And it's this crash, he says, that made the checklist the way a crew flew. The airplane went on to be the B-17. Nothing on that card was a discovery. Every item on it was something the pilots already knew. What the card did was make sure none of it got left out when it mattered.

12.8 What standing behind a grouping gets you

Chapter ten ended on buckets. The toll never needed them: it prices any yes-or-no question. Here a grouping gets a job, and an exact one.

Put two red buttons and two blue buttons on the table, numbered 1 to 4, with 1 and 2 red. The question is colour. Draw circles round 1 and 2, and round 3 and 4. Call a grouping sound when every two buttons inside one circle are the same colour. This one is. Split the red circle into two circles of one button each, and it stays sound; splitting can't put two different colours together. Join 1 and 3 into a circle instead, and it fails. Rename the circles North and South and nothing changes, because soundness only looks at which buttons share a circle.

Now the record. Whatever keeps the fact that a grouping was found sound, a mark on a card or a box in a machine, is a question that goes from not done to done and stays, so everything the first chapters said about that applies to it: the raise pays the toll, and keeping it takes two arrows meeting or fresh paper. None of that depends on what the mark is about. What the mark lets you lean on does depend on it.

Here's exactly what it lets you lean on. If your circles are sound, then any way of cutting them smaller is sound too, whatever the question is: every pair the smaller circles keep together was already together, and already matched. And for any grouping that isn't a cutting-smaller of yours, there's a question your circles pass and that one fails. To see it, find two buttons it puts together that your circles keep apart. Give everything in the first button's circle one colour and everything else another. Your circles are all one colour each, so they pass. The other grouping has both colours in one circle, so it fails.

So standing behind a sound grouping entitles you to exactly the groupings you get by cutting it smaller. A grouping that isn't a cutting-smaller can still happen to be sound for this question; paint all four buttons red and every grouping passes. But nothing in standing behind yours promises it. Change a button's colour after the mark was made, and the mark is still about the old colours. A record says what held when it was made, nothing more.

That's the three things from the start of this chapter in their smallest case: the object is the grouping, the checked relation is soundness for the question, and the paid event is the raise of the mark that keeps it. Like everything in this chapter, it says what somebody checking the run may conclude.

Which leaves the word on the cover. I named the toll "No Free Insight," and I've been using "insight" as if it meant one thing. The next chapter says what it means.

CHAPTER 13

What insight is

I named the toll "No Free Insight" before I had any careful account of what insight is. The book has been narrowing it down ever since. Chapter six took looking, copying and finding out off the list: those can be free. What's left is the part that stays. So it's time to say what that is, in a way that someone who studies real insight, in real heads, wouldn't laugh at. Who are those people, and what did they find? Here's what I ran into, pushed on as I went.

13.1 Sultan and the sticks

In 1913 the psychologist Wolfgang Köhler went to Tenerife to run a research station with a colony of chimpanzees, and the First World War kept him there until 1920, five years longer than planned, with the chimpanzees to watch. The famous one is Sultan. Food is outside the cage, out of reach. Two sticks are inside, each too short, but one fits into the end of the other. Sultan tries each stick and fails. He sulks. He plays with the sticks. And then, by Köhler's account, at a moment you can see, he joins them and reaches the food. Köhler's word was Einsicht, which came into English as "insight," and the whole school of psychology he belonged to, the Gestalt school, said that's what thinking is: a rearranging, the moment two sticks stop being two sticks and become one tool. Piling up tries until one works was, for them, something else.

Karl Duncker gave the human version, in German in 1935 and in English in 1945, five years after he died at thirty-seven; he'd spent his last years working with Köhler at Swarthmore. You can set it up yourself: a candle, a box of tacks, matches, and the job of fixing the candle to the wall. The hard part is seeing the box as a shelf while it's sitting there being the thing that holds the tacks. And he found that if you hand people the box empty, with the tacks poured out beside it, they solve it far more readily. Same objects. What changed was which relation between them was on the table.

I like both of these for the same reason: the insight didn't add an object. Sultan had both sticks the whole time. The box was there the whole time. What arrived was a relation, a piece of structure in chapter ten's sense, and it arrived all at once.

Hold on to that, because the thing the toll charges for also arrives in one step, and at its best it's also a relation.

13.2 It isn't a feeling

A feeling of getting closer is a different thing from solving the problem. In the 1980s the psychologist Janet Metcalfe asked people to solve problems while rating, every fifteen seconds, how close they felt to the answer, how "warm." On ordinary problems, like algebra, the warmth climbs steadily and peaks at the answer, which is what you'd expect. On insight problems, riddles of the candle-box kind, the warmth stays flat, flat, flat, and then the answer arrives. And the people who felt warmest beforehand were, if anything, more likely to be wrong. She put that in the title of a 1986 paper: premonitions of insight predict impending error. That's about how well warmth predicts in those tasks. It doesn't show that solving something comes with no feeling, or that everybody follows the same curve.

Try it yourself. Write a number from one to seven every fifteen seconds while you do a long sum, then while you do a riddle, and compare the two rows of numbers.

And the most famous insight story there is fits Metcalfe uncomfortably well. Archimedes, asked by Hiero, the king of Syracuse, whether a goldsmith had cheated him on a crown, gets into a bath, sees the water rise, and runs home naked shouting "eureka," I've found it. That comes from Vitruvius, a Roman writer on architecture, writing about two centuries later. Is it true? Galileo Galilei, at twenty-two, in 1586, took it as known that Archimedes caught the goldsmith. What he didn't buy was the method. Water spilling over the side, as Vitruvius tells it, was to him "a crude thing, far from scientific precision," and he guessed some writer had written down a rumour. So he wrote a little paper, La Bilancetta, proposing a balance weighed in air and in water instead, which he thought was probably what Archimedes actually did, since it rests on Archimedes's own results. Whether there was ever a bath, or a shout, nobody can check now.

Look at what the legend kept: the bath, the shout, running home naked. The feeling. What it left shaky is the part that would count as the check, the test that could have caught the goldsmith. The bit everybody remembers is the bit that tells you least.

So an account of insight that keys on the "aha" is keyed to something the evidence says is unreliable. Whatever insight is, it isn't the feeling of having it. That's a relief, because a machine doesn't have feelings, and a word that only meant a feeling would be cheating if I hung it on one. The version this book can price has to be keyed to something that happens in a step. The question is which step.

13.3 What happens when a computation learns

Keep the board from chapter four in front of you: the NO box, the YES box, the tally line, and the stamp. Add one more move that does nothing to the boxes; call it WORK. It stands in for every ordinary step a machine takes.

The first thing that can happen is plain looking. Something comes in and gets written down. A count ticks. Nothing is claimed about what it means, and the box doesn't move. The demon's notepad in chapter six was this. The statistician Dennis Lindley measured it in 1956, in a paper about how much information an experiment gives you, as how much your uncertainty is expected to drop. That's a real quantity, and it's about what was seen. On the price list it can cost nothing: the toll says nothing at all about a move that doesn't take NO to YES, so a price list may charge it zero. "May" is the word that matters. A price list is allowed to charge for looking anyway. And in a real device a look takes something too: chapter six showed it uses up a blank page, borrowed until the page is wiped.

Lindley's measure has a back story, and it runs through Bletchley Park. I. J. Good, who was Alan Turing's main statistical assistant there in 1941, wrote in 1979 that Turing had worked out how much evidence an experiment should be expected to give, to judge one of the codebreaking methods before running it, partly getting to Lindley's idea fifteen years early, in work that stayed secret. Turing measured evidence in "bans," with a tenth of one called a deciban, and the name came from the town of Banbury, where tens of thousands of the sheets used in that method were printed. And Good adds that Turing didn't know the measure he was using had already appeared in 1878, in a paper by Charles Sanders Peirce, the same Peirce from chapter nine, who'd called it the weight of evidence. It changes nothing here, but it shows how well trodden this kind of thing is. For a century and a half careful people have measured what an observation tells you, and every one of those measures is about what was seen. The toll leaves that side of the line alone.

The second thing that can happen is the raise. Some move takes the box from NO to YES, and that's where the toll applies: that move costs at least one, and every run from NO to YES pays at least one somewhere. What gets charged, at its best, is the Gestalt thing: a relation committed to, one tool claimed out of two sticks. I say "at its best" because chapter four was careful about what YES means: the coin is in YES, and nothing more. A raise that writes down which relation it's committing to carries Sultan's tool with it. A bare stamp commits to nothing at all, and it pays exactly the same.

The third thing isn't a kind of step. It's something about the price: it's the same, whatever got settled. Say the YES state also shows something striking, a deep relation, a result nobody could have guessed, Sultan's two sticks become one tool. The toll charges the raise at least one. It doesn't charge the striking raise more,

because the reason it charges anything only looks at the box before and after. On the board, the stamp from NO makes one mark whether you were thinking of Duncker's box or of lunch. A different price list could charge them differently. The toll doesn't make it.

The candle box has a footnote that says the same thing from the other side. In 1962 the psychologist Sam Glucksberg ran Duncker's problem with money on it: some people were told they could win five or twenty dollars depending on how fast they solved it. With the tacks in the box, the people chasing the money did worse than the people who weren't. With the box empty the problem was easier for everybody, and I'll leave what the money did there alone. Paying more didn't buy the insight, and on the hard version it got in the way. That's a fact about heads, not records, and I won't push it further. But it makes a nice parallel to the board. On the board a deeper raise isn't charged more, and in Glucksberg's room a bigger prize didn't bring the insight sooner. In neither place does the price measure the depth.

So: looking can stay off the bill, the raise can't, and the toll doesn't charge more for a deeper claim. Who decided that split? Not the prices alone. The toll prices exactly one thing, the move that settles. A price list is allowed to put a floor under more moves than that, and allowed to leave others at zero. The sorting into "moves that settle" and "moves that don't" is by whether the box moves, and that sorting is mine. The part that's forced is the floor under the move that settles, once you grant finite chairs, done staying done, and a price on two arrows meeting. That's chapter three. Anything a price list charges beyond that is a choice, and any real machine owes you its whole price list so you can see which choices it made.

13.4 Test, then commit

The philosopher Karl Popper, who you met in chapter nine, said a theory earns its standing by being exposed: you make a guess, you try to kill it, and what survives a real attempt counts for something, where "real" means the test could have failed. His word for that earned credit is corroboration.

Add a move to the board: CHECK. It looks at something, a sum, a coin under a cup, and it can come out pass or fail, and it doesn't move the coin. Charge it whatever you like. Charging it on the attempt, pass or fail, fits Popper, because what made his test count was that it really ran and could have failed. CHECK is the test, and it never takes NO to YES. The stamp is the commit. So the honest way to go is two moves: test, then commit, with two separate costs.

But nothing on the bare board makes the stamp wait for the check. The stamp takes NO to YES from anywhere. Press the stamp with no CHECK before it, and you land in YES and make one mark, the same box and the same mark as the player who checked first. Their tally lines differ by whatever CHECK was charged, and if

you charged it nothing, they don't differ at all. The receipt says a commitment was paid for. It doesn't say the commitment was earned. Popper's credit needs the test. The toll doesn't.

You can build a board that makes the stamp wait: let CHECK leave a mark when it passes, and let the stamp only take NO to YES when that mark is there. That's chapter eight's established record, and it's a good design. It's a design. Nothing in the definition asks for it. The gap between the word and the bare machine is real, and I'm leaving it where you can see it: the toll prices the commit, and whether a test came first is a fact about the run, or about a design somebody chose.

13.5 What the word means

So here's insight, in the sense this book prices: a distinction made to stay. A move that takes something from not settled to settled, for good, and that move costs at least one, on any machine with the toll, whatever it's about and however deep it is. At its best, what's settled is a relation committed to, a claim checked and closed. A bare stamp that commits to nothing pays the same, and the toll can't tell the two apart. An established record is one way to make sure it's the first kind.

Three limits, each from a different place. From Metcalfe: it isn't a feeling, so a machine's version has to be a step. From the board: the floor is the same for a shallow commit and a deep one, so price isn't depth. A machine can pay a lot to check something trivial, and Duncker's box, a real insight, is one small commit. And from test-then-commit: a paid stamp proves nothing on its own. Price and correctness are separate books.

Which leaves a suspicion I'd have in your place. Everything so far has been one box on one sheet of paper, with a rule I chose and a count I added. Is the shape of that box something I picked, or is it what any box the machine itself turns on has to look like, on any machine at all? That's the next chapter.

CHAPTER 14

A box over any machine

For the whole book so far I've been building up a box and a tally, and you'd be right to suspect that the whole thing is special pleading for one little board. So this chapter answers that suspicion with no particular machine in it at all. Take any machine that steps, anything with states and one next state for each. Put a box over it that watches it, under rules I'll spell out. Then ask: what can the box be?

The answer is one shape, and it's a shape you can build from a cup and a die.

14.1 A cup and a die

Put a cup upside down on the table. The rule is: the first time a six comes up, turn the cup right side up. There's no rule for turning it back over. Now roll a die twenty times. If a six comes up, the cup turns over on the first one and stays turned over. Write down which roll did it. That number is where the record happened, and the cup afterward is the record. If no six comes up, the cup stays down.

To make the die part of a machine with one next state, write the twenty rolls down first and read them off one at a time. The machine underneath is the list and a finger pointing at the next entry; each step moves the finger one place, whatever the cup is doing. The question the cup asks at each step is: is this entry a six? Nothing about the cup changes which entry comes next.

The cup is a latch, the same thing you built with two boxes in chapter four. Electrical engineers have had the circuit since 1918, when two British physicists teaching engineering in London, William Eccles and Frank Jordan, applied for a patent on a two-valve arrangement that holds whichever state you last put it in. Their 1919 paper calls it a trigger relay. It became the flip-flop, and the memory inside the device you're reading this on comes from it. Cut its reset wire and what's left has one input, "set," and one rule: the new state is the old state, or set. Once set, it stays set. The cup is that circuit with its reset wire cut.

Did anybody ever build a whole memory out of cups with no reset wire, on purpose? Somebody did, and for a reason close to this book's. In the mid-1950s Wen Tsing Chow, an engineer at the Arma division of the American Bosch Arma Corporation in Garden City, New York, was asked by the United States Air Force,

according to the Wikipedia history of the programmable read-only memory, for a safer way to hold the fixed numbers, the targeting constants, inside the guidance computer of the Atlas missile. What he and his colleague William Henrich patented, filed in 1957 and granted in 1962 under the plain title “Storage matrix,” was a grid with a connection at every crossing, where you set the numbers by taking out the connections you didn’t want, or burning them out with too much current. The word burn is right there in the patent, and people still say they burn a chip. A fuse blown in one stays blown. That proves nothing here, because nothing below cares what a cup is made of. But it’s good to know the shape wasn’t invented for a sheet of paper. Somebody needed it, for higher stakes than a die.

Here’s the claim, loosely first: every box that answers only to what the machine is doing, and never comes back down, is a cup with no reset wire, turned by some event the machine passes through.

14.2 What "answers only to the machine" has to mean

The content is in the rules, so here they are.

Call the machine underneath the base: the part the box watches. Now build a bigger machine that has the base inside it, plus whatever else you like, plus the box and a tally. The bigger machine has to run the base faithfully: start where the base starts, step when the base steps, and stop when the base stops. It can carry as much extra hidden stuff as it wants. The base doesn’t get to see any of that, and the extra stuff doesn’t get to change what the base does.

Now the five rules for the box and the tally.

The box’s next position depends only on what the base is doing right now and where the box is right now. Nothing else. No clock on the wall, no hidden dial, no outside hand. I’ll call that answering to the machine.

The tally never goes down.

The step that turns the box up costs at least one. That’s the toll.

Once up, the box stays up.

And some run, from a place the machine can start, actually turns it up. Otherwise the box is decoration.

Here’s what follows. Ask the bigger machine what it would do with the box down, above each state of the base: does the box go up on the next step, or not? Call that answer the event. Since the box answers only to the base, that’s one answer per base state; no hidden stuff can change it. Now take any actual moment. If the box is down, the event is exactly what happens to it. If the box is up, it stays up, whatever the event says. Put those together: the box’s next position is “already up, or the event holds right now.” That’s the cup’s rule. The cup turns over the first time the event holds and never turns back.

Look at which rules that used: only two of the five. Answering to the machine, and staying up. The tally, the toll, and the reachable turn played no part. A cup is what you get from "answers only to the base" and "never comes back down," all by themselves.

The other three rules matter for the other direction. Pick any base and any event the base can actually reach. Build the cup on that event, price the turn at one, everything else at zero. Check the five rules: the cup only looks at the base and itself; the tally only goes up; the turn costs one; the cup stays up; and the run that reaches the event turns it. So every cup is a box like that too. Put the two directions together: boxes that answer only to the machine and never come down are exactly cups, nothing more and nothing less, over any machine at all.

The closest relatives are in proving programs correct, and they're old. Martín Abadi and Leslie Lamport, in 1991, added "history variables" to descriptions of systems: extra state that records what has happened and never steers the system. A history variable that only ever turns on is the cup. And there's a logic for talking about the past, set out by Orna Lichtenstein, Amir Pnueli and Lenore Zuck in 1985, with an operator "once": "once h" is true now exactly when h has been true at some point so far. A program watching a running system keeps "once h" in a single bit with exactly the cup's rule, and Klaus Havelund and Grigore Roşu built watchers like that in 2002. What this chapter adds is the other direction: a box that answers to the machine and never comes down can't be anything else.

One thing about that argument cuts both ways: it never asks what the base is. You decide that, by deciding what goes in the base and what goes in the extra stuff. Draw the line low, with the base being only the machine's own working state, and "answers only to the base" is a real demand. Draw it high, put every table and every rule into the base, and it's easy to meet. The cup comes out the same either way. So the argument holds wherever you draw the line, and that's exactly why it can't tell you where a record's meaning lives. The meaning lives in what the event looks at, and that's in the base, by how you drew it. When you meet a real machine, ask it first what it puts below the line.

14.3 A base that never merges

What about a base where no two arrows ever meet? Can it carry a cup?

It can, and you can see exactly what it pays. Keep every earlier position of the cup on a strip that grows by one entry each step, newest first. Then the whole machine still never has two arrows meeting: if two whole states land in the same place, the strips are the same, so the old cup positions were the same, and the base parts were the same because the base never merges, so the states were the same. But the strip gets longer every step, so the machine never comes back to a state it's

been in. That's the fresh-paper column from chapter two. And on a machine with finitely many states, there's no fresh paper to be had, so a box that stays up can't be turned on by a move where no two arrows ever meet. That's the chairs, again.

People who study processes that talk to each other ran into the same thing from the other side. Vincent Danos and Jean Krivine, two computer scientists in Paris, wanted to pin down what a transaction is. In 2004 they built a version of Robin Milner's calculus of processes where every step can be rolled back, and in 2005 they modelled a transaction in it as a process that wanders freely through everything that can be undone, until it does something declared impossible to undo: the commit. As I read it, they start from steps that all come undone and ask what has to be added before anything sticks. This chapter starts from a box and asks what the step under it has to be. Theirs is declared. I thought mine was forced by counting, since on a finite machine the first turn of the cup makes two arrows meet whatever anyone calls it. Push on that and it gives: I choose the event, and on a base that grows its history forever, nothing merges at all, so there my staying up is declared too.

14.4 The clean start

There's an arrow of time in this, and I should say where it came from, because I put it there by hand. The cup starts down. That's the only reason a turned cup means the event happened: it was down at the start, and only the event turns it. Turn the cup up before the first roll and it tells you nothing about the die. So the clean start is assumed. It's a small version of what the philosopher David Albert, in 2000, named the past hypothesis: an assumed low-entropy beginning of the universe, an idea that goes back to Boltzmann. Several accounts of why records point to the past lean on it. Whether every trusted record leans on it is their argument, and I'm not settling it. Every result in this book that reads a turned cup as news that the event happened assumes a clean start and starts from there. The cup argument above doesn't need it, and that's exactly why, on its own, it can't say the event happened.

When I went looking for who else had found the arrow hiding in the start, I got five papers that read like five unrelated answers. They lined up once I asked each the same question: what does the cup being down at the start rest on? There are two places to put it. At the beginning, as a fact about how things started. Or in the structure of what causes what.

David Wolpert, a physicist, sorted memory systems in 1992, and one of his kinds works only because it begins in a prepared state, set up by a process that runs one way. That's the cup put down by somebody before the game. Wolpert and Jens Kipper, a philosopher, put that sorting on firmer footing in 2024, and they say plainly that they have no formal proof this kind can only remember the past. I trust them more for that sentence.

Aram Ebtekar and Marcus Hutter built a toy world in 2024 where every step can be reversed, imposed a clean start on it, and got records of the past and none of the future. As I read their route, the clean start goes through cause and effect first, and the arrow comes out of that. Thomas Blanchard, a philosopher, got the same lopsidedness of knowing out of a condition on cause and effect, also in 2024. So he starts in the middle of Ebtekar and Hutter's road. One is a toy world and the other is philosophy, and the fit between them is my reading.

And Carlo Rovelli, two years before his paper on the cost of choosing from chapter six, studied traces in a 2022 paper called "Memory and Entropy," and concluded that a memory turns a low-entropy start into information you can use. He's at the first place, and he says what the beginning gets spent on. Read next to the cup: that's the blank the record uses up. The cup can say yes once because it started at no.

So does the cup explain why records point backwards? It doesn't, and I'll say that plainly. The step runs forward and the runs start clean, and anything built on those two can only hand the arrow back. Even the strip that grows forever starts empty, so it leans on the same blank. On the map, this book sits at the first place, with Wolpert and Rovelli, and it sits there by assumption.

Constructor theory, David Deutsch and Chiara Marletto's way of writing physics from chapter two, has a 2025 paper on time saying that laws stated as which transformations are possible can't mention time at all, so duration has to come from a second system you count, a clock. This book is in that position by construction: the only time in it is the machine's own steps. The hidden clock in the next section gets thrown out by the first rule, which read next to them is a rule about which system is allowed to be the clock: the one being recorded. But their laws have no step in them, and I put a step into the base by definition. So I'm in their position only at the far end of their argument, where a clock already exists. The base is my clock, and I assumed it.

Physicists have built this cup on paper in quantum mechanics too, and it behaves the way the reversible base above says. The question that sorted them for me is where the cup puts what it forgets. Klaus Hepp, in 1972, wanted a measuring device inside quantum mechanics itself, and studied a particle that flips spins in a long chain as it passes, now called the Coleman-Hepp model. I haven't read Hepp's paper itself; I know it from John Bell's, which took it apart in 1975: the record is exactly definite only if the chain is endlessly long. That's the fresh-paper column, a history that never ends. Jonathan Halliwell, in 1999, used a two-state detector that flips once when a particle passes and is kept from flipping back by a large surrounding environment, and his summary calls it "effectively irreversible." I like that he puts the word "effectively" right there where you can see it. And Murray Gell-Mann, who named the quark, and James Hartle gave the property a name in 1995: the permanence of the past, past alternatives that stay settled as the story goes on.

Hepp's chain had to be endless. Halliwell's had to live in an environment. Mine stays up, and the toll writes a unit on a tally. That's fine in a model of computation, and it's exactly where this stops being physics.

14.5 Both rules do work

A result with two conditions should have two counterexamples, one for each condition dropped. It does, and they're the two ways a record goes wrong in the world.

Drop "stays up." Take a base that just counts, and a box that flips at every step: up, down, up, down. It answers to the machine. It doesn't stay up. And it's not a cup on any event. That's the coin from chapter three, the toggle: working state dressed up as a record. If a take-back has to stick, it's a second cup, "withdrawn," which is how lists of revoked web certificates actually work: you add an entry; you never delete the certificate. Alin Tomescu and his coauthors built add-only lists like that for public logs in 2019, and a revocation fits exactly: it goes in once and stays. Two cups can also look at each other: try it with two cups where the second only turns on an even roll if the first is already up, and you'll see the second can never turn before the first. Each is still a cup; its event just gets to look at the other one.

Drop "answers to the machine." Take a base that counts, and a box that turns up at the fifth tick of a hidden clock, whatever the base is doing. It stays up, and it doesn't answer to the machine. That's a timestamp. A timestamp says when, not what happened. A record switched by an outside party or a clock records that party or that clock, and leaves the computation out. Try it with the written-down die rolls: keep a second, hidden tally beside the list, and turn the cup when that tally reads five, then move both the tally and the finger. Start two runs at the same place on the list with the cup down, but one with the hidden tally at five and one at zero. The next cup positions disagree, although everything about the base agrees.

One more thing about the timestamp, because it's sneakier than it looks. Start it in step with the base, hidden tally and finger together. Then, on every state its runs actually reach, the hidden tally equals the position in the list, and the timestamp is exactly the cup on "this is position five." The first rule only throws it out because the rule looks at every state, including the out-of-step ones no run from a clean start ever reaches. So the honest version is: a timestamp with its clock free to start anywhere fails even on the states its runs reach. Started in step, it's a cup on the runs it has.

Permanent but not answering to the machine is a timestamp. Answering to the machine but not permanent is a toggle. A record is the thing that's both. That's the most useful single sentence in this book for looking at systems other than the ones I build.

14.6 Arguing with cups

People who study arguments built the two-cup rule for debates. Give each speaker a row of cups, one for every claim they've made. Everybody I read in this field is answering two questions about those cups: whose hand turns them, and can they come back down?

C. L. Hamblin, an Australian philosopher and logician who also came up with reverse Polish notation for early computers, wrote Fallacies in 1970, and it has a chapter where each speaker has a store of what they're on the hook for, and the rules of the debate update it, not the speaker. I haven't read that chapter itself. That's "answers to the machine," for arguments: the speaker's hand is kept off the cup. Henry Prakken, who works on AI and law, built debate games in 2005 on stores like that and made the claims sticky: a claim can be taken back only after it's been challenged, and taking back has a cost. He also floats a rule that once a speaker takes a claim back, they can't take it on again, which is a revocation that stays revoked. Erik Krabbe, an argumentation theorist, names the awkward part in 2001: taking things back is frowned on, and yet settling a dispute takes somebody taking back a doubt. A store that never gives can't settle anything, and a store that gives whenever asked isn't a store. This book answers Krabbe with a price: a take-back is a second cup, "withdrawn," and turning it costs at least one like any other turn. Where mine gives is Prakken's first condition: nothing in my cup asks why the second cup turned. Its event can look at the base and the first cup, and nothing requires a challenge.

The same field has something that looks a lot like earning. In Phan Minh Dung's 1995 setup, arguments attack each other, and the most careful way of accepting them starts from the arguments nobody attacks and keeps adding the ones they defend. Things go in and never come out: a row of cups turned one at a time. Pietro Baroni and Massimiliano Giacomin in 2007, and Martin Caminada in 2014, worked out a stronger version where every accepted argument is defended by others that don't lean back on it, so the support bottoms out. That's an earned claim, in argument form. Notice the bottom, though: an argument nobody attacks gets in on a check that can't fail, and chapter eight says a check that can't fail establishes nothing. Turn that on my own cups, because it lands. My first turn needs only that the event holds, and I charge one for it, and a charge isn't a check. So my earned records bottom out the same way Dung's do, in something let in without an argument. The one difference is that mine pays to get in.

14.7 A box that can change its mind

A box that's allowed to change its mind isn't lawless. It's somebody else's subject. Count how many times the cup may be turned. Once, from down to up, is the cup. More than once and you're somewhere else, and the people who went there sorted it by how many turns are allowed, and by how long the game runs.

The philosopher Hilary Putnam and the computer scientist E. Mark Gold, in back-to-back papers in 1965, looked at questions a machine answers by guessing and revising, where the answer is whatever the guesses finally settle on. Putnam's paper says outright that you're never sure you've got the right answer. What kind of answer can't you ever be sure you have? The way I read it: the cup's position at any moment is the current guess, the answer is where it ends up, and nobody watching can tell when it's done moving. Yuri Ershov, a logician in Novosibirsk, sorted those in 1968 by how many changes they're allowed. That's how Rod Downey, Carl Jockusch, Timothy McNicholl and Paul Schupp describe his ladder in 2014; I haven't read Ershov's paper. Allow one change, from no to yes, and you get exactly the sets a program can list, the bottom rung. That one you can check yourself. If a program lists the set, guess no for every number and switch to yes when the number shows up on the list. Going back, if every guess flips at most once, from no to yes, run all the guessing side by side and write a number down the moment its guess says yes.

Now put Putnam's worry on the cup, because it lands. On the bottom rung, a yes is final the moment it shows up, and a no never is, since down means "not yet." A single cup can establish that something happened, and can never establish that it didn't. A final no would take a second cup, turned by finding a reason, and that's the two cups again. That's what the bottom rung costs. This book stays on the bottom rung.

The ladder doesn't stop at endless time either. A definition that only ever adds, stage after stage, and keeps everything at each limit, is a cup run through the infinite; Wayne Richter and Peter Aczel set those out in 1974. Let it take things back and keep only what stays put from some stage on, and you have what John Burgess, in 1986, called quasi-inductive definitions, which he met in theories of truth that grew up beside Saul Kripke's 1975 theory of truth built up in stages. And Joel Hamkins, the same set theorist as in chapter one, and Andy Lewis let a Turing machine keep running past endless time in 2000. At each limit, a square keeps its value if it settled, and reads 1 if it kept flipping forever. Their favourite trick is the toggle: flash a flag every time a guess changes, and at the limit the flag is on exactly when the guess changed endlessly often. Past infinity, the toggle is the clever one, because a cup can only tell you a thing happened at least once. I threw the toggle out anyway, and I'd do it again, because before the limit its reading can't be leaned on, and nobody using a real machine gets to wait for a limit.

One finite-time fact belongs beside that, so nobody reads too much into permanence. Hao Wang, in 1957, described a machine whose only write is to mark a square and which can never erase one, and it computes everything a Turing machine does. Permanence of a machine's own memory costs it no computing power, given an endless tape: each mark goes on a square nobody marked before, which is fresh paper again. What this book prices is a box raised by a step that has to work whether or not the box is already up. So the cost is in what a box can claim, not in what the machine can compute.

14.8 One shape, two choices nobody forces

"There's only one way to carry a record" packs three different claims into one sentence, so take them apart.

Given the toll and a fixed price list, is there only one way to keep the books? Yes, on every state you can reach: that's chapter eleven's one honest count, and it assumes the price list.

Is every box over a base the same machine, charges and all? No, and the counterexample is embarrassingly simple. Take any box over any base, and give it a new tally that adds each step's old cost plus one more, whatever the step is. Walk the five rules. The box still answers only to the base and itself. The tally still only goes up. The turn still costs at least one; the toll is a floor and a surcharge only raises it. The box still stays up, and the same run still turns it. So the surcharged machine passes, and it isn't the one you started with. Do it with the cup and the die: charge one for every roll, six or not. The cup turns on the same roll as before. You've changed the cost and nothing about the record. A machine that charges for running time passes, and it isn't the one that doesn't.

And the second choice nobody forces, the one that still surprises me: which event is the record? Nothing in the five rules picks it, beyond the demand that some run reach it. You saw that above: for any event the base can reach, the cup on that event, priced at one, passes. Do it with the die. One cup turns on the first six. Another turns on the first one. Both pass. Roll a one first, and until a six shows up, the two cups disagree about whether anything has happened. So which permanent box counts as "the record" is a second thing nobody forces on you. The price list is one free choice and the event is the other, and that second one is what a later chapter is about: what, if anything, makes "a check passed" the event worth recording.

Now let the cup wobble. Say whatever turns it over sometimes does and sometimes doesn't, with chances you could write on a card. Does fixing the price list fix the chances? No: two cups with the same possible moves and the same charges, one turning with a fair coin and one with a two-to-one coin, pass the same rules

and aren't the same. And a cup that, from down, can either stay down or go up, depending on chance, isn't a cup on any one event at all.

And is there one set of states, up to renaming, that every box machine has to have? No, because the bigger machine can carry any hidden stuff it likes.

14.9 What leaving the base open buys

So here's the answer to the suspicion I started with. The box isn't special pleading for one board. Over any machine that steps, a box that answers to the machine and never comes down is a cup on some event, and the argument doesn't care what the machine is. What a particular machine adds on top of the cup is a price list and an event, and both are choices. The surcharged cup shows the first: the price can change and the record doesn't notice. The two cups on one die show the second: the rules are happy with either event. That's also why the chairs in chapter two never needed the word "check": they force two arrows to meet in whatever stays.

And chapter five already showed what happens when a box has more than two positions: it's a row of cups, one for each line, each one a cup like this one, where each cup's event may look at where the row already is.

CHAPTER 15

The machine with the record dropped

Chapter seven put the machine behind a window and asked what gets through. Two states can give the same view and differ in what matters, and a price worked out from the window can't be exact if there's a pair like that. That was for one box. Now, with a whole row of cups from chapter five and the cup over any machine from chapter fourteen, I can ask the sharper version, which is also the one I care about. Take the ordinary machine, the one with no cups and no tally, and look at what's missing from it. What's the shape of everything it can't see, and what does the toll pay for, in terms of that shape?

15.1 What's left when you cover the cups

Back in chapter four you covered a coin with your hand and watched the rest of the machine run, move for move, as if the coin weren't there. Do that with everything: cover the whole row of cups and the tally. What's left is a machine that steps the same way and knows nothing about either.

That only works if covering them doesn't change how the rest moves. If some step of the machine looks at a cup and goes one way or the other because of it, then covering the cups leaves you with a machine that can't decide what to do next. So the honest condition is: nothing in how the rest of the machine moves depends on the cups or the tally. Where that holds, call what's left the uncovered machine. It's the ordinary computer.

Over any one state of the uncovered machine there's a pile: all the full states that look like that one once you cover the cups, differing only in their cups and their tally. That's chapter seven's pile behind one picture, with the window being "everything but the cups."

Every move has piles of its own too: the states that move sends to the same landing place. Call those its landing piles. A pile behind a picture is everything that looks the same before anybody moves. A landing pile is everything that ends up in

the same place after one move. Keep the two kinds apart, because this chapter turns on how they meet.

15.2 Pricing through a window, with a whole row

First the window question again, with a row of cups. Say a clerk prices each step from what comes through a window before and after it. The price never misses if it charges at least one for every step that turns up a cup. It's exact if it also charges nothing for every step that turns up no cup. A clash is two steps that look the same through the window, where one turned up a cup and the other didn't.

With one box, the fix in chapter seven was to put the box in the window. So you might guess, for a row, that the window has to show some of the cups. Try the smallest case. Let the record be a count that climbs: 0, 1, 2, and so on, with a cup for "at least 1," a cup for "at least 2," and so on. Let the window show only the first cup, "at least 1." Let the count go up on some steps and sit still on others. Once it's past one, that first cup is up before and after both kinds of step. So a step that turned up a later cup looks exactly like a step that turned up nothing. That's a clash.

So, you might guess next, the window has to show every cup. That's wrong too, from the other side. Same counting record, with two moves: one adds one, the other leaves it alone. Now let the window show only whether the count is even. You can't read the count off that window: 0 and 2 look the same through it. But look at when evenness changes. It changes on exactly the steps that add one, and on no other step. So "charge one exactly when evenness changed" is exact, every time.

That changed how I think about windows. A window doesn't have to show the cups to price them. It has to show them move.

The rest of the clerk's story carries over from chapter seven unchanged. If the window has a clash, no price from it is exact, and any price that never misses has to overcharge at least once for every step that turns up nothing but looks like one that does. That's the least it can overcharge, and there's a price that overcharges exactly that much. If the window has no clash, an exact price exists, and on a machine with finitely many states you can work it out.

15.3 What a paid move destroys

Now the piles. Chapter two showed that on a finite machine, a move that settles something for good makes two arrows meet somewhere. Chapter five showed the same for a step that turns up any cup, once cups only go up. Which two arrows? It matters a great deal.

If the two states that meet sit in different piles, the uncovered machine lost a distinction too: two of its own states became one. If they sit in the same pile, the

uncovered machine lost nothing. What got crushed was only ever under your hand: two states that looked the same once the cups were covered, told apart only by their cups or their tally, now made one.

You might guess it's always the second kind, because the cups are where everything happens. It isn't always. It depends on whether the uncovered machine forgets anything on its own.

Here's the case where it's the second kind. Say the uncovered machine, for this move, never lands two of its own states on one. Two full states meet. Cover the cups, and their uncovered parts land in the same place too. But the uncovered machine keeps its states apart, so those two uncovered parts must have been the same to begin with. So the two states that met sat in the same pile. In that case every landing pile of the move lies inside one pile behind one picture.

And here's the case where it isn't. Take the stamp from chapter four, and let the window show the box itself, so each state is its own picture. The stamp sends NO and YES to YES. Two states meet, and they sit in different piles, because the uncovered machine is the stamp itself and the stamp merges. Nobody ever had to cover anything.

One more fact about landing piles, and it ties back to chapter three. The halving rule said each unit of cost pays for at most one halving. That holds exactly when no landing pile of a move has more than two to the power of its price in it: a cost of one allows piles of two, a cost of two allows piles of four. And that holds on a machine of any size.

So on a machine with finitely many states, a record that only grows, and an uncovered machine that forgets nothing, the toll is a price on crushing a pile: on taking two states that look the same once the cups are covered, told apart only by what's under your hand, and making them one.

15.4 What's left over is the ordinary computer

Put that together and you get the shape I've been circling since the night with the Flatland video that the afterword tells about.

An ordinary machine, read through anything its moves never raise, is what's left when you cover the cups of a machine that has them. Chapter four fitted the first cup. Over every state of the ordinary machine there's a whole pile, states that differ only in their cups and tally, and the ordinary machine can't tell them apart, as long as none of its steps looks at the cups. On a machine that obeys the toll, a step that turns up a cup costs at least one, whichever cup and however many. On a finite machine whose record only grows, when the ordinary machine forgets nothing, what that cost pays for is exactly crushing two states of one pile into one. A window

can charge the right price without showing the cups, if it shows them move. And with the halving rule, the cost in halvings is never less than what got crushed.

What I haven't shown is that any of this is out there in the world, that nature keeps a direction like this over its own uncovered machines. That's a belief of mine, and nothing in this book proves it. What I have shown, and you can check, comes with its conditions showing. A machine whose box answers to the computation and climbs an order is a row of cups, one cup for each line. If it also has finitely many states, never lets its record come down, and prices two arrows meeting, it has to pay this way. And when its uncovered machine forgets nothing, what it pays for is exactly this. It's less than the Flatland night promised me. It's also the part of it that held when I pushed.

15.5 Shortcuts

There's a question every machine with a record runs into once its programs can be handed their own code. A box is worth turning up because something later can glance at it and not have to work the thing out again, the way chapter ten's glance reads a block's bucket off a table without applying the rule from scratch. Call any move like that a shortcut, and call it sound when it gives the same answer the long way would have.

So can a machine like that decide, from inside, whether its own shortcuts are sound? That question runs into a wall, the same wall Kurt Gödel and Alan Turing hit, and I have to show you that the wall is there before I show you anything that pretends to be on the other side of it. That's the next chapter.

CHAPTER 16

What a machine can't decide about itself

In chapter ten I said you'd watch the same move get made again and again: somebody says "everything is X," it's true enough to do real work with, and then somebody draws the diagonal. I owe you the diagonal itself now. The last few chapters handed a box to any machine that steps, and the next thing anyone wants from a machine with a box is for it to tell sound shortcuts from unsound ones by itself. Before I let anything claim that, I have to show you the wall every such claim runs into, and then exactly where a machine with a box stands next to it.

It doesn't stand on the far side, because nobody's on the far side. A box keeps a distinction and pays for it. What it can't do is the judging.

16.1 Five names and a table

Do this first, with a pencil, because the whole family of arguments is this one trick.

Write five friends' names down the left of a page. Next to each name, write a row of five digits, each 0 or 1, anything you like. Now build a new row. Its first digit is the opposite of the first digit of the first row. Its second digit is the opposite of the second digit of the second row. And so on, down the diagonal. Look at what you've made. It differs from row one in place one, from row two in place two, from every row at that row's own place. So it's nobody's row. Your table of five missed something.

With five rows that's a cute fact. Georg Cantor made the point in 1891 that it works for any list, however long, even an endless one: hand me a complete list of all endless rows of two symbols, and I'll walk the diagonal flipping as I go and hand you back a row that's on no line of your list. So there's no complete list. There are more rows like that than there are places on any list, and that's where "some infinities are bigger than others" comes from. (He wrote his two symbols as m and w, and that paper wasn't even his first proof of the fact. The first was in 1874, by a

harder road, and in 1891 he called the new one much simpler. The diagonal was him finding the easy way.)

Now look at what the argument did, because it's going to happen again, in new clothes. There was a table: rows are the things being listed, columns are questions, and each square says "does row i do column j." There was the diagonal: the squares where a thing gets asked about itself. There was a flip, something with no fixed point, like turning yes into no, applied along the diagonal. And what got knocked down was the thing assumed at the start: the complete list. Nothing outside the assumption was touched. A table, a diagonal, a flip, and the assumption eating itself.

16.2 The same move, again and again

Russell, in 1902, from chapter ten: the collection of all collections that don't contain themselves. That's Cantor's flipped diagonal with "contains" as the table, and Russell said as much in 1903: he got there by running Cantor's argument on a supposed collection of everything. What got knocked down was the assumed collection of everything, and Zermelo's fix was to stop assuming it.

Jules Richard, a French mathematician who taught at schools in Tours, Dijon and Châteauroux, noticed in a short note in 1905 that some phrases define numbers ("the ratio of a circle's circumference to its diameter"). There are only as many phrases as there are whole numbers, so list the numbers they define, run Cantor's diagonal down their decimal digits, and you get a number no phrase defines, except that you just defined it, in the phrase that describes the diagonal. The lesson I take from it is the one this whole chapter turns on: keep apart the language a system talks in and the language you use to talk about it. Mix them and the diagonal bites you.

Richard's paradox has a cousin small enough to do at the table, and it comes with a footnote I can't resist. Russell, the first to put it in print, in 1906, credited it to G. G. Berry, a junior librarian at the Bodleian Library in Oxford. In the version in Principia Mathematica it's about syllables. Take the phrase "the least integer not nameable in fewer than nineteen syllables." There are only so many phrases shorter than nineteen syllables, so they name only so many numbers, so some least number is left over, and the phrase picks it out. Now count the phrase: the, least, in-te-ger, not, name-a-ble, in, few-er, than, nine-teen, syl-la-bles. Eighteen. In eighteen syllables it has named the number that can't be named in fewer than nineteen. "Nameable" is a word about the language, used inside the language, and that's where the bite is.

Kurt Gödel, 1931, with an improvement in 1936 by Barkley Rosser, a student of Alonzo Church, that lets it assume only that the system doesn't contradict itself. Take any such formal system, with rules you can check mechanically, strong enough

to do some arithmetic. Then there are sentences it can neither prove nor disprove. The engine lets you build, for any property you can write down, a sentence the system proves equivalent to "my own code number has that property." Apply it to "not provable," and you get a sentence equivalent, inside the system, to its own unprovability. The table is the sentences listed by code number, the diagonal is a sentence about its own code, and the flip turns provable into not provable. Two cautions, which the Stanford Encyclopedia stresses too: the sentence doesn't literally "say" anything about itself; it's provably equivalent to a claim about its code, which is less mystical and more exact. And the theorem doesn't say the sentence can't be known. It says this one system can't settle it.

How did the world take it? Mostly, at first, it didn't. Gödel, twenty-four years old, announced the first theorem in a round-table discussion on the last day of a conference in Königsberg in September 1930, and by the account of Solomon Feferman, one of the editors of Gödel's collected works, the room met it with general incomprehension, except for one man. John von Neumann pulled Gödel aside, kept thinking about it, and a couple of months later saw that the argument gave something more: a system like that can't prove it doesn't contradict itself. He wrote to Gödel with it in a letter dated 20 November 1930. Gödel had already got there; his paper had reached the journal three days earlier with that second theorem announced in it, and von Neumann left the credit with Gödel. It's "somebody got there first" again, except this time the person who got there second was von Neumann.

Alan Turing, 1936. Chapter one told how what he actually proved differs from the halting problem as it's taught. The taught version fits on two index cards. Card A is the Checker: hand it any program card and it announces "stops" or "runs forever." Card B is the Contrarian: "Hand me to the Checker. If it says I stop, run forever. If it says I run forever, stop." Hand B to A, and either answer is wrong. So there's no Checker. The table is programs against programs, the diagonal is a program run on itself, and the flip turns stop into loop.

Stephen Kleene, 1938, and this is the one that turns the paradox into a tool, so slow down. Everything above needs the thing that refers to itself to exist: Gödel's sentence has to be buildable; the Contrarian has to be able to "hand me to the Checker." Kleene's recursion theorem says: for any way of transforming programs that you can compute, there's a program that behaves exactly as if it had been handed its own source code. Any program may be written as though it already had a copy of itself in hand. He stated it almost in passing, in a paper about something else, and proved it in the last two lines of one section. I have that count from Yiannis Moschovakis, a logician who wrote a 2010 paper about it called "Kleene's Amazing Second Recursion Theorem," and he calls it a little gem. It's why programs that print their own source code exist in every ordinary programming language. Try one in

English: “Print the following twice, the second time in quotes: ’Print the following twice, the second time in quotes:’” Obey it.

Those programs are called quines, and the name comes from Douglas Hofstadter in 1979, in Gödel, Escher, Bach, in honour of the philosopher Willard Van Orman Quine, who had made a long study of self-reference that works indirectly. Quine left a sentence you can take apart at the table, from a 1962 essay on paradox: “yields falsehood when preceded by its quotation” yields falsehood when preceded by its quotation. Do what it says. Take the part in quotation marks, put a quoted copy of it in front of itself, and look: it’s the whole sentence, word for word. So the sentence says that it itself is false. It’s the liar paradox, built without any “this sentence,” just an instruction to quote and copy. Quoting and copying is Kleene’s trick in plain English. The paradox is what you get when the copy says something about its own truth. The tool is what you get when the copy just does something.

Henry Rice, 1953, whom you met in chapter seven: the general form. Any question about what a program does, not how it’s written, that some programs pass and some fail, has no program that answers it for all programs. “Does this program stop” is one. “Does this program ever print 7” is another. “Does this program contain the word ’if’” isn’t, because that’s about the text.

And the one that’s an attack. Ken Thompson, who built the original Unix operating system, gave a lecture published in 1984. Build a compiler that, when it compiles the login program, plants a back door, and when it compiles a compiler, plants both habits in that one too. Then delete the trick from the source code. The compiled compiler still carries it, and recompiling a clean source with the poisoned compiler plants it again, forever. Reading the source never shows it. Picture a locksmith who teaches an apprentice to cut a spare for every town-hall key, and to teach that to every apprentice they train, and never to write it in the manual. Years later the manual is spotless and every locksmith in town cuts the spare. Thompson’s compiler is Kleene’s trick with a knife in it.

Thompson wasn’t first either, and he said so: he wrote that he first read of the possibility in an Air Force review of the security of an early operating system called Multics, and asked anyone who could find it to send him the reference. It’s Paul Karger and Roger Schell’s 1974 security evaluation of Multics, which already described a trap door planted in a compiler that keeps itself alive. Is there any way out? A partial one. David A. Wheeler, in 2005 and in his 2009 dissertation, worked out a check: compile the suspect compiler’s source with a second, independently made compiler, use the result to compile that source again, and compare it bit for bit with the suspect. If they match, under his stated conditions, the source and the binary correspond. Two compilers having different authors is useful evidence, not a substitute for those conditions. The question has moved to a place where we can name what we trust.

And Bill Lawvere, in 1969, proved a theorem that, as I read it, makes all of these one theorem in disguise: if there's a function with no fixed point, a flip, then no thing can list every question about itself. Cantor, Russell, Gödel and Turing all come out as cases. Noson Yanofsky retold it in 2003 without the heavy machinery. I mention it because "it's the same wall" is a claim I make below, and it's fair at the level of the argument's shape. Every case has its own fine print about who counts as a judge, and that fine print is exactly where most overclaims live.

16.3 Where the diagonal gets misused

Before pointing it at a machine with a box, look at how it gets misused. Each time, the trouble is forgetting who the judge is and what it has to judge.

"Gödel shows minds aren't machines." John Lucas, a philosopher at Oxford, argued it in 1961. Roger Penrose, the Oxford mathematical physicist, revived it in two books, The Emperor's New Mind in 1989 and Shadows of the Mind in 1994. The standard replies, as far as I can tell, have held up: the Gödel sentence is true only if the system doesn't contradict itself, so a person who "sees" it's true has to know that, which is the hard part; and nobody can build the Gödel sentence of a system they can't write down, so people might be machines too complicated to diagonalise. The theorem has an "if" in it, and the argument drops the "if."

The part that's easy to miss is that Gödel himself didn't go as far as Lucas. In a lecture on 26 December 1951, at a meeting of the American Mathematical Society at Brown University, he drew only an either-or from his own theorems: either the human mind infinitely surpasses any finite machine, even in pure mathematics, or there are questions about whole numbers, of a certain simple kind, that no proof a human can conceive will ever settle. Feferman, writing about that lecture, says Gödel privately believed the first half and still stated only the either-or, because he didn't have a knock-down argument against the machine side. Gödel meant to publish the lecture and never did. So the man whose theorem gets quoted for "minds aren't machines" stated an either-or and left the deciding to whoever came next. The "if" is there again, kept by the person with the most right to drop it.

"Undecidable means unknowable." No. Two senses share the word: no single program that's right on every case, and a sentence one particular system can't settle. Neither means nobody can know a given answer. Any single case is easy in the first sense: for any one program, one of "say yes" and "say no" is right. What's ruled out is one inside procedure that's right on every case.

"Everything interesting is undecidable." No. Rice is about what programs do, over all programs. It says nothing against checking the text, nothing against answering for particular programs, nothing against checks allowed to say "I don't know," nothing

against checking finite machines. Real compilers and analysers live with Rice by being careful and incomplete, and what limits them in practice is mostly cost.

"Self-reference is a paradox." It's a theorem with a construction, as Kleene, the quines and Thompson show.

16.4 Now point it at the box

Here's the question with nothing in it but what this book has built. A machine steps. A box rides along and turns up on an event, chapter fourteen. When the event is "a check passed," chapter eight says the box lets you lean on a claim, and for a claim that a shortcut is sound, that's what makes the shortcut usable. So: can a machine like that have, as one of its own programs, a procedure that correctly decides, for every program, "does this program have a sound shortcut"? Call that an autopilot: hand it any program and it says yes or no, and it's always right.

The exact claim is narrower than that question in plain words, so here it is first. "Has a shortcut" can be any question about programs with two features: two programs that behave the same always get the same answer, and there's one program that passes and one that fails. That's all the argument gets to look at. It doesn't see soundness. Whether a particular question about shortcuts has those two features is a separate fact, checked one machine at a time. A procedure that only answers for some programs, or needs outside evidence, isn't what the wall rules out.

And because the argument only looks at what programs do, not how, "shortcut" here is a label. A question like "is there a cheaper program that gives the same answers?" isn't the argument's question. The nearest thing I know of is Manuel Blum's speedup theorem from 1967, and it goes somewhere stranger: there are functions with no fastest program at all, where every program for them has another that's faster on all but finitely many inputs. That's about cost, not deciding, and nothing in this book proves anything there.

The answer is no, under conditions, and the proof is the two index cards. Suppose there were an autopilot that says yes exactly when a program has a shortcut. Build the flip: "If the autopilot says yes about me, behave like the program that fails; if it says no, behave like the one that passes." If the machine has Kleene's trick, so a program can be handed its own code, and if that flip is something the machine can run, and if the question only depends on what programs do, and if there's one program each way, then there's a program that behaves exactly like its own flip. Ask the autopilot about it. If it says yes, the program behaves like the one that fails, and behaving alike means getting the same answer, so it fails: the autopilot was wrong. If it says no, the program behaves like the one that passes, so it passes: wrong again. Those are the only two answers.

Look at what that used: Kleene's trick, the flip being runnable, the two examples, and "behaves alike, answers alike." It never touched the tally, the toll, or the box. It's Rice's trick one floor over, with its own conditions, so it isn't Rice's theorem word for word, and it says nothing about whether this question is harder or easier than the halting problem.

So the wall is there. The honest part is who counts as the judge. The argument rules out a judge whose flip the machine itself can run. It says nothing about a judge standing outside. An outside judge can exist for the very same question, right on every program. The line between inside and outside is Richard's lesson: the language you talk about a system in isn't the language it talks in.

16.5 What this does to the book

It does two things. It keeps me honest: nothing in this book claims a machine with a box can check its own shortcuts on autopilot, because that's a wall, the same wall, and I've shown you the bricks. And it tells you what a box can do instead: it can carry a shortcut along with its receipt, so that "was this earned?" is answered by a step you can point at. No judge has to exist for that, which is lucky, because on a machine with Kleene's trick no inside one can. The diagonal says the judging can't be automated from inside. The cup says the commitment can be recorded. Different jobs, and a box does the second.

The diagonal has a famous cousin that looks like it ought to bite a box too. Gödel's second theorem says a system strong enough for arithmetic that doesn't contradict itself can't prove that it doesn't, and Martin Löb sharpened that in 1955: if such a system proves "if this is provable, it's true," it already proves the thing itself. So can a box vouch for itself, and does Löb stop it?

You might guess yes, since a box looks like a system talking about itself. It lands somewhere else, and mostly it's a relief. Hold two things side by side: "there's a receipt for this somewhere," and the receipt in your hand. Löb's theorem is about provability, the bare claim that some proof exists somewhere. A raised box isn't the receipt in your hand either. From a clean start, chapter eight says a check passed before the raise. The box doesn't say which claim, and it doesn't hold a proof of it. Sergei Artemov's line from chapter eight, between "there's a proof" and "this is the proof," is useful here, but a box doesn't turn one into the other. The trust in this book comes from a proof about the checker, the clean start and every move, made outside the machine. It isn't a proof inside the machine that the machine is sound, so it doesn't get around Gödel or Löb.

Push on that and it gives, because a receipt is only as good as whoever wrote it. Its trust rests on the checker meaning what it says, and that gets settled once, from outside. That's where I chose to stop this machine's chain of reasons. Eliezer

Yudkowsky and Marcello Herreshoff's 2013 draft on programs that trust their own rewrites describes a related problem they call the Löbian obstacle: how a program can justify trusting its successor's proofs. It's a cousin of this question, it doesn't settle it, and it doesn't make my design the only possible one. So I didn't get rid of the receipt somewhere. I moved it out of the machine and handed it to whoever reads the record.

One more thing follows from the same picture, and it's short enough to check by hand. Can a box rest on itself, the way the sentence "this sentence is certified" would like to? Not if it's established and permanent. The check behind the raise happened while the box was still down, because the box goes up only after that check passes. And once up it never comes down, so it goes up only once. So no claim whose truth needs the box already up can be the claim the box went up on. Saul Kripke's 1975 theory of truth calls a sentence that only ever leans on itself ungrounded, and established boxes are grounded in his sense: every one bottoms out in a check that didn't lean on it. Caspar Oesterheld's 2019 work on programs that cooperate by simulating each other needed a grounding step of the same kind to stop an endless regress.

16.6 Two walls

One last distinction, because the two are easy to run together, and running them together is the easiest way to misread this book.

There's a second wall, nothing to do with the diagonal. It's chapter seven's: two states that differ in their box can give the same view through a window that doesn't let the box through, and then no reader of the window can get the box back. That wall comes from missing information. Nothing about it is undecidable. It holds for small finite things with nothing to compute. Rice isn't underneath it.

So there are two walls, on two different axes. The diagonal is about what no inside procedure can decide. The window is about what no outside watcher can see. The book needs both, and I'll try not to call one by the other's name.

CHAPTER 17

Five real systems

So far there's been a toll, a counter, two kinds of wall and a cup, every one of them shown on paper, with cups and cards and dice I made up. It's time to go look at what people actually do. I picked five systems that keep records other people rely on, with real money or real security behind them, and asked each one the question from the paper: is there a price on the moment the record is written? Not on the work around it. On the write itself, the moment the mark goes on and stays.

I'll give you the answer first, because it's one sentence and it surprised me. Of the five, read through the pieces of them I copied out, one takes a fee at the moment of the write, and that one leaks at exactly the place chapter eleven said a counter leaks: where it goes back down.

"Price" needs one meaning for all five, or the answer is whatever I want it to be. So here it is, fixed before looking. A system prices an event if it attaches one of three costs to it. A fee: something taken at the event itself, whatever happens afterwards. A bond: something put at risk by the event and lost only if some later event happens, like a contradiction. A budget: a limited allowance the event draws down, so only so many such events can ever happen. A wall isn't a price under this. Nobody pays a scrambling function; undoing one is just too hard. Neither is work a system needs and never charges for. The table at the end has a column for each of the three, and because the definition came first, what the table shows is a finding about the five, not something I arranged.

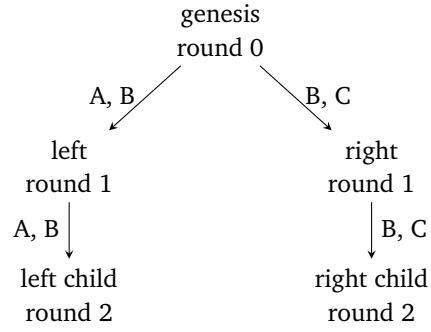
Three things are easy to blur here, so I'll keep them apart all the way through. There's the real system: the document that real money and real security run on. There's my copy: one piece of that document, copied out small enough to check by hand. And for some of the five there's a wrapper: a tiny model that takes one condition as given and says what follows. A claim checked at one level isn't a claim about the level above it. Near the end of the gas section I'll walk gas through all three, so you can see where each claim stops.

17.1 Proof-of-stake finality: the price is on going back

Ethereum, and chains like it, decide which blocks are final by having validators, people who've locked up money, their stake, vote on checkpoints: blocks picked out at regular numbered rounds, which they call epochs. A vote isn't for one checkpoint. It's a link from a checkpoint the validator already trusts to a later one, and every chain starts from one first block, the genesis block, which counts as trusted from the start. In the design Vitalik Buterin and Virgil Griffith published in 2017, called Casper, a checkpoint is justified when validators holding two thirds of the stake vote a link to it from a justified one, and final when it's justified and a two-thirds link runs from it to its direct child. The record is "this checkpoint is final."

Two rules carry the price. Don't vote for two different targets in the same round. And don't cast a vote whose span, from where it starts to where it lands, surrounds the span of another of your votes. Break either, and your deposit can be destroyed, which is called slashing. The theorem they prove, which they call accountable safety, says: if two checkpoints that contradict each other ever both become final, at least a third of the stake broke a rule and can be slashed. So the price is on going back, on making the record contradict itself. Writing the record, voting honestly to finalize, costs no stake at all. That isn't a claim that running a validator is free; it's a claim about what the deposit is taken for. In the terms above, Casper's price is a bond. The deposit stands behind every vote, the honest ones included, and it's only taken from a validator whose own votes contradict each other.

Do it with three validators of equal stake. Draw the genesis block at round 0, two competing blocks at round 1, and one child of each at round 2. Let A and B vote both links on the left branch, genesis to its first block and that block to the next. Let B and C vote both links on the right.



Each link has two of the three votes, two thirds of the stake. Both round-1 blocks are justified and each has a justified link to its child, so both are final, and they're siblings, so neither is an ancestor of the other. B voted for two different targets in round 1, so B's third of the stake can be slashed. A and C each voted one link landing in round 1 and one landing in round 2, which neither doubles nor surrounds, so

they broke no rule. The picture records votes; nothing in it takes anybody's money. What it shows is that the theorem's condition can actually happen: forks like that can be built, and the theorem fires on them.

Why does the theorem hold? Walk it. Say two contradicting checkpoints are both final, one in round v and the other in round w.

If v and w are the same round, then two groups, each with two thirds of the stake, voted links landing in that round on two different blocks. Two thirds plus two thirds is four thirds, more than everything, so the two groups share at least a third. Every validator in that shared part voted two different targets in one round.

If they're different rounds, say v is earlier, look at how the later one got justified: a chain of two-thirds links climbing from genesis, every block on it an ancestor of the later checkpoint. None of those blocks is the earlier final checkpoint or comes after it, because then the later checkpoint would too, and they contradict. So that chain has to get past round v without touching the earlier checkpoint, and some link in it starts at or before v and lands after v. Three ways that can go. It lands at exactly v plus one: then it and the earlier checkpoint's finalizing link land in the same round on different blocks, and the shared third voted both, a double vote. It starts at exactly v: then its starting block is a justified block at round v that isn't the earlier checkpoint, and the two links that justified those two blocks land in round v on different blocks, a double vote again. Otherwise it starts before v and lands after v plus one, so it surrounds the earlier checkpoint's finalizing link, which runs from v to v plus one, and the shared third made a surround vote. In every case a third of the stake broke a rule.

Everything there stands on the overlap. Take away "two groups of two thirds share a third," and the argument promises nothing.

The piece I copied out comes from a published proof by Runtime Verification and the University of Texas at Austin, based on Yoichi Hirai's, with that overlap taken as a condition: the copy takes it as given and doesn't do the arithmetic. It has no deposits, no rewards, and no rule for picking which branch to build on. On a chain with one validator, one vote finalizes the genesis block and nobody is slashed: that's "writing costs no stake," shown in the simplest case there is.

One thing that means for the record itself: finality can be undone at a price, the price is named, and a final block can still hold a false statement.

Is the 2017 design what Ethereum actually runs? Partly, and the paper's title is the clue: Casper the Friendly Finality Gadget. Its first sentence says it's a finality system laid over a chain still choosing its blocks by proof of work, so it's a gadget, bolted on top. In 2020 a larger group, Buterin among them, wrote up Gasper, joining Casper's finality to a rule for picking branches called LMD GHOST, and on 15 September 2022, in the switch people called the Merge, Ethereum stopped choosing blocks by proof of work at all and went over to proof of stake. So the checkpoints

and the two slashing rules are one part of a system that holds real money today, and my copy is of that part's safety argument, the way the 2017 design states it. Everything between the two, the branch rule and the rest of the running protocol, is the gap between my copy and the running system. None of it moves the price: finalizing still costs an honest validator no stake, and what costs is contradicting yourself.

17.2 Gas: the one that charges a fee at the write

Every operation on Ethereum costs gas, paid in ether by whoever sent the transaction. The yellow paper, Ethereum's formal specification, gives the reason in one line: every computation costs a fee, so the network can't be abused and questions about whether a program stops are sidestepped. The record I care about is a storage slot holding something other than zero, which every full copy of the chain in the world has to keep until it's cleared. Clearing a slot is allowed, and the refunds below exist to reward it.

Writing a fresh slot from zero to something costs twenty thousand gas, by the table in the yellow paper's appendix, plus twenty-one hundred for touching a cold slot, one the transaction hasn't touched yet. After the first touch it's warm. The point, as I read it, is to discourage storage growth, since storage grows every copy's database. That charge is a fee on the write, at the moment of the write, and it's the only fee in the five. Gas has a budget too: every block has a gas limit, and the gas its transactions use can't go over it, so only so many writes fit in a block.

Where does a number like twenty-one hundred come from? Somebody picked it, so the next question is who, and why, and the answer has a date on it. In September 2016, just as Ethereum's developers were gathering in Shanghai for Devcon 2, their developer conference (the third one, since they started counting at zero), attackers started filling blocks with an operation that was cheap in gas and expensive in fact: it sent every copy of the chain off to read data from its disk, and copies fell behind. The first repair, a change that October called Tangerine Whistle, raised the prices of the operations the attack leaned on. The cold-slot charge came later, in a proposal by Vitalik Buterin and Martin Swende called EIP-2929, switched on in the Berlin upgrade of April 2021, and the reason it gives is still those 2016 attacks. So twenty-one hundred isn't a law of nature or a measurement of anything deep. It's a scar: a price set after somebody found where the old price was wrong. That doesn't change the argument; the write is still priced at the moment of the write. It does change how I read the rest of that table. Some of its numbers, at least, are somebody's answer to "what did this cost us last time," which is a record of a different kind.

Now the leak, which is chapter eleven's leak. Set a slot and clear it again in the same transaction, and you get a refund. Count it through. The first write costs twenty-two thousand one hundred. Clearing it costs a hundred and credits a refund of nineteen thousand nine hundred. Net: twenty-three hundred. Leave it set, and you pay at least twenty thousand. So the write is priced, and the write you take back nearly isn't.

Twenty-three hundred on its own misleads, though, because of the 2021 change from chapter eleven, the one that ended the refund-hoarding token's advantage: the refund a transaction actually gets is capped at a fifth of all the gas it used, and that includes the twenty-one thousand every transaction pays just for existing. Take a transaction that does nothing but set and clear the slot. Count only that base and the two stores (the steps that load their arguments add about a dozen more): it uses 43,200 gas, so the cap is 8,640, so it pays 34,560. That's 13,560 on top of the base. Look at the slot alone and forget the base, which the rule doesn't do, and you'd get 17,760. Either way it's far above twenty-three hundred.

You can check the twenty-thousand floor with a pencil. My copy follows the specification's rule for one slot in one transaction, starting from zero: a store of something that isn't zero into a slot that's still zero costs twenty thousand; any other store costs a hundred; and putting the slot back to zero gives back nineteen thousand nine hundred. Keep two lines on a sheet. "Slot at zero: net so far at least zero." "Slot not at zero: net so far at least twenty thousand." The fresh slot meets the first line. Now check that every store keeps whichever line fits where it lands. Zero to zero adds a hundred: still at least zero. Zero to something adds twenty thousand: from at least zero to at least twenty thousand. Something to something else adds a hundred: still at least twenty thousand. Something back to zero adds a hundred and takes off nineteen thousand nine hundred, nineteen thousand eight hundred off in all: from at least twenty thousand down to at least two hundred, still at least zero. So whatever the list of stores, if the slot ends up holding something, the net is at least twenty thousand. The cold charge comes once, on the first store, and nothing after it depends on it, so from a cold slot it's at least twenty-two thousand one hundred.

That copy covers one slot in one transaction, with the gas and refund arithmetic copied out of Ethereum's execution specification. It has no other slots, no calls between programs, and no refund cap.

The wrapper is a different kind of thing. It's a gas schedule in the abstract: any states, any steps, a cost on each step, a box that's up or down, and a rule saying which steps get charged. There's no Ethereum in it at all. It's chapter seven's clerk under a new name. Ask a schedule for two things: every run pays at least as much as the number of times it turned the box up, and no run pays more than that. Both together hold exactly when the schedule charges one on each step that turns the box

up and nothing on any other. Both halves are easy to see on a one-step run. Leave one step that turns the box up uncharged, and that one-step run turns it up for free: the first fails. Charge one step that doesn't turn it up, and that one-step run pays more than the nothing it turned up: the second fails.

So walk gas through all three. Think of Ethereum as it runs as a kitchen, and my copy as a photocopy of one recipe from it. The photocopy says a set-then-clear nets twenty-three hundred, and inside the photocopy that's right. In the kitchen, the bill for that transaction is 34,560, because the kitchen has a base charge and a cap the photocopy doesn't. The wrapper isn't from that kitchen at all: it says what an exact price would have to look like, one per raise, and Ethereum's twenty thousand isn't that and isn't trying to be. A claim made with the photocopy in hand is a claim about the photocopy.

Gas prices storage and computation. It says nothing about whether what's stored is true.

17.3 Attestation: a quote about the past

A lot of computers carry a small security chip, a trusted platform module. It keeps registers that can only be extended: each new measurement gets scrambled together with the old value. A quote is the chip signing some of those registers along with a challenge from whoever's asking, so they can tell it's fresh, and the asker still has to check the signature, the challenge and the values they expected. That can give you a fresh report of what was measured. It doesn't tell you every important fact about the machine got measured.

Some of those registers can be reset, depending on the register and the rules of the platform. Some reset at startup; some allow resets partway through. So "you can only extend" describes a stretch of time with those resets left out, and a claim about an unbroken history has to say which resets it allows. Intel's SGX enclaves use their own measurements and reports for a similar purpose, but not these registers.

What does the specification price on the registers? Nothing: no fee, no bond, no budget. Forging a different measurement history that passes depends on breaking the scrambling or the trusted parts, which is a wall, not a price. An allowed reset is a separate thing, not an attack. That's the difference between expensive and impossible in practice, and it's easy to blur, which is why the definition at the start of this chapter leaves walls out.

So what stands behind a cost can come in three currencies. It can be money, a fee or a deposit the system takes: Casper and gas are there. It can be a wall, going back made too hard by something nobody knows how to undo, with no exact price on it: attestation and transparency are there. Or it can be physical, energy spent or

hardware worn, which none of these five charges as a fee or a bond; the nearest is the chip's write budget, below.

Why the registers, then, when something that lasts would make a better record? Because the quote is what a faraway asker reads, and the registers are what it signs. The same chip has longer-lasting records, closer to the cup than anything else in this chapter. Version 2.0 of its specification has slots of a kind it calls a bit field, changed only by a command that ORs new bits into the old value: while that slot exists, it can set a bit and can't clear one. Deleting a slot and making it again is governed by separate permissions, and any claim about how long the record lasts has to include them. The chip also has counters that only go up, and a new counter starts above every count the chip has held before, so deleting one and making a new one doesn't start over at zero. That's a promise about this chip, not about somebody swapping in another chip.

Those slots are rationed. Bryan Parno and his coauthors, building a system called Memoir in 2011, noted that the specification of the day only required a counter to be able to tick once every five seconds, and that the chip's storage was expected to survive about a hundred thousand writes in its whole life, so writing once a second would wear it out in under twenty-eight hours. Memoir writes to it twice per startup. That's an expected lifetime, not a budget the protocol enforces, and the quoted registers have no number like it at all. But on the bit field it's chapter five's row of cups, which can only turn up as many times as there are cups. Phones and small controllers go one floor further down, to fuses like Chow's in chapter fourteen: a version number kept as a row of fuses, one more blown at each security update, and startup code that refuses anything older. A 2021 guide by Rob Smart for PSA Certified, a security certification scheme for chips and devices, tells designers to make sure there are enough fuses to last the number of updates expected over the device's life. That's chapter five's count of cups, written as a design rule.

What it's all for is rollback. Memoir's example is a password checker put back to an old saved state to undo a user's password change. In 2017 a system called ROTE did the same job for Intel's enclaves by having several machines vouch for each other's counters, and its authors say the only way left to break it is to reset every one of them to its start. In 2016 Raoul Strackx and Frank Piessens's Ariadne got the job down to flipping one bit of lasting memory per update, which they call the lowest possible. Notice who the attacker is in every one of these: whoever gets to put an old state back. Not a bad program. A record kept by the step can't see that attacker. Everything an established record guarantees about its history is stated from a clean start. Put back a state the machine really reached, and every guarantee still holds of it, through the run that reached it first. None of them can say nothing came after it. The toll on each step still holds after a restore; it just can't tell you what happened before.

Systems run as several copies put the same kind of record somewhere else: inside each copy, where a copy that's been taken over still can't make it say two different things. Byung-Gon Chun and his coauthors built a log a copy can only add to, called A2M, in 2007. Dave Levin and his coauthors shrank that to a trusted counter, TrInc, in 2009. MinBFT, from Giuliana Veronese and her coauthors in 2013, uses a trusted counter to agree with 2f + 1 copies when f of them can lie, where the usual count is 3f + 1. Damysus, from Jérémie Decouchant, David Kozhaya, Vincent Rahli and Jiangshan Yu in 2022, has a small trusted service it calls the Checker, which keeps a count of where the protocol is that only goes up. And Allen Clement, Flavio Junqueira, Aniket Kate and Rodrigo Rodrigues worked out in 2012, with proofs, what such a record buys, and as I read their result it's a good warning for this book. On its own, stopping a copy from saying two different things doesn't lower the 3f + 1 at all when messages can be slow. Add signatures a third party can check, and it turns a system that survives crashes into one that survives liars, with no extra copies. The way I read it, a record helps the others only as far as they can check what it says. That's chapter seven from the other side.

Under those three currencies there's a fourth, and every permanent record on a finite chip pays in it: the chairs from chapter two. Here's how. Forgetting by itself doesn't pick out a record write; any step that overwrites a field that could hold two things forgets something. What picks it out is where the forgetting lands. Count the states where the record already reads yes, and call that number K. Take a step that never turns the record off anywhere, and turns it on at one state that read no. That step sends each of the K yes-states to a yes-state, and it sends the one no-state to a yes-state too. That's K plus one starting places and only K yes-states to land on, so two of them land together. Both came from the yes side or from the one it just raised. So what a permanent write destroys is a distinction on the record's own yes side: two states it already vouched for, or the one it just raised and one that was up already. Any overwrite merges something somewhere. A permanent write has to merge it there. That needs no assumption about money, keys or heat. The other three each need an assumption of their own: a fee rule, a wall nobody can climb, a law of physics.

What the quote can do is say what was measured. It can't say the measured code is correct, and it can't say anything about what was never folded in. A correctly measured buggy program gets a beautiful quote. Try it with the smallest pair there is: two machines with nothing measured at all, one with some running flag up and one with it down. Their registers are the same, so their quotes are the same, and anything that reads only the quote has to give them the same answer. No reader of the quote can recover the flag. That's chapter seven's window again. Anything that depends only on what was measured, the quote can answer; that's what it's for. And a "sign and check" interface by itself promises nothing: a check that always says yes

fits the interface perfectly and accepts a signature no key ever made. The protection has to come from the scrambling and the keys, which is a further assumption.

That limit, turned outward, is the honest edge of chapter eight's four conditions. Every one of them is about a thing the machine holds: same property, same thing, same edition, honest check. For a claim about the world, the oven is hot, the parcel arrived, there's no edition the machine controls. Whatever reads the world is a window onto it, and "nothing changed between the check and the commit" stops being something a step can enforce and becomes an assumption about the sensor. So a record can establish claims about the machine's own state, and claims about the world only through stated assumptions about whatever measured them. A correctly recorded sensor reading establishes what the sensor reported. Whether the oven was hot still depends on the sensor and on how its report connects to the world.

17.4 Certificate transparency: visible, unpriced

The certificates that put the padlock in your browser are issued by hundreds of authorities, and a bad or broken one can issue a certificate for a site it shouldn't. Certificate Transparency puts certificates in public logs you can only add to, built as trees of scrambled values. It was written up as RFC 6962 in 2013 and revised as RFC 9162 in 2021. I went looking for which one the running logs follow, and it's still 6962: Mozilla's reference for web developers says browsers are built to it, and as late as November 2025 Chrome's team was only proposing to drop its rule that every certificate carry a timestamp from at least one 6962 log. My copy is of 9162.

The tree is chapter four's chained running total, folded into a tree: Ralph Merkle's tree from the same chapter. Scramble each certificate; those are the leaves. Scramble the leaves together in pairs, then those in pairs, and so on, up to one value at the top, the root. To prove one leaf is in, the log hands you only the few values sitting beside the climb from that leaf to the root, and you redo the climb yourself. A second check shows that a bigger tree is a smaller one with leaves added on the end and nothing rewritten. The record is a leaf in the tree.

That kind of tree has a history of its own. The computer scientists Scott Crosby and Dan Wallach, at Rice University, built logs you can't quietly change on the same kind of tree in 2009, with a proof that an entry is in the log and a proof that a later version extends an earlier one, each only a handful of values long even for a huge log. A rewrite that doesn't extend a root somebody already saw gets caught by the second check.

Do the climb by hand. Lay out seven cards, d0 through d6. Join neighbours with paper brackets: a joins d0 and d1, b joins d2 and d3, c joins d4 and d5. d6 stands alone. Then L joins a and b, and R joins c and d6. The root joins L and R.

Step	Card handed over	What you rebuild
Start	d_3 itself	d_3
1	d_2, on the left	b, joining d_2 and d_3
2	a, on the left	L, joining a and b
3	R, on the right	the root, joining L and R

To check d3, you only need three cards: d2, a and R. Join d2 on the left of d3 and you've rebuilt b. Join a on the left of that and you've rebuilt L. Join R on the right and you've rebuilt the root. You never opened a or R. Swap left and right anywhere, though, and the bracket changes: the proof has to carry the order as well as the cards. The same seven cards check every other leaf too. d0 needs d1, b and R, all on the right. d4 needs d5 and d6 on the right and then L on the left. d6, standing alone, needs only c and then L, both on the left.

Now the second check. Say you saw the tree when it had four cards, and its root was L. Now it has seven. The log hands you one card, R. Put L on the left of R and you get the new root. So the old tree is the left half of the new one, untouched, and only cards were added on the end.

My brackets are brackets, not numbers, so what you just checked is how the tree is put together. A real log uses short scrambled values, and "two different trees can't give the same root" is then a separate promise, from the wall.

What does the RFC price? Nothing. It says in its first section that logs let people detect a bad certificate afterwards. They don't prevent one. And a log that shows different trees to different people can beat the checking, so logs are still trusted parties. What it buys is visibility.

So why does any authority bother logging? Nothing in the RFC makes it. A browser does. Chrome's team set the rule, enforced from Chrome 68 in July 2018, that every certificate issued after 30 April 2018 had to meet its transparency policy, and a publicly trusted certificate that doesn't simply isn't trusted by Chrome: the visitor gets a full-page warning in front of the site. So there is a price after all, though it isn't in the specification. It lives in one company's browser, and it's the price of being shown to that browser's users at all. Which means the strongest lever on this record is a policy one company can change, and that's a fact about who holds the stamp, not about the tree. It changes nothing in the table, which asks what each specification prices. But keep two sentences apart: "the specification prices nothing" and "nothing is priced."

17.5 Proof-carrying code: no toll required

George Necula's idea, from chapter twelve, is to send untrusted code with a proof that it follows the receiving computer's rules; the receiving computer works out what would have to be true and checks the proof with a small checker before running anything. The record is the checked proof, or the result of checking it,

and a computer may keep either. Nothing in the idea requires a permanent box, or charges anything for accepting. Producing and checking the proof both take work, and nothing here makes that work the toll.

17.6 The same question, five ways

Lay them out by the one definition of price from the start. Casper puts a bond behind the write and takes it on going back. Gas takes a fee at the write, mostly refunds it if you take the write back, up to a cap, and rations writes with a gas limit per block. The chip's quoted registers have no fee and no budget; its separate bit fields have only so many bits to set, which isn't a quota on everything the chip does. Transparency charges nothing at the write; what it gives is that a bad certificate can be seen afterwards. Proof-carrying code requires neither a toll nor keeping anything.

System	Fee at the write	Bond	Budget	What my copy shows
Casper finality	none	the deposit, taken on a contradicting vote	none	two contradicting final checkpoints mean a third of the stake can be slashed
Gas	20000 for a fresh slot, mostly refunded if taken back, up to a cap	none	the gas limit of each block	a write left standing in an empty slot nets at least 20000
Attestation	none on the quoted registers	none	none on those registers	the quote can't tell you what it never measured
Transparency	none; a bad certificate can be seen	none	none	the RFC's own examples pass
Proof-carrying code	none required	none	none	a comparison of ideas; no checker is built here

The first four rows come with a piece copied out and checked; the fifth is a comparison of ideas. Each copy shows only what it shows. To read a row as a claim about the running system you'd need evidence from that system.

So the thing the toll charges for, the moment the record is written, gets a fee in one of the five, and in that one the designers had to patch the place where the charge could be clawed back. I don't find that a win or a loss. It's a fact about what people built, and what they built was mostly walls and penalties for contradicting yourself. A toll on the write was rare. Whether a toll is a better design than a wall is something I can't prove, and I won't pretend the five say so. Because the definition came first, that last part is a count: of the fifteen fee, bond and budget squares in the table, three are filled, one of each kind, and only gas takes a payment at the moment of the write.

What I read in them, and it's a reading of five cases I chose, is that each one has something it treats as the record, they last for different lengths of time, and they all picked something different: final, committed, measured, logged, checked. Is there anything that picks out one event as the one worth recording, or is it a free choice every time? That's the next chapter.